Exam : **AIP-C01**

Title : AWS Certified Generative AI Developer - Professional

Vendor : Amazon Web Services

Version : V12.35

NO.1 A GenAI developer is building a Retrieval Augmented Generation (RAG)-based customer support application that uses Amazon Bedrock foundation models (FMs). The application needs to process 50 GB of historical customer conversations that are stored in an Amazon S3 bucket as JSON files. The application must use the processed data as its retrieval corpus. The application's data processing workflow must extract relevant data from customer support documents, remove customer personally identifiable information (PII), and generate embeddings for vector storage. The processing workflow must be cost-effective and must finish within 4 hours.
Which solution will meet these requirements with the LEAST operational overhead?

A. Use AWS Lambda and Amazon Comprehend to process files in parallel, remove PII, and call Amazon Bedrock APIs to generate vectors. Configure Lambda concurrency limits and memory settings to optimize throughput.

B. Create an AWS Glue ETL job to run PII detection scripts on the data. Use Amazon SageMaker Processing to run the HuggingFaceProcessor to generate embeddings by using a pre-trained model. Store the embeddings in Amazon OpenSearch Service.

C. Deploy an Amazon EMR cluster that runs Apache Spark with user-defined functions (UDFs) that call Amazon Comprehend to detect PII. Use Amazon Bedrock APIs to generate vectors. Store outputs in Amazon Aurora PostgreSQL with the pgvector extension.

D. Implement a data processing pipeline that uses AWS Step Functions to orchestrate a workload that uses Amazon Comprehend to detect PII and Amazon Bedrock to generate embeddings. Directly integrate the workflow with Amazon OpenSearch Serverless to store vectors and provide similarity search capabilities.

Answer: D

Explanation:
Comprehensive and Detailed 250 to 350 words of Explanation From AWS Generative AI concepts and services documents:
Option D is the best solution because it delivers a fully managed, scalable pipeline with minimal infrastructure management while meeting the 50 GB and 4-hour constraint. AWS Step Functions provides a serverless orchestration layer that can coordinate parallel processing steps, retries, and error handling without managing clusters or tuning long-running compute.
Using Amazon Comprehend for PII detection fulfills the requirement to remove customer PII in a managed and consistent way. Step Functions can coordinate Comprehend calls at scale and route sanitized outputs into the embedding step. Generating embeddings with Amazon Bedrock keeps the entire workflow within AWS managed services, eliminates the need to maintain custom embedding models, and supports consistent vector representations for downstream retrieval.
Direct integration with Amazon OpenSearch Serverless provides a low-operations vector store that can handle large-scale indexing and similarity search without cluster sizing, node maintenance, or shard management.
This aligns strongly with the requirement for least operational overhead and supports growth beyond the initial 50 GB corpus. Step Functions can batch and parallelize ingestion into OpenSearch Serverless to meet the 4-hour completion goal in a cost-effective manner by controlling concurrency, chunk sizes, and failure handling.
Option A can be difficult and costly at this scale because Lambda concurrency and per-invocation overhead can become complex to tune for 50 GB within 4 hours. Option B introduces SageMaker Processing and embedding model management, increasing operational complexity. Option C requires EMR cluster management and tuning, which is the opposite of minimal overhead.

Therefore, Option D is the most operationally efficient, scalable, and managed approach to build the required PII-sanitized embedding pipeline for a RAG corpus.

NO.2 A pharmaceutical company is developing a Retrieval Augmented Generation application that uses an Amazon Bedrock knowledge base. The knowledge base uses Amazon OpenSearch Service as a data source for more than 25 million scientific papers. Users report that the application produces inconsistent answers that cite irrelevant sections of papers when queries span methodology, results, and discussion sections of the papers.
The company needs to improve the knowledge base to preserve semantic context across related paragraphs on the scale of the entire corpus of data.
Which solution will meet these requirements?

A. Configure the knowledge base to use fixed-size chunking. Set a 300-token maximum chunk size and a
10% overlap between chunks. Use an appropriate Amazon Bedrock embedding model.

B. Configure the knowledge base to use hierarchical chunking. Use parent chunks that contain 1,000 tokens and child chunks that contain 200 tokens. Set a 50-token overlap between chunks.

C. Configure the knowledge base to use semantic chunking. Use a buffer size of 1 and a breakpoint percentile threshold of 85% to determine chunk boundaries based on content meaning.

D. Configure the knowledge base not to use chunking. Manually split each document into separate files before ingestion. Apply post-processing reranking during retrieval.

Answer: B

Explanation:
Option B is the best fit because hierarchical chunking is designed to preserve local detail while keeping broader document context available during retrieval, which directly addresses the problem of questions spanning methodology, results, and discussion. In large scientific papers, a single answer often depends on linked paragraphs across adjacent sections. If the knowledge base retrieves only small, isolated chunks, the RAG system can cite text that is semantically close to a query term but not contextually correct, producing inconsistent answers and irrelevant citations.
With hierarchical chunking, the knowledge base creates child chunks that are small enough for high-precision vector similarity matching, such as 200 tokens, which improves the likelihood that the retrieved text is tightly related to the user's query. At the same time, each child chunk is associated with a larger parent chunk, such as 1,000 tokens, which retains the surrounding narrative and section-level context. This structure helps the retrieval pipeline return passages that include the relevant subsection plus the explanatory framing that prevents misinterpretation, which is especially important in scientific writing where methods, results, and discussion are interdependent.
The configured overlap further reduces boundary effects where key statements split across chunks. This improves continuity for paragraphs that bridge sections, such as a results paragraph that references the methodological setup or a discussion paragraph interpreting a specific metric.
Option A can improve consistency slightly, but fixed-size chunking still risks separating related paragraphs and does not provide a built-in mechanism to retrieve broader context linked to precise matches. Option C can create more meaningful boundaries, but it does not guarantee the parent-level context that hierarchical chunking provides at retrieval time. Option D increases operational burden and is not practical at the scale of
25 million

NO.3 A company uses an organization in AWS Organizations with all features enabled to manage

multiple AWS accounts. Employees use Amazon Bedrock across multiple accounts. The company must prevent specific topics and proprietary information from being included in prompts to Amazon Bedrock models. The company must ensure that employees can use only approved Amazon Bedrock models. The company wants to manage these controls centrally.
Which combination of solutions will meet these requirements? (Select TWO.)

A. Create an IAM permissions boundary for each employee's IAM role. Configure the permissions boundary to require an approved Amazon Bedrock guardrail identifier to invoke Amazon Bedrock models. Create an SCP that allows employees to use only approved models.

B. Create an SCP that allows employees to use only approved models. Configure the SCP to require employees to specify a guardrail identifier in calls to invoke an approved model.

C. Create an SCP that prevents an employee from invoking a model if a centrally deployed guardrail identifier is not specified in a call to the model. Create a permissions boundary on each employee's IAM role that allows each employee to invoke only approved models.

D. Use AWS CloudFormation to create a custom Amazon Bedrock guardrail that has a block filtering policy. Use stack sets to deploy the guardrail to each account in the organization.

E. Use AWS CloudFormation to create a custom Amazon Bedrock guardrail that has a mask filtering policy. Use stack sets to deploy the guardrail to each account in the organization.

Answer: C D

Explanation:
The correct combination is C and D because together they enforce centralized governance over both model access and prompt content controls, which are the two core requirements of the scenario.
To ensure employees can use only approved Amazon Bedrock models, governance must be enforced at the organization level and not rely on individual application logic. Service Control Policies (SCPs) are the strongest control mechanism available in AWS Organizations because they define the maximum permissions an account or principal can have. In option C, the SCP prevents any Amazon Bedrock model invocation unless a centrally approved guardrail identifier is specified. This ensures that guardrails are always enforced, regardless of how or where the invocation originates. The additional use of IAM permissions boundaries ensures that even within allowed accounts, employees are restricted to invoking only explicitly approved foundation models.
To prevent specific topics and proprietary information from being included in prompts, Amazon Bedrock Guardrails must be used. Guardrails operate inline during model invocation and can block disallowed content before it is processed by the model. Option D correctly specifies a block filtering policy, which is appropriate when content must be prevented entirely rather than partially redacted. Deploying the guardrail using AWS CloudFormation StackSets allows the company to centrally manage and consistently deploy the same guardrail configuration across all accounts in the organization, ensuring uniform enforcement.
Option E uses mask filtering, which is better suited for redacting sensitive output rather than preventing prohibited content from being submitted in prompts. Option B attempts to use SCPs alone but does not enforce guardrail deployment or content filtering. Option A incorrectly places guardrail enforcement in permissions boundaries, which are not designed to validate request parameters such as guardrail identifiers.
By combining SCP-based enforcement with centrally deployed Bedrock guardrails, options C and D together provide strong, scalable, and centrally managed controls for both content safety and model governance across the organization.

NO.4 A company has a recommendation system. The system's applications run on Amazon EC2 instances. The applications make API calls to Amazon Bedrock foundation models (FMs) to analyze customer behavior and generate personalized product recommendations.
The system is experiencing intermittent issues. Some recommendations do not match customer preferences.
The company needs an observability solution to monitor operational metrics and detect patterns of operational performance degradation compared to established baselines. The solution must also generate alerts with correlation data within 10 minutes when FM behavior deviates from expected patterns.
Which solution will meet these requirements?

A. Configure Amazon CloudWatch Container Insights for the application infrastructure. Set up CloudWatch alarms for latency thresholds. Add custom metrics for token counts by using the CloudWatch embedded metric format. Create CloudWatch dashboards to visualize the data.

B. Implement AWS X-Ray to trace requests through the application components. Enable CloudWatch Logs Insights for error pattern detection. Set up AWS CloudTrail to monitor all API calls to Amazon Bedrock. Create custom dashboards in Amazon QuickSight.

C. Enable Amazon CloudWatch Application Insights for the application resources. Create custom metrics for recommendation quality, token usage, and response latency by using the CloudWatch embedded metric format with dimensions for request types and user segments. Configure CloudWatch anomaly detection on the model metrics. Establish log pattern analysis by using CloudWatch Logs Insights.

D. Use Amazon OpenSearch Service with the Observability plugin. Ingest model metrics and logs by using Amazon Kinesis. Create custom Piped Processing Language (PPL) queries to analyze model behavior patterns. Establish operational dashboards to visualize anomalies in real time.

Answer: C

Explanation:
Option C best satisfies the requirements because it combines application-aware observability, metric baselining, anomaly detection, and correlated alerting using fully managed AWS services with minimal operational overhead. Amazon CloudWatch Application Insights is designed to automatically monitor application health by analyzing metrics, logs, and events across EC2-based workloads. This aligns directly with the need to detect intermittent performance issues and deviations from expected behavior.
By publishing custom metrics using the CloudWatch embedded metric format, the application can track generative AI-specific signals such as recommendation quality indicators, token usage, request volume, and response latency from Amazon Bedrock foundation model calls. Adding dimensions such as request type or user segment enables fine-grained visibility into which workloads or customer groups are impacted when recommendation quality degrades.
A critical requirement is detecting degradation compared to established baselines and generating alerts within
10 minutes. CloudWatch anomaly detection automatically builds statistical models of normal behavior for time-series metrics and flags deviations without requiring manually tuned thresholds. This capability is well suited for monitoring foundation model behavior, which can vary subtly over time. When anomalies are detected, CloudWatch alarms can trigger notifications with contextual metric data quickly, meeting the alerting requirement.
CloudWatch Logs Insights complements the metric-based view by enabling log pattern analysis and

correlation. Engineers can query application logs and model response logs to identify recurring error patterns or shifts in output behavior that explain why recommendations no longer align with user preferences.
Application Insights further correlates metrics and logs to surface probable root causes, reducing mean time to resolution.
The other options lack one or more critical elements. Option A focuses on infrastructure-level metrics without baseline anomaly detection. Option B emphasizes tracing and auditing but does not provide automated performance deviation analysis. Option D offers flexibility but requires significantly more development and operational effort than a native CloudWatch-based solution.

NO.5 A company runs a generative AI (GenAI)-powered summarization application in an application AWS account that uses Amazon Bedrock. The application architecture includes an Amazon API Gateway REST API that forwards requests to AWS Lambda functions that are attached to private VPC subnets. The application summarizes sensitive customer records that the company stores in a governed data lake in a centralized data storage account. The company has enabled Amazon S3, Amazon Athena, and AWS Glue in the data storage account.
The company must ensure that calls that the application makes to Amazon Bedrock use only private connectivity between the company's application VPC and Amazon Bedrock. The company's data lake must provide fine-grained column-level access across the company's AWS accounts.
Which solution will meet these requirements?

A. In the application account, create interface VPC endpoints for Amazon Bedrock runtimes. Run Lambda functions in private subnets. Use IAM conditions on inference and data-plane policies to allow calls only to approved endpoints and roles. In the data storage account, use AWS Lake Formation LF-tag- based access control to create table-level and column-level cross-account grants.

B. Run Lambda functions in private subnets. Configure a NAT gateway to provide access to Amazon Bedrock and the data lake. Use S3 bucket policies and ACLs to manage permissions. Export AWS CloudTrail logs to Amazon S3 to perform weekly reviews.

C. Create a gateway endpoint only for Amazon S3 in the application account. Invoke Amazon Bedrock through public endpoints. Use database-level grants in AWS Lake Formation to manage data access. Stream AWS CloudTrail logs to Amazon CloudWatch Logs. Do not set up metric filters or alarms.

D. Use VPC endpoints to provide access to Amazon Bedrock and Amazon S3 in the application account.Use only IAM path-based policies to manage data lake access. Send AWS CloudTrail logs to Amazon CloudWatch Logs. Periodically create dashboards and allow public fallback for cross-Region reads to reduce setup time.

Answer: B

Explanation:
The first option labeled B is the correct solution because it fully satisfies both private connectivity and fine- grained cross-account data governance requirements using AWS-native services.
Creating interface VPC endpoints for Amazon Bedrock runtimes ensures that all inference calls remain on the AWS private network and never traverse the public internet. Running AWS Lambda functions in private subnets enforces network isolation, and using IAM conditions that restrict access to specific VPC endpoints and roles prevents unauthorized inference calls.
For the governed data lake, AWS Lake Formation LF-tag-based access control is the recommended AWS mechanism for enforcing cross-account, column-level permissions. LF-tags allow the company to define data access policies once and apply them consistently across accounts, databases, tables,

and even individual columns. This is required for sensitive customer records and is not achievable with S3 bucket policies or IAM alone.
The second option labeled B uses a NAT gateway, which violates the private connectivity requirement.
Option C uses public Bedrock endpoints and only database-level grants, which are insufficient. Option D relies on IAM path-based policies, which cannot enforce column-level access and introduces public fallback paths.
Therefore, the first option labeled B is the only solution that meets all networking, security, and data governance requirements.

NO.6 A company upgraded its Amazon Bedrock-powered foundation model (FM) that supports a multilingual customer service assistant. After the upgrade, the assistant exhibited inconsistent behavior across languages.
The assistant began generating different responses in some languages when presented with identical questions.
The company needs a solution to detect and address similar problems for future updates. The evaluation must be completed within 45 minutes for all supported languages. The evaluation must process at least 15,000 test conversations in parallel. The evaluation process must be fully automated and integrated into the CI/CD pipeline. The solution must block deployment if quality thresholds are not met.
Which solution will meet these requirements?

A. Create a distributed traffic simulation framework that sends translation-heavy workloads to the assistant in multiple languages simultaneously. Use Amazon CloudWatch metrics to monitor latency, concurrency, and throughput. Run simulations before production releases to identify infrastructure bottlenecks.

B. Deploy the assistant in multiple AWS Regions with Amazon Route 53 latency-based routing and AWS Global Accelerator to improve global performance. Store multilingual conversation logs in Amazon S3.
Perform weekly post-deployment audits to review consistency.

C. Create a pre-processing pipeline that normalizes all incoming messages into a consistent format before sending the messages to the assistant. Apply rule-based checks to flag potential hallucinations in the outputs. Focus evaluation on normalized text to simplify testing across languages.

D. Set up standardized multilingual test conversations with identical meaning. Run the test conversations in parallel by using Amazon Bedrock model evaluation jobs. Apply similarity and hallucination thresholds. Integrate the process into the CI/CD pipeline to block releases that fail.

Answer: D
Explanation:
Option D is the correct solution because it directly evaluates multilingual output consistency and quality in an automated, scalable, and deployment-gating workflow. Amazon Bedrock model evaluation jobs are designed to run large-scale, repeatable evaluations against defined datasets and to produce quantitative metrics that can be used as objective release criteria.
The core issue is semantic inconsistency across languages for equivalent inputs. The most reliable way to detect this is to create standardized test conversations where each language version expresses the same intent and constraints. Running those tests through the updated model and comparing results with similarity metrics (for example, semantic similarity between expected and

actual answers, or between language variants) surfaces regressions that infrastructure testing cannot detect.
Bedrock evaluation jobs support running evaluations at scale and are well suited for processing large datasets quickly. By parallelizing evaluation runs across languages and conversations, the company can meet the 45- minute requirement while executing at least 15,000 conversations. Because the process is standardized, it also allows consistent baseline comparisons across releases.
Applying hallucination thresholds ensures that answers remain grounded and do not introduce fabricated details, which is particularly important when language-specific behavior shifts after a model upgrade.
Integrating evaluation jobs into the CI/CD pipeline enables fully automated execution on every model or configuration update. The pipeline can enforce a hard quality gate that blocks deployment if thresholds are not met, preventing regressions from reaching production.
Option A focuses on performance and infrastructure bottlenecks, not multilingual response quality. Option B is post-deployment and too slow to prevent regressions. Option C normalizes inputs but does not measure multilingual output equivalence or provide robust, quantitative gating.
Therefore, Option D best meets the automation, scale, timing, and deployment-blocking requirements.

NO.7 An insurance company uses existing Amazon SageMaker AI infrastructure to support a web-based application that allows customers to predict what their insurance premiums will be. The company stores customer data that is used to train the SageMaker AI model in an Amazon S3 bucket. The dataset is growing rapidly. The company wants a solution to continuously re-train the model. The solution must automatically re-train and re- deploy the model to the application when an employee uploads a new customer data file to the S3 bucket.
Which solution will meet these requirements?

A. Use AWS Glue to run an ETL job on each uploaded file. Configure the ETL job to use the AWS SDK to invoke the SageMaker AI model endpoint. Use real-time inference with the endpoint to re-deploy the model after it is re-trained on the updated customer dataset.

B. Create an AWS Lambda function and webhook handlers to generate an event when an employee uploads a new file. Configure SageMaker Pipelines to re-deploy the model after it is re-trained on the updated customer dataset. Use Amazon EventBridge to create an event bus. Set the Lambda function event as the source and SageMaker Pipelines as the target.

C. Create an AWS Step Functions Express workflow with AWS SDK integrations to retrieve the customer data from the S3 bucket when an employee uploads a new file to the S3 bucket. Use a SageMaker Data Wrangler flow to export the data from the S3 bucket to SageMaker Autopilot. Use the SageMaker Autopilot to re-deploy the model after it has been re-trained on the updated customer dataset.

D. Create an AWS Step Functions Standard workflow. Configure the first state to call an AWS Lambda function to respond when an employee uploads a new file to the S3 bucket. Use a pipeline in SageMaker Pipelines to re-deploy the model after it has been re-trained on the updated customer dataset. Use the next state in the workflow to run the pipeline when the first state receives a response.

Answer: D
Explanation:
Option D is the best fit because it implements a reliable event-driven MLOps workflow that

automates retraining and redeployment with clear orchestration, auditability, and production-grade error handling. The requirement is explicit: whenever a new file is uploaded to Amazon S3, the system must retrain and then redeploy the model used by a web application. A common AWS pattern is to use an S3 event notification to trigger an AWS Lambda function, which then starts a controlled workflow. In option D, Lambda serves as the event handler that reacts immediately to the S3 upload event and passes the necessary context (bucket, object key, dataset version) into an AWS Step Functions Standard state machine.
Step Functions Standard is appropriate for model retraining pipelines because training and deployment steps can be long-running and benefit from durable state, retries, and failure handling. It provides execution history, making it easier to troubleshoot why a particular retraining run failed and to prove which dataset version produced which model version. This operational visibility is critical when the dataset is "growing rapidly" and retraining is frequent.
Within the workflow, Amazon SageMaker Pipelines is the right service to run the ML lifecycle stages in a repeatable way: data processing (if needed), training, evaluation/quality checks, model registration, and deployment to an endpoint used by the application. SageMaker Pipelines is purpose-built for CI/CD-style ML, supporting automated redeployments when a new approved model artifact is produced. By calling a pipeline execution from Step Functions, the company can add governance gates (for example, only deploy if evaluation metrics meet thresholds), and can apply consistent rollback and notification steps when deployment fails.
The other options are weaker: A confuses inference with retraining and does not provide deployment orchestration. B adds unnecessary webhook complexity and describes an awkward event bus configuration. C introduces Autopilot/Data Wrangler, which may be useful but adds extra moving parts and is not required to meet the trigger-and-redeploy requirement.

NO.8 A healthcare company is using Amazon Bedrock to build a system to help practitioners make clinical decisions. The system must provide treatment recommendations to physicians based only on approved medical documentation and must cite specific sources. The system must not hallucinate or produce factually incorrect information.
Which solution will meet these requirements with the LEAST operational overhead?
A. Integrate Amazon Bedrock with Amazon Kendra to retrieve approved documents. Implement custom post-processing to compare generated responses against source documents and to include citations.
B. Deploy an Amazon Bedrock Knowledge Base and connect it to approved clinical source documents.
Use the Amazon Bedrock RetrieveAndGenerate API to return citations from the knowledge base.
C. Use Amazon Bedrock and Amazon Comprehend Medical to extract medical entities. Implement verification logic against a medical terminology database.
D. Use an Amazon Bedrock knowledge base with Retrieve API calls and InvokeModel API calls to retrieve approved clinical source documents. Implement verification logic to compare against retrieved sources and to cite sources.
Answer: B
Explanation:
Option B is the correct solution because Amazon Bedrock Knowledge Bases with the RetrieveAndGenerate API provide a fully managed Retrieval Augmented Generation (RAG) capability that directly addresses grounding, citation, and hallucination prevention with the least operational

overhead.
Amazon Bedrock Knowledge Bases automatically manage document ingestion, chunking, embedding, retrieval, and ranking from approved data sources. When used with the RetrieveAndGenerate API, the model is constrained to generate responses only from retrieved, approved clinical documentation, significantly reducing the risk of hallucinations or unsupported claims. The API also returns explicit source citations, which satisfies regulatory and clinical transparency requirements without requiring custom comparison or validation logic.
This approach aligns with AWS best practices for healthcare GenAI workloads, where correctness and traceability are critical. Because retrieval and generation are tightly integrated, the system avoids multi-step orchestration, custom verification pipelines, or additional compute layers that would increase latency and maintenance burden.
Option A introduces Amazon Kendra and custom post-processing logic, increasing operational complexity.
Option C focuses on entity extraction rather than controlled knowledge grounding and does not guarantee citation or hallucination prevention. Option D requires manual orchestration between retrieval and generation and custom verification logic, which increases development and maintenance effort.
Therefore, Option B delivers accurate, grounded, and cited clinical recommendations with minimal infrastructure and operational overhead.

NO.9 A company is creating a generative AI (GenAI) application that uses Amazon Bedrock foundation models (FMs). The application must use Microsoft Entra ID to authenticate. All FM API calls must stay on private network paths. Access to the application must be limited by department to specific model families. The company also needs a comprehensive audit trail of model interactions. Which solution will meet these requirements?

A. Configure SAML federation between Microsoft Entra ID and AWS Identity and Access Management.
Create department-specific IAM roles that allow only the required ModelId values. Create AWS PrivateLink interface VPC endpoints for Amazon Bedrock runtime services. Enable AWS CloudTrail to capture Amazon Bedrock API calls. Configure Amazon Bedrock model invocation logging to record detailed model interactions.

B. Create an identity provider (IdP) connection in IAM to authenticate by using Microsoft Entra ID. Assign department permission sets to control access to specific model families. Deploy AWS Lambda functions in private subnets with a NAT gateway for egress to Amazon Bedrock public endpoints. Enable CloudWatch Logs to capture model interactions for auditing purposes.

C. Create a SAML identity provider (IdP) in IAM to authenticate by using Microsoft Entra ID. Use IAM permissions boundaries to limit department roles' access to specific model families. Configure public Amazon Bedrock API endpoints with VPC routing to maintain private network connectivity. Set up CloudTrail with Amazon S3 Lifecycle rules to manage audit logs of model interactions.

D. Configure OpenID Connect (OIDC) federation between Microsoft Entra ID and IAM. Use attribute-based access control to map department attributes to specific model access permissions. Apply SCP policies to restrict access to Amazon Bedrock FM families based on department. Use Microsoft Entra ID's built-in logging capabilities to maintain an audit trail of model interactions.

Answer: A
Explanation:

Option A is the correct solution because it satisfies authentication, private connectivity, fine-grained authorization, and auditing using AWS-recommended patterns.
SAML federation between Microsoft Entra ID and IAM is a mature, well-supported integration that enables centralized enterprise authentication. Department-specific IAM roles allow precise control over which Bedrock ModelId values each department can invoke, enforcing access by model family.
Using AWS PrivateLink interface VPC endpoints for Amazon Bedrock runtime services ensures that all inference traffic stays on private AWS network paths, with no public internet exposure. NAT gateways and public endpoints, as used in other options, violate this requirement.
AWS CloudTrail provides authoritative audit logs of all Bedrock API calls, which is required for compliance.
Amazon Bedrock model invocation logging complements CloudTrail by capturing detailed prompt and response metadata for deeper auditing and investigation.
Option B uses public endpoints via NAT. Option C incorrectly claims public endpoints can be private. Option D relies on IdP-side logs, which do not capture Bedrock API activity.
Therefore, Option A is the only solution that fully meets security, compliance, and observability requirements.

NO.10 A company is developing a generative AI (GenAI)-powered customer support application that uses Amazon Bedrock foundation models (FMs). The application must maintain conversational context across multiple interactions with the same user. The application must run clarification workflows to handle ambiguous user queries. The company must store encrypted records of each user conversation to use for personalization. The application must be able to handle thousands of concurrent users while responding to each user quickly.
Which solution will meet these requirements?
A. Use an AWS Step Functions Express workflow to orchestrate conversation flow. Invoke AWS Lambda functions to run clarification logic. Store conversation history in Amazon RDS and use session IDs as the primary key.
B. Use an AWS Step Functions Standard workflow to orchestrate clarification workflows. Include Wait for a Callback patterns to manage the workflows. Store conversation history in Amazon DynamoDB. Purchase on-demand capacity and configure server-side encryption.
C. Deploy the application by using an Amazon API Gateway REST API to route user requests to an AWS Lambda function to update and retrieve conversation context. Store conversation history in Amazon S3 and configure server-side encryption. Save each interaction as a separate JSON file.
D. Use AWS Lambda functions to call Amazon Bedrock inference APIs. Use Amazon SQS queues to orchestrate clarification steps. Store conversation history in an Amazon ElastiCache (Redis OSS) cluster. Configure encryption at rest.
Answer: B
Explanation:
Option B is the correct solution because it provides a scalable, durable, and secure architecture for conversational GenAI workloads that require multi-step clarification workflows and persistent memory.
AWS Step Functions Standard workflows are designed for long-running, stateful workflows with high reliability, which is ideal for clarification loops that may require multiple back-and-forth interactions. The Wait for a Callback pattern allows the workflow to pause while awaiting additional user input, making it well- suited for handling ambiguous queries without losing execution state.

Storing conversation history in Amazon DynamoDB enables millisecond-latency reads and writes at massive scale, supporting thousands of concurrent users. DynamoDB's on-demand capacity mode automatically scales with traffic, eliminating capacity planning. Server-side encryption ensures that stored conversation data is encrypted at rest, meeting security and compliance requirements for personalized data.
Option A uses Step Functions Express and Amazon RDS, which is not ideal for long-lived conversational workflows and introduces scaling and connection management challenges. Option C stores conversations as individual S3 objects, which increases latency and complicates context retrieval. Option D relies on Amazon ElastiCache, which is optimized for ephemeral caching rather than durable, auditable conversation history.
Therefore, Option B best balances scalability, performance, durability, and security for a conversational Amazon Bedrock-based customer support application.

NO.11 A company uses Amazon Bedrock to build a Retrieval Augmented Generation (RAG) system. The RAG system uses an Amazon Bedrock Knowledge Bases that is based on an Amazon S3 bucket as the data source for emergency news video content. The system retrieves transcripts, archived reports, and related documents from the S3 bucket.
The RAG system uses state-of-the-art embedding models and a high-performing retrieval setup. However, users report slow responses and irrelevant results, which cause decreased user satisfaction. The company notices that vector searches are evaluating too many documents across too many content types and over long periods of time.
The company determines that the underlying models will not benefit from additional fine-tuning. The company must improve retrieval accuracy by applying smarter constraints and wants a solution that requires minimal changes to the existing architecture.
Which solution will meet these requirements?

A. Enhance embeddings by using a domain-adapted model that is specifically trained on emergency news content for improved vector similarity.

B. Migrate to Amazon OpenSearch Service. Use vector fields and metadata filters to define the scope of results retrieval.

C. Enable metadata-aware filtering within the Amazon Bedrock knowledge base by indexing S3 object metadata.

D. Migrate to an Amazon Q Business index to perform structured metadata filtering and document categorization during retrieval.

Answer: C
Explanation:
Option C is the correct solution because it directly addresses the root cause of the problem-overly broad retrieval-while requiring minimal architectural change. Amazon Bedrock Knowledge Bases support metadata-aware filtering, which allows the system to constrain retrieval queries based on indexed metadata such as content type, publication date, source, or category.
By indexing Amazon S3 object metadata, the company can restrict vector searches to relevant subsets of the corpus, such as recent emergency reports, specific content formats, or trusted sources. This significantly reduces the number of documents evaluated during retrieval, which improves both latency and result relevance without changing embedding models or retrieval infrastructure.
This approach aligns with AWS best practices for optimizing RAG systems: when embeddings are already strong, retrieval quality is often improved by narrowing the candidate set rather than increasing model complexity. Metadata filtering reduces noise and ensures that retrieved documents

are more contextually aligned with user queries.
Option A requires retraining or adapting embedding models, which the company has already determined will not provide additional benefit. Option B introduces a migration to OpenSearch, which adds operational overhead and deviates from the existing Bedrock knowledge base architecture. Option D requires moving to a different indexing service, increasing complexity and implementation effort.
Therefore, Option C provides the most effective and low-effort solution to improve retrieval accuracy and performance in the existing Amazon Bedrock RAG system.

NO.12 A healthcare company is using Amazon Bedrock to build a Retrieval Augmented Generation (RAG) application that helps practitioners make clinical decisions. The application must achieve high accuracy for patient information retrievals, identify hallucinations in generated content, and reduce human review costs.
Which solution will meet these requirements?
A. Use Amazon Comprehend to analyze and classify RAG responses and to extract medical entities and relationships. Use AWS Step Functions to orchestrate automated evaluations. Configure Amazon CloudWatch metrics to track entity recognition confidence scores. Configure CloudWatch to send an alert when accuracy falls below specified thresholds.
B. Implement automated large language model (LLM)-based evaluations that use a specialized model that is fine-tuned for medical content to assess all responses. Deploy AWS Lambda functions to parallelize evaluations. Publish results to Amazon CloudWatch metrics that track relevance and factual accuracy.
C. Configure Amazon CloudWatch Synthetics to generate test queries that have known answers on a regular schedule, and track model success rates. Set up dashboards that compare synthetic test results against expected outcomes.
D. Deploy a hybrid evaluation system that uses an automated LLM-as-a-judge evaluation to initially screen responses and targeted human reviews for edge cases. Use a built-in Amazon Bedrock evaluation to track retrieval precision and hallucination rates.
Answer: D
Explanation:
Option D is the correct solution because it directly addresses all three requirements: high retrieval accuracy, hallucination detection, and reduced human review costs. AWS recommends a layered evaluation strategy for high-stakes domains such as healthcare, where generative outputs must be both accurate and safe.
Using an automated LLM-as-a-judge evaluation enables scalable, consistent assessment of generated responses for factual grounding, relevance, and hallucination risk. This automated screening significantly reduces the number of responses that require manual inspection. Only responses that fall below defined quality thresholds or exhibit ambiguous behavior are escalated to targeted human reviews, which optimizes review effort and cost.
The use of Amazon Bedrock built-in evaluations provides standardized metrics specifically designed for RAG systems, including retrieval precision, faithfulness to source documents, and hallucination rates. These evaluations integrate directly with Amazon Bedrock knowledge bases and models, eliminating the need to build and maintain custom evaluation pipelines.
Option A focuses on entity extraction confidence, which does not reliably detect hallucinations in generative text. Option B requires maintaining and scaling a separate fine-tuned evaluation model,

increasing complexity and cost. Option C is useful for regression testing but cannot detect hallucinations in real-world, open-ended clinical queries.
Therefore, Option D provides the most effective and operationally efficient approach to maintaining clinical- grade accuracy while minimizing human review effort.

NO.13 A media company must use Amazon Bedrock to implement a robust governance process for AI-generated content. The company needs to manage hundreds of prompt templates. Multiple teams use the templates across multiple AWS Regions to generate content. The solution must provide version control with approval workflows that include notifications for pending reviews. The solution must also provide detailed audit trails that document prompt activities and consistent prompt parameterization to enforce quality standards.
Which solution will meet these requirements?

A. Configure Amazon Bedrock Studio prompt templates. Use Amazon CloudWatch dashboards to display prompt usage metrics. Store approval status in Amazon DynamoDB. Use AWS Lambda functions to enforce approvals.

B. Use Amazon Bedrock Prompt Management to implement version control. Configure AWS CloudTrail for audit logging. Use AWS Identity and Access Management policies to control approval permissions.
Create parameterized prompt templates by specifying variables.

C. Use AWS Step Functions to create an approval workflow. Store prompts in Amazon S3. Use tags to implement version control. Use Amazon EventBridge to send notifications.

D. Deploy Amazon SageMaker Canvas with prompt templates stored in Amazon S3. Use AWS CloudFormation for version control. Use AWS Config to enforce approval policies.

Answer: B

Explanation:
Option B is the correct solution because Amazon Bedrock Prompt Management is purpose-built to manage, govern, and standardize prompt usage at scale across teams and Regions. It provides native version control, allowing teams to track prompt changes over time and ensure that only approved versions are used in production workflows.
Prompt Management supports approval workflows that align with enterprise governance requirements.
Approval permissions can be enforced through IAM policies, ensuring that only authorized reviewers can approve or publish prompt versions. This removes the need for custom workflow engines or external storage systems, significantly reducing operational overhead.
Parameterized prompt templates enable consistent prompt structure while allowing controlled variation through defined variables. This ensures consistent quality standards and reduces prompt drift, which is critical when hundreds of prompts are reused across multiple applications and teams.
AWS CloudTrail integrates natively with Amazon Bedrock to provide immutable audit logs for prompt creation, updates, approvals, and usage. These detailed audit trails satisfy compliance requirements and allow security and governance teams to trace prompt activity across Regions and users.
Option A requires significant custom development to coordinate approvals and maintain state.
Option C relies on general-purpose workflow services and manual versioning mechanisms that are error-prone and difficult to scale. Option D uses services not designed for large-scale GenAI prompt governance and introduces unnecessary complexity.
Therefore, Option B best meets the requirements for scalable, auditable, and low-overhead

governance of AI- generated content using Amazon Bedrock.

NO.14 A healthcare company is using Amazon Bedrock to develop a real-time patient care AI assistant to respond to queries for separate departments that handle clinical inquiries, insurance verification, appointment scheduling, and insurance claims. The company wants to use a multi-agent architecture.
The company must ensure that the AI assistant is scalable and can onboard new features for patients. The AI assistant must be able to handle thousands of parallel patient interactions. The company must ensure that patients receive appropriate domain-specific responses to queries.
Which solution will meet these requirements?
A. Isolate data for each agent by using separate knowledge bases. Use IAM filtering to control access to each knowledge base. Deploy a supervisor agent to perform natural language intent classification on patient inquiries. Configure the supervisor agent to route queries to specialized collaborator agents to respond to department-specific queries. Configure each specialized collaborator agent to use Retrieval Augmented Generation (RAG) with the agent's department-specific knowledge base.
B. Create a separate supervisor agent for each department. Configure individual collaborator agents to perform natural language intent classification for each specialty domain within each department. Integrate each collaborator agent with department-specific knowledge bases only. Implement manual handoff processes between the supervisor agents.
C. Isolate data for each department in separate knowledge bases. Use IAM filtering to control access to each knowledge base. Deploy a single general-purpose agent. Configure multiple action groups within the general-purpose agent to perform specific department functions. Implement rule-based routing logic within the general-purpose agent instructions.
D. Implement multiple independent supervisor agents that run in parallel to respond to patient inquiries for each department. Configure multiple collaborator agents for each supervisor agent. Integrate all agents with the same knowledge base. Use external routing logic to merge responses from multiple supervisor agents.
Answer: A
Explanation:
Option A is the most appropriate design because it provides scalable multi-agent orchestration, clear domain separation, and strong governance with minimal operational complexity. A supervisor-agent pattern is a standard AWS-recommended approach for multi-agent systems: one agent performs intent classification and routing, while specialized agents handle domain-specific tasks.
Isolating data with separate knowledge bases ensures that each specialized collaborator agent retrieves only the information relevant to its department. This improves response accuracy, reduces hallucinations, and supports privacy controls because clinical content, claims content, and scheduling content can have different access policies. IAM-based filtering ensures that each agent has permission only to the knowledge base it is authorized to use.
Routing patient inquiries through a supervisor agent supports high concurrency and extensibility. New departments or features can be added by introducing new collaborator agents and knowledge bases without redesigning the entire system. Because routing is handled centrally, changes in classification logic do not require updates across many independent supervisors.
Using RAG within each collaborator agent ensures that responses are grounded in department-approved information sources, which is critical in healthcare settings to reduce unsafe or incorrect guidance. This approach also improves performance because each retrieval scope is smaller and

more relevant, supporting thousands of parallel interactions.
Option B introduces manual handoffs that do not scale. Option C relies on rule-based routing inside one general agent, which becomes brittle and difficult to govern as complexity grows. Option D mixes all departments into a single knowledge base and merges responses externally, increasing risk of incorrect domain answers and operational overhead.
Therefore, Option A best meets the scalability, correctness, and multi-agent onboarding requirements.

NO.15 A company has a recommendation system running on Amazon EC2 instances. The applications make API calls to Amazon Bedrock foundation models (FMs) to analyze customer behavior and generate personalized product recommendations.
The system experiences intermittent issues where some recommendations do not match customer preferences.
The company needs an observability solution to monitor operational metrics and detect patterns of performance degradation compared to established baselines. The solution must generate alerts with correlation data within 10 minutes when FM behavior deviates from expected patterns.
Which solution will meet these requirements?

A. Configure Amazon CloudWatch Container Insights. Set up alarms for latency thresholds. Add custom token metrics using the CloudWatch embedded metric format.

B. Implement AWS X-Ray. Enable CloudWatch Logs Insights. Set up AWS CloudTrail and create dashboards in Amazon QuickSight.

C. Enable Amazon CloudWatch Application Insights. Create custom metrics for recommendation quality, token usage, and response latency using the CloudWatch embedded metric format with dimensions for request types and user segments. Configure CloudWatch anomaly detection on model metrics. Use CloudWatch Logs Insights for pattern analysis.

D. Use Amazon OpenSearch Service with the Observability plugin. Ingest metrics and logs through Amazon Kinesis and analyze behavior with custom queries.

Answer: C

Explanation:
Option C best satisfies the requirement for rapid, correlated detection of model-related performance degradation. Amazon CloudWatch Application Insights provides automated observability across application components running on Amazon EC2, identifying abnormal behavior patterns without requiring extensive manual configuration.
Using custom metrics for recommendation quality, token usage, and response latency allows the company to directly monitor FM behavior, not just infrastructure health. Applying dimensions such as request type and user segment enables fine-grained correlation between performance issues and specific customer interactions or workloads.
CloudWatch anomaly detection is critical because it establishes dynamic baselines from historical data and detects deviations automatically. This enables alerts to be generated within minutes when FM behavior changes unexpectedly, satisfying the 10-minute alerting requirement without static thresholds that can miss subtle degradations.
CloudWatch Logs Insights complements metrics by enabling rapid analysis of log patterns, error messages, or unusual request flows associated with degraded recommendations. Because all data remains within CloudWatch, correlation between metrics, logs, and alerts is straightforward and operationally efficient.
Option A focuses on infrastructure metrics and lacks behavioral baselining. Option B provides tracing

but not automated anomaly detection. Option D adds significant operational overhead and ingestion complexity for a use case already well supported by CloudWatch-native features.
Therefore, Option C delivers the most effective, scalable, and low-overhead observability solution for detecting FM-related performance deviations.

NO.16 A company has deployed an AI assistant as a React application that uses AWS Amplify, an AWS AppSync GraphQL API, and Amazon Bedrock Knowledge Bases. The application uses the GraphQL API to call the Amazon Bedrock RetrieveAndGenerate API for knowledge base interactions. The company configures an AWS Lambda resolver to use the RequestResponse invocation type. Application users report frequent timeouts and slow response times. Users report these problems more frequently for complex questions that require longer processing.
The company needs a solution to fix these performance issues and enhance the user experience.
Which solution will meet these requirements?

A. Use AWS Amplify AI Kit to implement streaming responses from the GraphQL API and to optimize client-side rendering.

B. Increase the timeout value of the Lambda resolver. Implement retry logic with exponential backoff.

C. Update the application to send an API request to an Amazon SQS queue. Update the AWS AppSync resolver to poll and process the queue.

D. Change the RetrieveAndGenerate API to the InvokeModelWithResponseStream API. Update the application to use an Amazon API Gateway WebSocket API to support the streaming response.

Answer: A

Explanation:
Option A is the best solution because it directly addresses both observed problems: user-perceived latency and resolver timeouts that occur more frequently for complex prompts. In the current design, an AWS AppSync Lambda resolver is configured with synchronous RequestResponse behavior. That means the client receives nothing until the entire retrieval and generation workflow completes. For longer-running knowledge base queries, this increases the likelihood of hitting request time limits in the synchronous path and creates a poor user experience because the UI appears stalled.
Using AWS Amplify AI Kit to implement streaming responses allows the application to return partial output incrementally as the model produces tokens. This improves perceived responsiveness because users can see the answer forming immediately, even when the full response takes longer. Streaming also reduces the impact of variable model latency and retrieval time because the client no longer waits for a single final payload before rendering content. From a troubleshooting perspective, streaming makes it easier to distinguish "slow generation" from "no response," and it provides faster feedback during testing of complex questions.
Option B is not sufficient because increasing timeouts and adding retries can worsen load and cost while still producing a stalled UI experience. Retries also risk duplicating requests to the knowledge base and can amplify token usage. Option C introduces an awkward polling model for GraphQL interactions and adds significant operational complexity, while not inherently improving interactivity. Option D adds major architectural changes by replacing the knowledge base RetrieveAndGenerate call path with a different streaming invocation API and introducing a WebSocket layer, which is unnecessary when the goal is primarily to fix timeouts and improve UX within the existing AppSync and Amplify design.
Therefore, streaming through Amplify AI Kit is the most effective and lowest-friction improvement.
Thought for 24s

NO.17 A company is using Amazon Bedrock to design an application to help researchers apply for grants. The application is based on an Amazon Nova Pro foundation model (FM). The application contains four required inputs and must provide responses in a consistent text format. The company wants to receive a notification in Amazon Bedrock if a response contains bullying language. However, the company does not want to block all flagged responses.
The company creates an Amazon Bedrock flow that takes an input prompt and sends it to the Amazon Nova Pro FM. The Amazon Nova Pro FM provides a response.
Which additional steps must the company take to meet these requirements? (Select TWO.)
A. Use Amazon Bedrock Prompt Management to specify the required inputs as variables. Select an Amazon Nova Pro FM. Specify the output format for the response. Add the prompt to the prompts node of the flow.
B. Create an Amazon Bedrock guardrail that applies the hate content filter. Set the filter response to block.
Add the guardrail to the prompts node of the flow.
C. Create an Amazon Bedrock prompt router. Specify an Amazon Nova Pro FM. Add the required inputs as variables to the input node of the flow. Add the prompt router to the prompts node. Add the output format to the output node.
D. Create an Amazon Bedrock guardrail that applies the insults content filter. Set the filter response to detect. Add the guardrail to the prompts node of the flow.
E. Create an Amazon Bedrock application inference profile that specifies an Amazon Nova Pro FM.Specify the output format for the response in the description. Include a tag for each of the input variables. Add the profile to the prompts node of the flow.
Answer: A D
Explanation:
The correct answers are A and D because they collectively satisfy the requirements for structured inputs, consistent output formatting, and non-blocking detection of bullying language.
Option A is required because Amazon Bedrock Prompt Management enables prompt templates with explicit input variables and defined output formats. By defining the four required inputs as variables, the company ensures that every invocation of the Amazon Nova Pro FM receives the correct structured inputs. Specifying the output format ensures consistent responses, which is essential for a grants application workflow. Adding the managed prompt to the prompts node of the flow allows Bedrock Flows to invoke the model using this standardized configuration.
Option D addresses the requirement to receive notifications when bullying language is detected without blocking responses. Amazon Bedrock guardrails support content filters with configurable actions. By applying the insults content filter and setting the response action to detect, the system flags responses containing bullying or insulting language while still allowing the response to be returned. This enables monitoring, alerting, and auditing without interrupting application functionality.
Option B is incorrect because setting the filter response to block contradicts the requirement not to block all flagged responses. Option C introduces a prompt router, which is unnecessary because the application uses a single Amazon Nova Pro FM. Option E incorrectly attempts to enforce input variables and output formatting through an inference profile, which does not provide prompt-level variable enforcement or formatting guarantees.
Therefore, A and D together provide structured prompt management and non-blocking safety

detection with minimal operational complexity.

NO.18 A company needs a system to automatically generate study materials from multiple content sources. The content sources include document files (PDF files, PowerPoint presentations, and Word documents) and multimedia files (recorded videos). The system must process more than 10,000 content sources daily with peak loads of 500 concurrent uploads. The system must also extract key concepts from document files and multimedia files and create contextually accurate summaries. The generated study materials must support real- time collaboration with version control.
Which solution will meet these requirements?

A. Use Amazon Bedrock Data Automation (BDA) with AWS Lambda functions to orchestrate document file processing. Use Amazon Bedrock Knowledge Bases to process all multimedia. Store the content in Amazon DocumentDB with replication. Collaborate by using Amazon SNS topic subscriptions. Track changes by using Amazon Bedrock Agents.

B. Use Amazon Bedrock Data Automation (BDA) with foundation models (FMs) to process document files. Integrate BDA with Amazon Textract for PDF extraction and with Amazon Transcribe for multimedia files. Store the processed content in Amazon S3 with versioning enabled. Store the metadata in Amazon DynamoDB. Collaborate in real time by using AWS AppSync GraphQL subscriptions and DynamoDB.

C. Use Amazon Bedrock Data Automation (BDA) with Amazon SageMaker AI endpoints to host content extraction and summarization models. Use Amazon Bedrock Guardrails to extract content from all file types. Store document files in Amazon Neptune for time series analysis. Collaborate by using Amazon Bedrock Chat for real-time messaging.

D. Use Amazon Bedrock Data Automation (BDA) with AWS Lambda functions to process batches of content files. Fine-tune foundation models (FMs) in Amazon Bedrock to classify documents across all content types. Store the processed data in Amazon ElastiCache (Redis OSS) by using Cluster Mode with sharding. Use Prompt management in Amazon Bedrock for version control.

Answer: B

Explanation:

Option B best fulfills all functional, scalability, and collaboration requirements by combining purpose-built AWS services with Amazon Bedrock capabilities. Amazon Bedrock Data Automation is designed to orchestrate large-scale, multimodal data processing pipelines and integrates naturally with foundation models for summarization and concept extraction. Using BDA to process document files ensures consistent preprocessing and model invocation at scale, which is essential for handling more than 10,000 sources per day with high concurrency.

Integrating Amazon Textract for PDFs enables accurate extraction of structured and unstructured text from scanned and digital documents, while Amazon Transcribe is the appropriate service for converting recorded videos into text for downstream semantic analysis. These services are optimized for their respective media types and feed clean, normalized inputs into Bedrock foundation models, improving the quality of contextual summaries.

Storing processed content in Amazon S3 with versioning enabled directly addresses the requirement for version control. S3 versioning provides immutable object history and rollback capabilities without additional complexity. Metadata storage in Amazon DynamoDB supports high-throughput, low-latency access patterns and scales automatically to handle peak upload concurrency.

Real-time collaboration is achieved through AWS AppSync GraphQL subscriptions combined with DynamoDB. AppSync enables real-time updates to connected clients whenever study materials are

created or modified, making it well suited for collaborative editing and live synchronization. DynamoDB streams integrate seamlessly with AppSync to propagate changes efficiently.
The other options misuse services or fail to meet key requirements. Amazon SNS does not support collaborative state synchronization, Amazon DocumentDB is not optimized for versioned document storage, Amazon Neptune is unsuitable for document-centric workloads, and Amazon ElastiCache is not designed for durable storage or version control. Option B aligns with AWS best practices for scalable, multimodal generative AI systems built on Amazon Bedrock.

NO.19 A company is designing a solution that uses foundation models (FMs) to support multiple AI workloads.
Some FMs must be invoked on demand and in real time. Other FMs require consistent high-throughput access for batch processing.
The solution must support hybrid deployment patterns and run workloads across cloud infrastructure and on- premises infrastructure to comply with data residency and compliance requirements.
Which combination of steps will meet these requirements? (Select TWO.)

A. Use AWS Lambda to orchestrate low-latency FM inference by invoking FMs hosted on Amazon SageMaker AI asynchronous endpoints.

B. Configure provisioned throughput in Amazon Bedrock to ensure consistent performance for high-volume workloads.

C. Deploy FMs to Amazon SageMaker AI endpoints with support for edge deployment by using Amazon SageMaker Neo. Orchestrate the FMs by using AWS Lambda to support hybrid deployment.

D. Use Amazon Bedrock with auto-scaling to handle unpredictable traffic surges.

E. Use Amazon SageMaker JumpStart to host and invoke the FMs.

Answer: B C

Explanation:
The correct combination is B and C because together they address both workload diversity and hybrid deployment requirements with minimal custom engineering.
Option B provides consistent, high-throughput access by configuring provisioned throughput in Amazon Bedrock. Provisioned throughput guarantees predictable capacity and performance, which is essential for batch processing workloads that require sustained inference rates. This eliminates cold starts and throttling concerns that can occur with purely on-demand usage, making it well suited for high-volume enterprise workloads.
Option C enables hybrid deployment across cloud and on-premises environments by deploying foundation models to Amazon SageMaker AI endpoints and using Amazon SageMaker Neo for edge and on-premises optimization. SageMaker Neo compiles models for target hardware, allowing inference to run efficiently outside the AWS cloud while still using AWS-managed tooling. Orchestrating these deployments with AWS Lambda allows consistent invocation patterns across environments.
Option A uses asynchronous endpoints, which are not suitable for real-time, low-latency inference. Option D addresses scaling but does not support on-premises or hybrid deployment. Option E simplifies model onboarding but does not address hybrid execution or guaranteed throughput. Therefore, Options B and C together provide real-time and batch support, predictable performance, and true hybrid deployment while minimizing operational overhead.

NO.20 A company provides a service that helps users from around the world discover new restaurants. The service has 50 million monthly active users. The company wants to implement a

semantic search solution across a database that contains 20 million restaurants and 200 million reviews. The company currently stores the data in PostgreSQL.
The solution must support complex natural language queries and return results for at least 95% of queries within 500 ms. The solution must maintain data freshness for restaurant details that update hourly. The solution must also scale cost-effectively during peak usage periods.
Which solution will meet these requirements with the LEAST development effort?

A. Migrate the restaurant data to Amazon OpenSearch Service. Implement keyword-based search rules that use custom analyzers and relevance tuning to find restaurants based on attributes such as cuisine type, features, and location. Create Amazon API Gateway HTTP API endpoints to transform user queries into structured search parameters.

B. Migrate the restaurant data to Amazon OpenSearch Service. Use a foundation model (FM) in Amazon Bedrock to generate vector embeddings from restaurant descriptions, reviews, and menu items. When users submit natural language queries, convert the queries to embeddings by using the same FM.
Perform k-nearest neighbors (k-NN) searches to find semantically similar results.

C. Keep the restaurant data in PostgreSQL and implement a pgvector extension. Use a foundation model (FM) in Amazon Bedrock to generate vector embeddings from restaurant data. Store the vector embeddings directly in PostgreSQL. Create an AWS Lambda function to convert natural language queries to vector representations by using the same FM. Configure the Lambda function to perform similarity searches within the database.

D. Migrate restaurant data to an Amazon Bedrock knowledge base by using a custom ingestion pipeline.Configure the knowledge base to automatically generate embeddings from restaurant information. Use the Amazon Bedrock Retrieve API with built-in vector search capabilities to query the knowledge base directly by using natural language input.

Answer: B

Explanation:
Option B best satisfies the requirements while minimizing development effort by combining managed semantic search capabilities with fully managed foundation models. AWS Generative AI guidance describes semantic search as a vector-based retrieval pattern where both documents and user queries are embedded into a shared vector space. Similarity search (such as k-nearest neighbors) then retrieves results based on meaning rather than exact keywords.
Amazon OpenSearch Service natively supports vector indexing and k-NN search at scale. This makes it well suited for large datasets such as 20 million restaurants and 200 million reviews while still achieving sub- second latency for the majority of queries. Because OpenSearch is a distributed, managed service, it automatically scales during peak traffic periods and provides cost-effective performance compared with building and tuning custom vector search pipelines on relational databases.
Using Amazon Bedrock to generate embeddings significantly reduces development complexity. AWS manages the foundation models, eliminates the need for custom model hosting, and ensures consistency by using the same FM for both document embeddings and query embeddings. This aligns directly with AWS- recommended semantic search architectures and removes the need for model lifecycle management.
Hourly updates to restaurant data can be handled efficiently through incremental re-indexing in OpenSearch without disrupting query performance. This approach cleanly separates transactional data storage from search workloads, which is a best practice in AWS architectures.

Option A does not meet the semantic search requirement because keyword-based search cannot reliably interpret complex natural language intent. Option C introduces scalability and performance risks by running large-scale vector similarity searches inside PostgreSQL, which increases operational complexity. Option D adds unnecessary ingestion and abstraction layers intended for retrieval-augmented generation, not high- throughput semantic search.
Therefore, Option B provides the optimal balance of performance, scalability, data freshness, and minimal development effort using AWS Generative AI services.

NO.21 A company is building a generative AI (GenAI) application that uses Amazon Bedrock APIs to process complex customer inquiries. During peak usage periods, the application experiences intermittent API timeouts that cause issues such as broken response chunks and delayed data delivery. The application struggles to ensure that prompts remain within token limits when handling complex customer inquiries of varying lengths.
Users have reported truncated inputs and incomplete responses. The company has also observed foundation model (FM) invocation failures.
The company needs a retry strategy that automatically handles transient service errors and prevents overwhelming Amazon Bedrock during peak usage periods. The strategy must also adapt to changing service availability and support response streaming and token-aware request handling.
Which solution will meet these requirements?

A. Implement a standard retry strategy that uses a 1-second fixed delay between attempts and a 3-retry maximum for all errors. Handle streaming response timeouts by restarting streams. Cap token usage for each session.

B. Implement an adaptive retry strategy that uses exponential backoff with jitter and a circuit breaker pattern that temporarily disables retries when error rates exceed a predefined threshold. Implement a streaming response handler that monitors for chunk delivery timeouts. Configure the handler to buffer successfully received chunks and intelligently resume streaming from the last received chunk when connections are re-established.

C. Use the AWS SDK to configure a retry strategy in standard mode. Wrap Amazon Bedrock API calls in try-catch blocks that handle timeout exceptions. Return cached completions for failed streaming requests. Enforce a global token limit for all users. Add jitter-based retry logic and lightweight token trimming for each request. Resume broken streams by requesting only missing chunks from the point of failure. Maintain a small in-memory buffer of the most recent chunks.

D. Set Amazon Bedrock client request timeouts to 30 seconds. Implement client-side load shedding.Buffer partial results and stop new requests when application performance degrades. Set static token usage caps for all requests. Configure exponential backoff retries, dynamic chunk sizing, and context- aware token limits.

Answer: B
Explanation:
Option B best meets all requirements because it combines AWS-recommended resiliency patterns for transient failures with streaming-aware handling and adaptive protection against cascading retries during peak load. When timeouts and throttling occur, naive retries can amplify traffic and worsen outages. Exponential backoff with jitter is the standard AWS best practice because it spreads retry attempts over time, reduces synchronized retry storms, and lowers the probability of repeatedly colliding with service limits.
The requirement also states the strategy must "adapt to changing service availability" and "prevent

overwhelming Amazon Bedrock." A circuit breaker pattern directly addresses this by temporarily stopping or reducing retries when failure rates exceed a threshold, allowing the system to degrade gracefully instead of continually hammering the service. This is a key mechanism to prevent cascading failures during throttling events.
Because the application uses response streaming and experiences broken chunks, the retry strategy must be streaming-aware. A streaming response handler that detects chunk delivery timeouts and buffers already received chunks prevents the user from losing progress when a connection drops. Resuming from the last successfully received chunk minimizes redundant generation and reduces additional load on the model compared with restarting the entire stream. This supports better user experience and better service efficiency during intermittent failures.
Token-aware request handling is supported in this architecture because the application can apply token budgeting before invoking the model (for example, trimming or summarizing excessive context) while still preserving streaming output behavior. Option B provides the correct backbone for this by focusing on adaptive control and robust streaming recovery.
Option A is too simplistic and risks retry storms. Option C combines conflicting elements (global token limit, cached completions for streaming) and includes impractical "request only missing chunks" behavior that is not a reliable property of streamed generative output. Option D includes useful ideas (load shedding) but relies on static caps and does not provide as strong adaptive retry control as circuit breaking.
Therefore, Option B is the most correct and operationally safe strategy for peak-load Bedrock streaming workloads.

NO.22 A specialty coffee company has a mobile app that generates personalized coffee roast profiles by using Amazon Bedrock with a three-stage prompt chain. The prompt chain converts user inputs into structured metadata, retrieves relevant logs for coffee roasts, and generates a personalized roast recommendation for each customer.
Users in multiple AWS Regions report inconsistent roast recommendations for identical inputs, slow inference during the retrieval step, and unsafe recommendations such as brewing at excessively high temperatures. The company must improve the stability of outputs for repeated inputs. The company must also improve app performance and the safety of the app's outputs. The updated solution must ensure 99.5% output consistency for identical inputs and achieve inference latency of less than 1 second. The solution must also block unsafe or hallucinated recommendations by using validated safety controls.
Which solution will meet these requirements?
A. Deploy Amazon Bedrock with provisioned throughput to stabilize inference latency. Apply Amazon Bedrock guardrails that have semantic denial rules to block unsafe outputs. Use Amazon Bedrock Prompt Management to manage prompts by using approval workflows.
B. Use Amazon Bedrock Agents to manage chaining. Log model inputs and outputs to Amazon CloudWatch Logs. Use logs from Amazon CloudWatch to perform A/B testing for prompt versions.
C. Cache prompt results in Amazon ElastiCache. Use AWS Lambda functions to pre-process metadata and to trace end-to-end latency. Use AWS X-Ray to identify and remediate performance bottlenecks.
D. Use Amazon Kendra to improve roast log retrieval accuracy. Store normalized prompt metadata within Amazon DynamoDB. Use AWS Step Functions to orchestrate multi-step prompts.
Answer: A
Explanation:
Option A best meets the combined requirements of low latency, stability, and validated safety

controls by using purpose-built Amazon Bedrock features designed for production GenAI operations. The company's latency target of under 1 second and its observation of degradation during spikes strongly indicate capacity and throughput variability. Provisioned throughput for Amazon Bedrock is intended to deliver more predictable performance by reserving inference capacity for a chosen model, reducing throttling risk and stabilizing response times under load. This directly improves operational consistency across Regions where on-demand capacity can vary.
The requirement to "block unsafe or hallucinated recommendations" is most directly addressed by Amazon Bedrock Guardrails. Guardrails provide managed safety enforcement, including sensitive information controls and configurable content policies. Using semantic denial rules enables the application to prevent unsafe guidance such as dangerous brewing temperatures or other harmful procedural instructions, enforcing safety at the model boundary rather than relying on downstream filtering.
The remaining requirement is "99.5% output consistency for identical inputs." While generative models can be probabilistic, production systems achieve practical consistency by controlling prompt versions, inputs, and policy behavior. Amazon Bedrock Prompt Management supports controlled prompt lifecycle practices, including versioning and approval workflows, which reduce unintended drift across deployments and Regions. By ensuring the same approved prompt templates and parameters are used consistently, the company can materially improve repeatability for the same structured inputs and retrieval context, which is essential in multi-stage prompt chains.
The other options are incomplete. B improves experimentation and observability but does not enforce safety controls or stabilize latency. C can improve performance, but it does not provide validated safety enforcement at inference time. D can help retrieval relevance, but it does not address unsafe outputs or inference stability.
Therefore, A is the only option that simultaneously targets predictable latency, governance of prompt behavior, and strong safety controls within Amazon Bedrock.

NO.23 A company is building a multicloud generative AI (GenAI)-powered secret resolution application that uses Amazon Bedrock and Agent Squad. The application resolves secrets from multiple sources, including key stores and hardware security modules (HSMs). The application uses AWS Lambda functions to retrieve secrets from the sources. The application uses AWS AppConfig to implement dynamic feature gating. The application supports secret chaining and detects secret drift. The application handles short-lived and expiring secrets. The application also supports prompt flows for templated instructions. The application uses AWS Step Functions to orchestrate agents to resolve the secrets and to manage secret validation and drift detection.
The company finds multiple issues during application testing. The application does not refresh expired secrets in time for agents to use. The application sends alerts for secret drift, but agents still use stale data. Prompt flows within the application reuse outdated templates, which cause cascading failures. The company must resolve the performance issues.
Which solution will meet this requirement?

A. Use Step Functions Map states to run agent workflows in parallel. Pass updated secret metadata through Lambda function outputs. Use AWS AppConfig to version all prompt flows to gate and roll back faulty templates.

B. Use Amazon Bedrock Agents only. Configure Amazon Bedrock guardrails to restrict prompt variation.
Use an inline JSON schema for a single agent's workflow definition to chain tool calls.

C. Use a centralized Amazon EventBridge pipeline to invoke each agent. Store intermediate prompts

in Amazon DynamoDB. Resolve agent ordering by using TTL-based backoff and retries.

D. Use Amazon EventBridge Pipes to invoke resolvers based on Amazon CloudWatch log patterns. Store response metadata in DynamoDB with TTL and versioned writes. Use Amazon Q Developer to dynamically generate fallback prompts.

Answer: A

Explanation:

Option A is the correct solution because it directly addresses all identified failure modes while preserving the existing Step Functions-based orchestration architecture with minimal redesign.

Using Step Functions Map states enables parallel execution of secret resolution workflows, which improves refresh latency for short-lived and expiring secrets. This ensures that secrets are refreshed in time before downstream agents require them. Passing updated secret metadata through Lambda outputs guarantees that subsequent steps always consume the latest resolved values, preventing agents from using stale data even after drift alerts are generated.

Versioning prompt flows in AWS AppConfig is critical to resolving cascading failures caused by outdated templates. AppConfig natively supports version control, validation, staged rollout, and rollback of configuration artifacts. By gating prompt flows through AppConfig, the company can immediately roll back faulty templates and prevent agents from reusing outdated instructions.

This solution maintains clear separation of concerns: Step Functions handle orchestration and parallelism, Lambda handles secret retrieval and metadata propagation, and AppConfig governs prompt lifecycle management. No additional event pipelines or custom retry coordination layers are required.

Option B oversimplifies the architecture and does not address secret lifecycle or drift. Option C introduces event-driven ordering complexity without solving prompt versioning. Option D introduces unnecessary tooling and dynamic prompt generation risk.

Therefore, Option A best resolves performance, correctness, and stability issues while minimizing operational overhead.

NO.24 A company uses Amazon Bedrock to generate technical content for customers. The company has recently experienced a surge in hallucinated outputs when the company's model generates summaries of long technical documents. The model outputs include inaccurate or fabricated details. The company's current solution uses a large foundation model (FM) with a basic one-shot prompt that includes the full document in a single input.

The company needs a solution that will reduce hallucinations and meet factual accuracy goals. The solution must process more than 1,000 documents each hour and deliver summaries within 3 seconds for each document.

Which combination of solutions will meet these requirements? (Select TWO.)

A. Implement zero-shot chain-of-thought (CoT) instructions that require step-by-step reasoning with explicit fact verification before the model generates each summary.

B. Use Retrieval Augmented Generation (RAG) with an Amazon Bedrock knowledge base. Apply semantic chunking and tuned embeddings to ground summaries in source content.

C. Configure Amazon Bedrock guardrails to block any generated output that matches patterns that are associated with hallucinated content.

D. Increase the temperature parameter in Amazon Bedrock.

E. Prompt the Amazon Bedrock model to summarize each full document in one pass.

Answer: B C

Explanation:
The correct answers are B and C because they directly address hallucination reduction while maintaining high throughput and low latency.
Option B reduces hallucinations at their source by grounding model outputs in verified content through Retrieval Augmented Generation (RAG). Using an Amazon Bedrock knowledge base with semantic chunking ensures that long technical documents are broken into meaningfully coherent sections. This allows the model to retrieve only the most relevant chunks, rather than processing an entire document in one pass, which significantly improves factual accuracy and reduces cognitive overload on the model. This approach scales efficiently and supports processing more than 1,000 documents per hour.
Option C adds a defense-in-depth safety layer by using Amazon Bedrock guardrails to detect and block hallucination-like output patterns. Guardrails operate at inference time with minimal performance overhead, making them suitable for low-latency requirements. While guardrails do not eliminate hallucinations entirely, they effectively prevent unsafe or clearly fabricated outputs from reaching users.
Option A increases latency and cost due to explicit reasoning steps and does not scale well for high-throughput workloads. Option D increases randomness and worsens hallucinations. Option E repeats the existing flawed approach.
Therefore, Options B and C together provide scalable grounding and runtime protection that meet accuracy, performance, and throughput requirements.

NO.25 A company is developing a generative AI (GenAI) application by using Amazon Bedrock. The application will analyze patterns and relationships in the company's data. The application will process millions of new data points daily across AWS Regions in Europe, North America, and Asia before storing the data in Amazon S3.
The application must comply with local data protection and storage regulations. Data residency and processing must occur within the same continent. The application must also maintain audit trails of the application's decision-making processes and provide data classification capabilities.
Which solution will meet these requirements?
A. Deploy the application in each Region with local IAM policies. Use Amazon Bedrock cross-Region inference to distribute the workload. Use Amazon CloudWatch to log AI decision-making processes. Manually track compliance certifications across Regions.
B. Use SCPs with AWS Organizations to manage location-specific permissions. Use AWS CloudTrail immutable logs to audit decision-making processes. Import a custom model into Amazon Bedrock and deploy the model to each Region.
C. Use Amazon S3 Object Lock with Region-specific S3 bucket policies. Pre-process the data points within the Region based on geographic origin before sending the data points to Amazon Bedrock. Use Amazon Macie to classify the data. Use AWS CloudTrail immutable logs to audit the decision-making processes.
D. Create separate AWS accounts for each Region with individual compliance frameworks. Use Amazon SageMaker AI with custom monitoring. Create manual compliance reports for each regulatory jurisdiction.
Answer: C
Explanation:
This scenario requires strict data residency, regional processing, classification, and auditable decision

trails, which Option C addresses using AWS-native governance services.
Region-specific Amazon S3 buckets enforce geographic data boundaries. Amazon S3 Object Lock ensures immutability of stored data and logs, supporting regulatory retention and non-repudiation requirements. Pre- processing data within the same Region before invoking Amazon Bedrock ensures that inference and data handling do not cross continental boundaries.
Amazon Macie provides managed, automated data classification for sensitive data types such as PII and financial records, fulfilling the classification requirement without custom tooling.
AWS CloudTrail immutable logs provide comprehensive audit trails of all API calls, model invocations, and data access events, ensuring traceability of AI decision-making processes.
Option A violates residency rules through cross-Region inference. Option B does not provide data classification. Option D introduces high operational overhead and relies on manual compliance reporting.
Therefore, Option C is the most compliant, scalable, and operationally efficient solution for regionally governed GenAI workloads.

NO.26 A book publishing company wants to build a book recommendation system that uses an AI assistant. The AI assistant will use ML to generate a list of recommended books from the company's book catalog. The system must suggest books based on conversations with customers.
The company stores the text of the books, customers' and editors' reviews of the books, and extracted book metadata in Amazon S3. The system must support low-latency responses and scale efficiently to handle more than 10,000 concurrent users.
Which solution will meet these requirements?

A. Use Amazon Bedrock Knowledge Bases to generate embeddings. Store the embeddings as a vector store in Amazon OpenSearch Service. Create an AWS Lambda function that queries the knowledge base. Configure Amazon API Gateway to invoke the Lambda function when handling user requests.

B. Use Amazon Bedrock Knowledge Bases to generate embeddings. Store the embeddings as a vector store in Amazon DynamoDB. Create an AWS Lambda function that queries the knowledge base. Configure Amazon API Gateway to invoke the Lambda function when handling user requests.

C. Use Amazon SageMaker AI to deploy a pre-trained model to build a personalized recommendation engine for books. Deploy the model as a SageMaker AI endpoint. Invoke the model endpoint by using Amazon API Gateway.

D. Create an Amazon Kendra GenAI Enterprise Edition index that uses the S3 connector to index the book catalog data stored in Amazon S3. Configure built-in FAQ in the Kendra index. Develop an AWS Lambda function that queries the Kendra index based on user conversations. Deploy Amazon API Gateway to expose this functionality and invoke the Lambda function.

Answer: A
Explanation:
Option A best meets the requirements because it directly implements a Retrieval Augmented Generation pattern for conversational recommendations using managed Amazon Bedrock capabilities and a scalable vector store. The company's source data already resides in Amazon S3, which aligns naturally with Amazon Bedrock Knowledge Bases ingestion workflows. A knowledge base can ingest book text, reviews, and metadata, generate embeddings using a supported embedding model, and persist those vectors in a purpose- built vector backend such as Amazon OpenSearch Service. This enables semantic retrieval that is well suited to conversation-driven intent, where user prompts are often descriptive and do not map cleanly to keyword filters.

The requirement to suggest books based on conversations implies the system must interpret natural language context and retrieve relevant passages, reviews, and metadata to ground the recommendation. Knowledge Bases provide managed orchestration for embedding creation and retrieval, which reduces development effort compared to building custom embedding pipelines. OpenSearch Service provides scalable vector search and k- nearest neighbors style similarity retrieval, which supports low-latency responses when properly indexed and sized.
For scaling to more than 10,000 concurrent users, the API layer design in option A is a common AWS pattern: Amazon API Gateway provides a managed front door with throttling and request handling, while AWS Lambda scales horizontally with demand and can invoke the knowledge base retrieval operations. This separates compute scaling from the vector store scaling and helps keep latency predictable under load.
Option B is not the best choice because DynamoDB is not the standard native vector store target for Amazon Bedrock Knowledge Bases in this context and would introduce additional implementation complexity around vector indexing and similarity search behavior. Option C requires substantial ML lifecycle work, model hosting, tuning, and continuous iteration to achieve quality recommendations at scale. Option D provides strong enterprise search, but it focuses on retrieval and FAQs rather than a managed RAG recommendation workflow grounded in embeddings and conversational context for generative responses.

NO.27 A company has set up Amazon Q Developer Pro licenses for all developers at the company. The company maintains a list of approved resources that developers must use when developing applications. The approved resources include internal libraries, proprietary algorithmic techniques, and sample code with approved styling.
A new team of developers is using Amazon Q Developer to develop a new Java-based application. The company must ensure that the new developer team uses the company's approved resources. The company does not want to make project-level modifications.
Which solution will meet these requirements?

A. Create a Git repository that contains all of the approved internal libraries, algorithms, and code samples.
Include this Git repository in the application project locally as part of the workspace. Ensure that the developers use the workspace context to retrieve suggestions from the Git repository.

B. In the project root folder, create a folder named amazonq/rules. Add the approved internal libraries, algorithms, and code samples to the folder.

C. Create a folder in the application project named rules. Store the guidelines and code in the folder for Amazon Q Developer to reference for code suggestions.

D. Create an Amazon Q Developer customization that includes the approved data sources. Ensure that the developers use the customization to develop the application.

Answer: D
Explanation:
Option D is the correct solution because Amazon Q Developer customizations are designed to incorporate organization-approved knowledge and coding guidance without requiring per-project changes. A customization can point Amazon Q Developer to curated internal sources such as approved libraries, coding standards, architectural patterns, and proprietary techniques. This allows the assistant's suggestions to align with company policies and preferred implementations consistently across teams and repositories.
The key requirement is that the company does not want to make project-level modifications. Options

A, B, and C all require adding files or repositories into the project workspace, which directly violates this constraint.
They also rely on developer behavior to "use workspace context," which is harder to enforce and can lead to inconsistent adherence to standards.
With a customization, the organization centrally manages and updates approved resources. This reduces operational overhead because updates to libraries, patterns, or guidelines propagate automatically to developers using the customization, without requiring changes to each project. This is especially valuable for a new team, where consistent enforcement of approved practices is important to reduce compliance risk, security issues, and inconsistent code style.
Additionally, customizations support governance by allowing the company to standardize how Amazon Q Developer responds, ensuring that suggestions reflect approved internal content rather than generic public patterns.
Therefore, Option D best satisfies the requirement for centralized enforcement of approved resources with minimal ongoing management and no project-level modifications.

NO.28 A company has a generative AI (GenAI) application that uses Amazon Bedrock to provide real-time responses to customer queries. The company has noticed intermittent failures with API calls to foundation models (FMs) during peak traffic periods.
The company needs a solution to handle transient errors and provide detailed observability into FM performance. The solution must prevent cascading failures during throttling events and provide distributed tracing across service boundaries to identify latency contributors. The solution must also enable correlation of performance issues with specific FM characteristics.
Which solution will meet these requirements?

A. Implement a custom retry mechanism with a fixed delay of 1 second between retries. Configure Amazon CloudWatch alarms to monitor the application's error rates and latency metrics.

B. Configure the AWS SDK with standard retry mode and exponential backoff with jitter. Use AWS X-Ray tracing with annotations to identify and filter service components.

C. Implement client-side caching of all FM responses. Add custom logging statements in the application code to record API call durations.

D. Configure the AWS SDK with adaptive retry mode. Use AWS CloudTrail distributed tracing to monitor throttling events.

Answer: B

Explanation:
Option B best meets the combined resiliency and observability requirements because it applies AWS-recommended retry behavior for transient throttling and enables true distributed tracing across service boundaries. During peak traffic, intermittent failures are commonly caused by throttling and other transient conditions. The AWS SDK standard retry mode provides exponential backoff with jitter, which reduces synchronized retry storms, prevents cascading failures, and improves overall system stability. Jitter is important because it spreads retry attempts over time, reducing load amplification during throttling events.
For observability, AWS X-Ray provides distributed tracing that follows a request across components such as API Gateway or load balancers, application services, and downstream calls to Amazon Bedrock. X-Ray can identify where latency is being introduced and which downstream call is contributing most to end-to-end response time. This is required to "identify latency contributors" and isolate performance issues under load.
The requirement also states that the company must correlate performance issues with specific FM

characteristics. X-Ray annotations are designed for this purpose: the application can annotate traces with the model ID, inference parameters, region, or inference profile used. This enables filtering and analysis (for example, comparing latency or error patterns by model, parameter set, or endpoint configuration) without building a separate telemetry system.
Option A's fixed-delay retries increase synchronized retry behavior and do not provide distributed tracing.
Option C does not prevent cascading failures and cannot provide cross-service tracing. Option D is incorrect because CloudTrail is an audit logging service and does not provide distributed tracing for request latency analysis.
Therefore, Option B provides the correct combination of resilient retries and deep, model-correlated distributed observability for Amazon Bedrock workloads.

NO.29 A company is designing a canary deployment strategy for a payment processing API. The system must support automated gradual traffic shifting between multiple Amazon Bedrock models based on real-time inference metrics, historical traffic patterns, and service health. The solution must be able to gradually increase traffic to new model versions. The system must increase traffic if metrics remain healthy and decrease traffic if the performance degrades below acceptable thresholds.
The company needs to comprehensively monitor inference latency and error rates during the deployment phase. The company must also be able to halt deployments and revert to a previous model version without any manual intervention.
Which solution will meet these requirements?

A. Use Amazon Bedrock with provisioned throughput to host model versions. Configure an Amazon EventBridge rule to invoke an AWS Step Functions workflow when a new model version is released. Configure the workflow to shift traffic in stages, wait for a specified time period, and invoke an AWS Lambda function to check Amazon CloudWatch performance metrics. Configure the workflow to increase traffic if metrics meet thresholds and to trigger a traffic rollback if performance metrics fall below thresholds.

B. Use AWS Lambda functions to invoke various Amazon Bedrock model versions. Use an Amazon API Gateway HTTP API with stage variables and weighted routing to shift traffic gradually. Use Amazon CloudWatch to monitor performance. Use external logic to adjust traffic and roll back if performance falls below thresholds.

C. Use Amazon SageMaker AI endpoint variants to represent multiple Amazon Bedrock model versions.
Use variant weights to shift traffic. Use Amazon CloudWatch and SageMaker Model Monitor to trigger rollbacks. Use EventBridge to roll back deployments if an anomaly is detected.

D. Use Amazon OpenSearch Service to track inference logs. Configure OpenSearch Service to invoke an AWS Systems Manager Automation runbook to update Amazon Bedrock model endpoints to shift traffic based on inference logs.

Answer: A
Explanation:
Option A is the most complete solution because it provides a fully automated canary strategy with staged traffic shifts, metric-based decisioning, and automatic rollback, all using managed AWS services. The requirement emphasizes automation, health-based traffic progression, and zero manual intervention to revert if performance degrades.

AWS Step Functions is well suited for orchestrating controlled deployment workflows with deterministic stages, waits, and conditional branches. By shifting traffic in stages and pausing for observation windows, the system can evaluate real-time inference latency and error rates before promoting more traffic to the new model version. Amazon CloudWatch provides the necessary real-time metrics and alarms for latency and error monitoring.
Invoking a Lambda function to evaluate CloudWatch metrics enables dynamic logic: increase traffic if thresholds remain healthy, reduce traffic or roll back if error rates rise or latency exceeds limits. Step Functions can halt the deployment by stopping progression or triggering rollback steps immediately, meeting the requirement for automated revert without human action.
Amazon EventBridge provides reliable automation triggers when a new model version is released, ensuring the deployment process is event-driven and repeatable.
Option B depends on "external logic," which introduces operational risk and does not guarantee automatic rollback without custom systems. Option C incorrectly uses SageMaker endpoint variants to represent Bedrock model versions, which is not the intended integration model. Option D is overly indirect and operationally complex, using log pipelines and automation runbooks instead of direct metric-based traffic control.
Therefore, Option A best meets the requirements for automated gradual traffic shifting, real-time monitoring, and automatic rollback for Amazon Bedrock model deployments in a canary strategy.

NO.30 A financial services company needs to build a document analysis system that uses Amazon Bedrock to process quarterly reports. The system must analyze financial data, perform sentiment analysis, and validate compliance across batches of reports. Each batch contains 5 reports. Each report requires multiple foundation model (FM) calls. The solution must finish the analysis within 10 seconds for each batch. Current sequential processing takes 45 seconds for each batch.
Which solution will meet these requirements?
A. Use AWS Lambda functions with provisioned concurrency to process each analysis type sequentially.
Configure the Lambda function timeouts to 10 seconds. Configure automatic retries with exponential backoff.
B. Use AWS Step Functions with a Parallel state to invoke separate AWS Lambda functions for each analysis type simultaneously. Configure Amazon Bedrock client timeouts. Use Amazon CloudWatch metrics to track execution time and model inference latency.
C. Create an Amazon SQS queue to buffer analysis requests. Deploy multiple AWS Lambda functions with reserved concurrency. Configure each Lambda function to process different aspects of each report sequentially and then combine the results.
D. Deploy an Amazon ECS cluster that runs containers that process each report sequentially. Use a load balancer to distribute batch workloads. Configure an auto-scaling policy based on CPU utilization.
Answer: B
Explanation:
Option B is the correct solution because it parallelizes independent foundation model inference tasks while maintaining orchestration, observability, and time-bound execution. AWS Generative AI best practices emphasize reducing end-to-end latency by parallelizing independent inference calls rather than scaling individual calls vertically.
In this scenario, each report requires multiple independent analyses such as financial extraction,

sentiment analysis, and compliance validation. These tasks do not depend on each other's output, making them ideal candidates for parallel execution. AWS Step Functions provides a Parallel state that can invoke multiple AWS Lambda functions simultaneously, drastically reducing total processing time compared to sequential execution.
By invoking Amazon Bedrock from separate Lambda functions in parallel, the system can reduce batch execution time from 45 seconds to well under the 10-second requirement, assuming each inference call remains within acceptable latency bounds. Step Functions also provide built-in error handling, retries, and state tracking, which improves reliability without increasing complexity. CloudWatch metrics allow teams to monitor both workflow execution time and individual model inference latency, enabling performance tuning and operational visibility. Configuring client-side timeouts ensures that slow or failed model invocations do not block the entire batch.
Option A still processes tasks sequentially and therefore cannot meet the strict latency requirement.
Option C introduces queuing delays and sequential processing within each report, which increases total execution time.
Option D relies on container-based sequential processing and adds unnecessary operational overhead for a workload that is event-driven and latency-sensitive.
Therefore, Option B best meets the performance, scalability, and operational efficiency requirements for high- speed batch document analysis using Amazon Bedrock.

NO.31 A company provides a service that helps users from around the world discover new restaurants. The service has 50 million monthly active users. The company wants to implement a semantic search solution across a database that contains 20 million restaurants and 200 million reviews. The company currently stores the data in a PostgreSQL database.
The solution must support complex natural language queries and return results for at least 95% of queries within 500 ms. The solution must maintain data freshness for restaurant details that update hourly. The solution must also scale cost-effectively during peak usage periods.
Which solution will meet these requirements with the LEAST development effort?

A. Migrate the restaurant data to Amazon OpenSearch Service. Implement keyword-based search rules that use custom analyzers and relevance tuning to find restaurants based on attributes such as cuisine type, feature, and location. Create Amazon API Gateway HTTP API endpoints to transform user queries into structured search parameters.

B. Migrate the restaurant data to Amazon OpenSearch Service. Use a foundation model (FM) in Amazon Bedrock to generate vector embeddings from restaurant descriptions, reviews, and menu items. When users submit natural language queries, convert the queries to embeddings by using the same FM.
Perform k-nearest neighbors (k-NN) searches to find semantically similar results.

C. Keep the restaurant data in PostgreSQL and implement a pgvector extension. Use a foundation model (FM) in Amazon Bedrock to generate vector embeddings from restaurant data. Store the vector embeddings directly in PostgreSQL. Create an AWS Lambda function to convert natural language queries to vector representations by using the same FM. Configure the Lambda function to perform similarity searches within the database.

D. Migrate the restaurant data to an Amazon Bedrock knowledge base by using a custom ingestion pipeline. Configure the knowledge base to automatically generate embeddings from restaurant information. Use the Amazon Bedrock Retrieve API with built-in vector search capabilities to query the knowledge base directly by using natural language input.

Answer: D
Explanation:
Option D requires the least development effort because it uses a managed retrieval workflow that bundles the most time-consuming parts of semantic search: embedding generation, vector indexing, and natural language retrieval. With an Amazon Bedrock knowledge base, the application does not need to implement and operate separate services to (1) generate embeddings for hundreds of millions of records, (2) store and manage vectors, (3) build query-time embedding conversion logic, and (4) implement k-NN search orchestration.
Instead, the knowledge base is configured to automatically create embeddings during ingestion, and the application queries it using the Amazon Bedrock Retrieve API, which accepts natural language input and performs the vector search as a managed capability.
The performance requirement (95% of queries within 500 ms) is best served by a purpose-built vector search backend rather than running similarity search directly inside a transactional PostgreSQL system at this scale.
A knowledge base is designed for retrieval patterns and can be backed by scalable vector stores, which helps meet latency goals under heavy concurrency. The hourly freshness requirement maps naturally to ingestion updates: the pipeline can re-ingest updated restaurant details on a schedule so the knowledge base remains current without building custom re-embedding workflows in application code.
Cost-effective scaling during peak periods is also easier with a managed retrieval layer because scaling the retrieval workload is separated from the operational database. This avoids overprovisioning PostgreSQL for peak semantic-search traffic and reduces the engineering effort to tune performance, sharding, indexing, and retry logic.
Options B and C can work, but they require the team to build and maintain embedding pipelines, query embedding generation, vector index management, and operational scaling strategies. Option A does not provide semantic search because it relies on keyword-based matching rather than embeddings.

NO.32 Example Corp provides a personalized video generation service that millions of enterprise customers use.
Customers generate marketing videos by submitting prompts to the company's proprietary generative AI (GenAI) model. To improve output relevance and personalization, Example Corp wants to enhance the prompts by using customer-specific context such as product preferences, customer attributes, and business history.
The customers have strict data governance requirements. The customers must retain full ownership and control over their own data. The customers do not require real-time access. However, semantic accuracy must be high and retrieval latency must remain low to support customer experience use cases.
Example Corp wants to minimize architectural complexity in its integration pattern. Example Corp does not want to deploy and manage services in each customer's environment unless necessary.
Which solution will meet these requirements?

A. Ensure that each customer sets up an Amazon Q Business index that includes the customer's internal data. Ensure that each customer designates Example Corp as a data accessor to allow Example Corp to retrieve relevant content by using a secure API to enrich prompts at runtime.

B. Use federated search with Model Context Protocol (MCP) by deploying real-time MCP servers for each customer. Retrieve data in real time during prompt generation.

C. Ensure that each customer configures an Amazon Bedrock knowledge base. Allow cross-account querying so Example Corp can retrieve structured data for prompt augmentation.

D. Configure Amazon Kendra to crawl customer data sources. Share the resulting indexes across accounts so Example Corp can query each customer's Amazon Kendra index to retrieve augmentation data.

Answer: A

Explanation:

Option A is the correct solution because Amazon Q Business is explicitly designed to provide secure, governed access to enterprise data while preserving customer ownership and control. Each customer maintains their own Amazon Q Business index, which ensures that data never leaves the customer's control boundary unless explicitly shared through approved access mechanisms.

By designating Example Corp as a data accessor, customers can allow controlled, auditable access to their indexed content through secure APIs. This model satisfies strict data governance requirements, including data ownership, access transparency, and revocation capability. Customers do not need to expose raw data or deploy infrastructure in Example Corp's environment.

Amazon Q Business provides high semantic accuracy through managed indexing, ranking, and retrieval optimizations. Because real-time access is not required, this approach avoids the complexity and latency challenges of live federated retrieval while still delivering fast query performance suitable for customer experience use cases.

Option B introduces unnecessary operational complexity by requiring real-time MCP servers per customer.

Option C requires customers to manage Amazon Bedrock knowledge bases and enable cross-account access, which increases integration complexity and governance risk. Option D requires shared Amazon Kendra indexes across accounts, which complicates access control and data ownership boundaries.

Therefore, Option A provides the cleanest, lowest-overhead architecture that meets data governance, accuracy, performance, and scalability requirements while minimizing operational burden for both Example Corp and its customers.

NO.33 An enterprise application uses an Amazon Bedrock foundation model (FM) to process and analyze 50 to 200 pages of technical documents. Users are experiencing inconsistent responses and receiving truncated outputs when processing documents that exceed the FM's context window limits .

Which solution will resolve this problem?

A. Configure fixed-size chunking at 4,000 tokens for each chunk with 20% overlap. Use application-level logic to link multiple chunks sequentially until the FM's maximum context window of 200,000 tokens is reached before making inference calls.

B. Use hierarchical chunking with parent chunks of 8,000 tokens and child chunks of 2,000 tokens. Use Amazon Bedrock Knowledge Bases built-in retrieval to automatically select relevant parent chunks based on query context. Configure overlap tokens to maintain semantic continuity.

C. Use semantic chunking with a breakpoint percentile threshold of 95% and a buffer size of 3 sentences.

Use the RetrieveAndGenerate API to dynamically select the most relevant chunks based on embedding similarity scores.

D. Create a pre-processing AWS Lambda function that analyzes document token count by using the

FM's tokenizer. Configure the Lambda function to split documents into equal segments that fit within 80% of the context window. Configure the Lambda function to process each segment independently before aggregating the results.

Answer: C

Explanation:

Option C directly addresses the root cause of truncated and inconsistent responses by using AWS-recommended semantic chunking and dynamic retrieval rather than static or sequential chunk processing.

Amazon Bedrock documentation emphasizes that foundation models have fixed context windows and that sending oversized or poorly structured input can lead to truncation, loss of context, and degraded output quality.

Semantic chunking breaks documents based on meaning instead of fixed token counts. By using a breakpoint percentile threshold and sentence buffers, the content remains coherent and semantically complete. This approach reduces the likelihood that important concepts are split across chunks, which is a common cause of inconsistent summarization results.

The RetrieveAndGenerate API is designed specifically to handle large documents that exceed a model's context window. Instead of forcing all content into a single inference call, the API generates embeddings for chunks and dynamically selects only the most relevant chunks based on similarity to the user query. This ensures that the FM receives only high-value context while staying within its context window limits.

Option A is ineffective because chaining chunks sequentially does not align with how FMs process context and risks exceeding context limits or introducing irrelevant information. Option B improves structure but still relies on larger parent chunks, which can lead to inefficiencies when processing very large documents. Option D processes segments independently, which often causes loss of global context and inconsistent summaries.

Therefore, Option C is the most robust, AWS-aligned solution for resolving truncation and consistency issues when processing large technical documents with Amazon Bedrock.

NO.34 A medical company is building a generative AI (GenAI) application that uses Retrieval Augmented Generation (RAG) to provide evidence-based medical information. The application uses Amazon OpenSearch Service to retrieve vector embeddings. Users report that searches frequently miss results that contain exact medical terms and acronyms and return too many semantically similar but irrelevant documents. The company needs to improve retrieval quality and maintain low end-user latency, even as the document collection grows to millions of documents.

Which solution will meet these requirements with the LEAST operational overhead?

A. Configure hybrid search by combining vector similarity with keyword matching to improve semantic understanding and exact term and acronym matching.

B. Increase the dimensions of the vector embeddings from 384 to 1536. Use a post-processing AWS Lambda function to filter out irrelevant results after retrieval.

C. Replace OpenSearch Service with Amazon Kendra. Use query expansion to handle medical acronyms and terminology variants during pre-processing.

D. Implement a two-stage retrieval architecture in which initial vector search results are re-ranked by an ML model hosted on Amazon SageMaker.

Answer: A

Explanation:

Option A is the correct solution because hybrid search directly addresses the core retrieval failure modes while maintaining low latency and minimal operational overhead. In medical and scientific domains, exact terminology, abbreviations, and acronyms (for example, drug names, procedures, or conditions) are critical.
Pure vector similarity search often underweights these exact matches, leading to missed results and excessive semantically related but irrelevant documents.
Amazon OpenSearch Service natively supports hybrid search, which combines keyword-based retrieval (such as BM25) with vector similarity search. Keyword search ensures precise matching for exact terms and acronyms, while vector search captures semantic meaning and contextual similarity. By blending these approaches, the retrieval system improves both precision and recall without introducing additional infrastructure.
Hybrid search operates within the same OpenSearch index and query path, which preserves low end-user latency even at large scale. This is especially important as the document collection grows to millions of documents. Because OpenSearch handles scoring and ranking internally, no additional orchestration layers or post-processing steps are required.
Option B increases computational cost and latency while failing to address exact-term recall. Option C introduces a new service and ingestion pipeline, increasing operational overhead and latency. Option D adds model hosting, re-ranking infrastructure, and complexity that is unnecessary when OpenSearch provides native hybrid retrieval.
Therefore, Option A delivers the best balance of retrieval quality, scalability, latency, and operational simplicity for medical RAG workloads.

NO.35 A financial services company is developing a Retrieval Augmented Generation (RAG) application to help investment analysts query complex financial relationships across multiple investment vehicles, market sectors, and regulatory environments. The dataset contains highly interconnected entities that have multi-hop relationships. Analysts must examine relationships holistically to provide accurate investment guidance. The application must deliver comprehensive answers that capture indirect relationships between financial entities and must respond in less than 3 seconds.
Which solution will meet these requirements with the LEAST operational overhead?

A. Use Amazon Bedrock Knowledge Bases with GraphRAG and Amazon Neptune Analytics to store financial data. Analyze multi-hop relationships between entities and automatically identify related information across documents.

B. Use Amazon Bedrock Knowledge Bases and an Amazon OpenSearch Service vector store to implement custom relationship identification logic that uses AWS Lambda to query multiple vector embeddings in sequence.

C. Use Amazon OpenSearch Serverless vector search with k-nearest neighbor (k-NN). Implement manual relationship mapping in an application layer that runs on Amazon EC2 Auto Scaling.

D. Use Amazon DynamoDB to store financial data in a custom indexing system. Use AWS Lambda to query relevant records. Use Amazon SageMaker to generate responses.

Answer: A

Explanation:
Option A best satisfies the requirement to capture multi-hop, highly interconnected relationships with minimal operational overhead. Traditional vector similarity search excels at finding semantically similar text but is not optimized for reasoning over explicit entity-to-entity relationships, especially

when analysts need indirect, multi-hop connections (for example, fund # holding # issuer # sector # regulation). Graph-based retrieval is designed specifically for these kinds of relationship traversals. GraphRAG combines retrieval-augmented generation with graph-aware context selection. By representing entities and their relationships in a graph store, the system can traverse multiple hops to assemble a holistic set of relevant facts. This improves completeness and reduces the chance that the model misses indirect relationships that are essential for accurate investment guidance.
Amazon Neptune Analytics provides a managed graph analytics environment capable of efficiently traversing and analyzing complex relationship networks. When integrated with Amazon Bedrock Knowledge Bases, it reduces custom engineering by providing managed ingestion, retrieval, and orchestration patterns suitable for GenAI applications. This lowers operational overhead compared to building and maintaining custom multi- stage retrieval logic.
Meeting the sub-3-second requirement is also more feasible with a graph-optimized engine because multi-hop traversals can be executed efficiently compared to chaining multiple vector searches and joining results in an application layer. The managed nature of Knowledge Bases and Neptune Analytics reduces maintenance, scaling, and operational burden while enabling strong performance.
Option B and C require extensive custom logic and orchestration, increasing complexity and latency.
Option D is not designed for graph-style multi-hop exploration and would require significant custom indexing and retrieval logic.
Therefore, Option A is the most AWS-aligned and operationally efficient approach for multi-hop relationship- aware RAG with strong performance.

NO.36 A bank is developing a generative AI (GenAI)-powered AI assistant that uses Amazon Bedrock to assist the bank's website users with account inquiries and financial guidance. The bank must ensure that the AI assistant does not reveal any personally identifiable information (PII) in customer interactions.
The AI assistant must not send PII in prompts to the GenAI model. The AI assistant must not respond to customer requests to provide investment advice. The bank must collect audit logs of all customer interactions, including any images or documents that are transmitted during customer interactions.
Which solution will meet these requirements with the LEAST operational effort?

A. Use Amazon Macie to detect and redact PII in user inputs and in the model responses. Apply prompt engineering techniques to force the model to avoid investment advice topics. Use AWS CloudTrail to capture conversation logs.

B. Use an AWS Lambda function and Amazon Comprehend to detect and redact PII. Use Amazon Comprehend topic modeling to prevent the AI assistant from discussing investment advice topics. Set up custom metrics in Amazon CloudWatch to capture customer conversations.

C. Configure Amazon Bedrock guardrails to apply a sensitive information policy to detect and filter PII.
Set up a topic policy to ensure that the AI assistant avoids investment advice topics. Use the Converse API to log model invocations. Enable delivery and image logging to Amazon S3.

D. Use regex controls to match patterns for PII. Apply prompt engineering techniques to avoid returning PII or investment advice topics to customers. Enable model invocation logging, delivery logging, and image logging to Amazon S3.

Answer: C
Explanation:
Option C is the correct solution because Amazon Bedrock guardrails are purpose-built to enforce

defense-in- depth safety controls for GenAI applications with minimal operational overhead. Guardrails provide managed, policy-based enforcement that operates before prompts are sent to the foundation model and after responses are generated, which directly satisfies the requirement that PII must not be sent to the model and must not appear in outputs.
By configuring a sensitive information policy, the application can automatically detect and redact PII in user inputs and model responses without building custom preprocessing pipelines. This approach is more reliable and scalable than regex or prompt engineering techniques, which are brittle and error-prone for sensitive data handling.
The topic policy capability in Amazon Bedrock guardrails allows the bank to explicitly block investment advice topics, ensuring regulatory compliance. This policy-based approach is safer and more auditable than attempting to steer the model only through prompt instructions.
Using the Converse API enables structured, standardized interactions with the model and supports consistent logging of requests and responses. Enabling delivery logging and image logging to Amazon S3 ensures that all customer interactions, including documents and images, are captured in a durable, auditable storage layer.
This directly supports compliance, regulatory audits, and forensic analysis.
Option A incorrectly relies on Amazon Macie, which is designed for data-at-rest discovery rather than real- time conversational filtering. Option B introduces custom Lambda pipelines and topic modeling, increasing operational complexity. Option D relies on regex and prompt engineering, which do not meet financial-grade compliance standards.
Therefore, Option C delivers the strongest security, governance, and auditability with the least operational effort.

NO.37 A financial technology company is using Amazon Bedrock to build an assessment system for the company's customer service AI assistant. The AI assistant must provide financial recommendations that are factually accurate, compliant with financial regulations, and conversationally appropriate. The company needs to combine automated quality evaluations at scale with targeted human reviews of critical interactions.
What solution will meet these requirements?
A. Configure a pipeline in which financial experts manually score all responses for accuracy, compliance, and conversational quality. Use Amazon SageMaker notebooks to analyze results to identify improvement areas.
B. Configure Amazon Bedrock evaluations that use Anthropic Claude Sonnet as a judge model to assess response accuracy and appropriateness. Configure custom Amazon Bedrock guardrails to check responses for compliance with financial policies. Add Amazon Augmented AI (Amazon A2I) human reviews for flagged critical interactions.
C. Create an Amazon Lex bot to manage customer service interactions. Configure AWS Lambda functions to check responses against a static compliance database. Configure intents that call the Lambda functions. Add an additional intent to collect end-user reviews.
D. Configure Amazon CloudWatch to monitor response patterns from the AI assistant. Configure CloudWatch alerts for potential compliance violations. Establish a team of human evaluators to review flagged interactions.
Answer: B
Explanation:
Option B meets the requirement to combine scalable automated evaluation with targeted human

oversight using managed AWS GenAI capabilities. Amazon Bedrock evaluations enable systematic, repeatable quality assessment across large volumes of interactions. Using an LLM-as-a-judge approach with a strong evaluator model such as Anthropic Claude Sonnet allows the company to automatically score outputs for dimensions like factual accuracy, conversational appropriateness, and policy alignment. This directly supports "automated quality evaluations at scale" without building custom scoring models.
However, financial recommendations add higher risk because regulatory compliance requires additional enforcement beyond general quality scoring. Amazon Bedrock guardrails provide a dedicated policy enforcement layer that can block or intervene when responses violate compliance constraints. Guardrails are particularly important for preventing disallowed financial guidance patterns and ensuring consistent behavior across deployments.
The requirement also calls for "targeted human reviews of critical interactions." Amazon Augmented AI (A2I) is a managed human review service that supports routing specific items to human reviewers based on rules or confidence thresholds. In this design, the system can automatically send only high-risk or policy- flagged interactions to qualified financial experts for review, keeping human effort focused where it matters most while maintaining scale.
Option A is not scalable because it requires manual review of all responses. Option C relies on static rules and end-user feedback, which is insufficient for regulatory compliance and factual accuracy assurance. Option D provides monitoring but not structured quality evaluation or policy enforcement.
Therefore, Option B provides the most complete, AWS-aligned solution for scalable evaluation plus human oversight in a regulated financial context.

NO.38 A company is developing a customer communication platform that uses an AI assistant powered by an Amazon Bedrock foundation model (FM). The AI assistant summarizes customer messages and generates initial response drafts.
The company wants to use Amazon Comprehend to implement layered content filtering. The layered content filtering must prevent sharing of offensive content, protect customer privacy, and detect potential inappropriate advice solicitation. Inappropriate advice solicitation includes requests for unethical practices, harmful activities, or manipulative behaviors.
The solution must maintain acceptable overall response times, so all pre-processing filters must finish before the content reaches the FM.
Which solution will meet these requirements?

A. Use parallel processing with asynchronous API calls. Use toxicity detection for offensive content. Use prompt safety classification for inappropriate advice solicitation. Use personally identifiable information (PII) detection without redaction.

B. Use custom classification to build an FM that detects offensive content and inappropriate advice solicitation. Apply personally identifiable information (PII) detection as a secondary filter only when messages pass the custom classifier.

C. Deploy a multi-stage process. Configure the process to use prompt safety classification first, then toxicity detection on safe prompts only, and finally personally identifiable information (PII) detection in streaming mode. Route flagged messages through Amazon EventBridge for human review.

D. Use toxicity detection with thresholds configured to 0.5 for all categories. Use parallel processing for both prompt safety classification and personally identifiable information (PII) detection with entity redaction. Apply Amazon CloudWatch alarms to filter metrics.

Answer: D

Explanation:

Option D best satisfies all functional, performance, and governance requirements while minimizing architectural complexity. The requirement explicitly states that all filtering must complete before content reaches the foundation model, which rules out asynchronous or streaming-based approaches that could delay enforcement.

Amazon Comprehend supports toxicity detection, prompt safety classification, and PII detection with entity redaction as managed capabilities. Running these filters in parallel ensures low end-to-end latency, which is essential for customer-facing communication platforms. Parallel execution avoids the cumulative latency that would be introduced by sequential pipelines.

Toxicity detection identifies offensive or abusive content early. Prompt safety classification detects requests for unethical, harmful, or manipulative advice, which directly addresses inappropriate advice solicitation requirements. PII detection with entity redaction ensures that customer privacy is preserved before data is sent to the FM, preventing sensitive information from being processed or echoed in generated responses.

Configuring thresholds allows fine-grained control over sensitivity while maintaining acceptable false-positive rates. Using CloudWatch metrics and alarms enables continuous monitoring of filtering behavior and intervention rates without adding custom routing or human review pipelines that would slow responses.

Option A lacks PII redaction. Option B introduces unnecessary model-building complexity and delayed PII checks. Option C adds sequential latency and introduces human review routing, which violates the response- time requirement.

Therefore, Option D provides the most robust, performant, and AWS-aligned layered content filtering solution.

NO.39 A software company is using Amazon Q Business to build an AI assistant that allows employees to access company information and personal information by using natural language prompts. The company stores this information in an Amazon S3 bucket.

Each department in the company has a dedicated prefix in the S3 bucket. Each object name includes the S3 prefix of the department that it belongs to. Each department can belong to only a single group in AWS IAM Identity Center. Each employee belongs to a single department.

The company configures Amazon Q Business to access data stored in an S3 bucket as a data source. The company needs to ensure that the AI assistant respects access controls based on the user's IAM Identity Center group membership.

Which solution will meet this requirement with the LEAST operational overhead?

A. Create a JSON file named acl.json in each department folder. In each file, create access control entries that specify the IAM Identity Center group that should have access to that department's data. Indicate the location of the JSON file in the Access Control section of the data source settings.

B. Create a single JSON file named acl.json at the top level of the S3 bucket. Add access control entries that map each department's S3 prefix to its corresponding IAM Identity Center group. Indicate the location of the JSON file in the Access Control section of the data source settings.

C. For each IAM Identity Center group, create a separate permissions set that denies access to all prefixes in the S3 bucket. Add a StringNotEquals condition key to the permissions set for each group that specifies the department each group is associated with. Attach the permissions sets to the Identity Center groups.

D. Create a metadata file named metadata.json at the top level of the S3 bucket. Add an AccessControlList object to the file that specifies the S3 path of each department's prefix. Specify the IAM Identity Center group that should have access to each department's prefix. Reference the file location in the data source metadata settings.

Answer: B

Explanation:

Option B is the correct solution because Amazon Q Business natively supports access control lists (ACLs) for S3 data sources using a single, centralized JSON file that maps S3 prefixes to IAM Identity Center groups.

This approach directly aligns with the company's data organization model, where each department's data is stored under a distinct S3 prefix and each employee belongs to exactly one department group .

Using a single acl.json file at the bucket root minimizes operational overhead by centralizing access control logic in one location. Administrators can update department mappings without touching individual folders or changing IAM permissions, which simplifies governance and reduces the risk of configuration drift. Amazon Q Business automatically evaluates the user's IAM Identity Center group membership at query time and filters accessible documents accordingly.

Option A increases operational complexity by requiring a separate ACL file in every department folder, which becomes difficult to maintain as departments or prefixes change. Option C attempts to enforce access using IAM permissions sets, but Amazon Q Business access control for S3 data sources is not designed to be managed through IAM condition logic and would significantly increase complexity. Option D introduces a custom metadata structure that is not the supported mechanism for Amazon Q Business access enforcement.

Therefore, Option B provides the cleanest, most scalable, and AWS-recommended solution for enforcing department-based access control with the least operational effort.

NO.40 A company is building a generative AI (GenAI) application that processes financial reports and provides summaries for analysts. The application must run two compute environments. In one environment, AWS Lambda functions must use the Python SDK to analyze reports on demand. In the second environment, Amazon EKS containers must use the JavaScript SDK to batch process multiple reports on a schedule. The application must maintain conversational context throughout multi-turn interactions, use the same foundation model (FM) across environments, and ensure consistent authentication.

Which solution will meet these requirements?

A. Use the Amazon Bedrock InvokeModel API with a separate authentication method for each environment. Store conversation states in Amazon DynamoDB. Use custom I/O formatting logic for each programming language.

B. Use the Amazon Bedrock Converse API directly in both environments with a common authentication mechanism that uses IAM roles. Store conversation states in Amazon ElastiCache. Create programming language-specific wrappers for model parameters.

C. Create a centralized Amazon API Gateway REST API endpoint that handles all model interactions by using the InvokeModel API. Store interaction history in application process memory in each Lambda function or EKS container. Use environment variables to configure model parameters.

D. Use the Amazon Bedrock Converse API and IAM roles for authentication. Pass previous messages in the request messages array to maintain conversational context. Use programming language-

specific SDKs to establish consistent API interfaces.

Answer: D

Explanation:

Option D is the correct solution because the Amazon Bedrock Converse API is purpose-built for multi-turn conversational interactions and is designed to work consistently across SDKs and compute environments. The Converse API standardizes how messages, roles, and context are represented, which ensures consistent behavior whether the application is running in AWS Lambda with Python or in Amazon EKS with JavaScript.

By passing previous messages in the messages array, the application explicitly maintains conversational context across turns without relying on external state stores. This approach is recommended by AWS for conversational GenAI workflows because it avoids state synchronization complexity and ensures deterministic model behavior across environments.

Using IAM roles for authentication provides a single, consistent security model for both Lambda and EKS.

IAM roles integrate natively with AWS SDKs, eliminating the need for custom authentication logic or environment-specific credentials. This aligns with AWS best practices for least privilege and simplifies governance.

Option A introduces inconsistent authentication and custom formatting logic, increasing complexity.

Option B unnecessarily introduces ElastiCache for state management, which is not required when using the Converse API correctly. Option C stores state in process memory, which is unsafe and unreliable for serverless and containerized workloads.

Therefore, Option D best satisfies the requirements for conversational consistency, multi-environment support, shared model usage, and consistent authentication with minimal operational overhead.

NO.41 A retail company is using Amazon Bedrock to develop a customer service AI assistant. Analysis shows that

70% of customer inquiries are simple product questions that a smaller model can effectively handle. However,

30% of inquiries are complex return policy questions that require advanced reasoning.

The company wants to implement a cost-effective model selection framework to automatically route customer inquiries to appropriate models based on inquiry complexity. The framework must maintain high customer satisfaction and minimize response latency.

Which solution will meet these requirements with the LEAST implementation effort?

A. Create a multi-stage architecture that uses a small foundation model (FM) to classify the complexity of each inquiry. Route simple inquiries to a smaller, more cost-effective model. Route complex inquiries to a larger, more capable model. Use AWS Lambda functions to handle routing logic.

B. Use Amazon Bedrock intelligent prompt routing to automatically analyze inquiries. Route simple product inquiries to smaller models and route complex return policy inquiries to more capable larger models.

C. Implement a single-model solution that uses an Amazon Bedrock mid-sized foundation model (FM) with on-demand pricing. Include special instructions in model prompts to handle both simple and complex inquiries by using the same model.

D. Create separate Amazon Bedrock endpoints for simple and complex inquiries. Implement a rule-

based routing system based on keyword detection. Use on-demand pricing for the smaller model and provisioned throughput for the larger model.

Answer: B

Explanation:

Option B is the correct solution because it leverages native Amazon Bedrock intelligent prompt routing, which is specifically designed to reduce cost and complexity in multi-model GenAI architectures. Intelligent prompt routing automatically analyzes incoming prompts and selects the most appropriate foundation model based on prompt characteristics and complexity-without requiring custom classification logic or orchestration code.

This approach directly meets the requirement for least implementation effort. The company does not need to deploy additional Lambda functions, maintain routing rules, or manage separate classification stages. Routing decisions are handled by Bedrock, which simplifies architecture and reduces operational risk.

By routing the majority (70%) of simple product inquiries to smaller, lower-cost models, the company minimizes inference cost and latency. More complex return policy inquiries are automatically routed to larger models that provide better reasoning capabilities, preserving response quality and customer satisfaction.

Because routing is handled inline by Bedrock, response latency remains low compared to multi-stage architectures that require an additional classification model call before inference. This is critical for customer service scenarios where responsiveness directly impacts satisfaction.

Option A introduces additional inference steps and custom logic. Option C increases cost by overusing a mid- sized model for all queries. Option D relies on brittle keyword rules and increases operational overhead through endpoint management.

Therefore, Option B delivers the optimal balance of cost efficiency, performance, and simplicity for dynamic model selection in Amazon Bedrock.

NO.42 A medical device company wants to feed reports of medical procedures that used the company's devices into an AI assistant. To protect patient privacy, the AI assistant must expose patient personally identifiable information (PII) only to surgeons. The AI assistant must redact PII for engineers. The AI assistant must reference only medical reports that are less than 3 years old.

The company stores reports in an Amazon S3 bucket as soon as each report is published. The company has already set up an Amazon Bedrock Knowledge Bases. The AI assistant uses Amazon Cognito to authenticate users.

Which solution will meet these requirements?

A. Enable Amazon Macie PII detection on the S3 bucket. Use an S3 trigger to invoke an AWS Lambda function that redacts PII from the reports. Configure the Lambda function to delete outdated documents and invoke knowledge base syncing.

B. Invoke an AWS Lambda function to sync the S3 bucket and the knowledge base when a new report is uploaded. Use a second Lambda function with Amazon Comprehend to redact PII for engineers. Use S3 Lifecycle rules to remove reports older than 3 years.

C. Set up an S3 Lifecycle configuration to remove reports that are older than 3 years. Schedule an AWS Lambda function to run daily syncs between the bucket and the knowledge base. When users interact with the AI assistant, apply a guardrail configuration selected based on the user's Cognito user group to redact PII from responses when required.

D. Create a second knowledge base. Use Lambda and Amazon Comprehend to redact PII before

syncing to the second knowledge base. Route users to the appropriate knowledge base based on Cognito group membership.

Answer: C

Explanation:

Option C is the correct solution because it enforces privacy controls at inference time, not at ingestion time, which is required when different user roles require different visibility into the same underlying data.

Using an S3 Lifecycle configuration ensures that documents older than 3 years are automatically removed, guaranteeing that the knowledge base references only compliant, recent medical reports. Scheduling Lambda- based syncs keeps the knowledge base aligned with the bucket contents without introducing complex per- upload orchestration.

The most important requirement is role-based PII exposure. Amazon Bedrock guardrails support dynamic application at inference time, allowing the system to select a guardrail configuration based on the authenticated user's Amazon Cognito group. Surgeons can receive full responses, while engineers receive responses with PII masked-without duplicating data or maintaining multiple knowledge bases.

This approach preserves a single source of truth for medical reports while enforcing privacy through response- level controls. It also maintains full auditability of access and redaction behavior.

Option A permanently removes PII and violates surgeon access requirements. Option B redacts data inconsistently and couples privacy logic to ingestion. Option D doubles storage, increases cost, and introduces data drift risk.

Therefore, Option C best meets privacy, compliance, scalability, and operational efficiency requirements.

NO.43 A publishing company is developing a chat assistant that uses a containerized large language model (LLM) that runs on Amazon SageMaker AI. The architecture consists of an Amazon API Gateway REST API that routes user requests to an AWS Lambda function. The Lambda function invokes a SageMaker AI real-time endpoint that hosts the LLM.

Users report uneven response times. Analytics show that a high number of chats are abandoned after 2 seconds of waiting for the first token. The company wants a solution to ensure that p95 latency is under 800 ms for interactive requests to the chat assistant.

Which combination of solutions will meet this requirement? (Select TWO.)

A. Enable model preload upon container startup. Implement dynamic batching to process multiple user requests together in a single inference pass.

B. Select a larger GPU instance type for the SageMaker AI endpoint. Set the minimum number of instances to 0. Continue to perform per-request processing. Lazily load model weights on the first request.

C. Switch to a multi-model endpoint. Use lazy loading without request batching.

D. Set the minimum number of instances to greater than 0. Enable response streaming.

E. Switch to Amazon SageMaker Asynchronous Inference for all requests. Store requests in an Amazon S3 bucket. Set the minimum number of instances to 0.

Answer: A D

Explanation:

The correct answers are A and D because they directly reduce time-to-first-token and stabilize p95 latency for interactive, real-time chat workloads hosted on Amazon SageMaker AI real-time

endpoints.
Option D addresses the biggest driver of uneven latency: cold starts and scale-to-zero behavior. By setting the minimum number of instances to greater than 0, the endpoint always has warm capacity and loaded runtime resources, eliminating the first-request penalty that causes users to wait multiple seconds. Enabling response streaming improves perceived latency by returning the first tokens as soon as they are generated rather than waiting for the complete response. This directly targets the abandonment problem described (users leaving after waiting for the first token).
Option A further improves p95 latency and throughput by removing model loading overhead during inference and improving GPU utilization. Preloading model weights during container startup ensures the model is ready before traffic arrives and avoids unpredictable on-demand weight loading.
Dynamic batching increases efficiency by grouping compatible requests into a single inference pass, reducing per-request overhead and improving GPU saturation. When tuned properly for interactive workloads, batching can reduce tail latency while preserving responsiveness by enforcing small batch windows.
Option B makes latency worse because setting minimum instances to 0 and lazily loading weights guarantees cold-start delays and unpredictable first-token performance. Option C similarly increases cold-start behavior through lazy loading and offers no batching benefits. Option E is designed for non-interactive workloads and introduces queueing and storage latency, which conflicts with the 800 ms p95 requirement for interactive chat.
Therefore, A and D are the best combination to achieve consistently low p95 latency and fast first-token streaming for a SageMaker-hosted chat assistant.

NO.44 A company is building an AI advisory application by using Amazon Bedrock. The application will provide recommendations to customers. The company needs the application to explain its reasoning process and cite specific sources for data. The application must retrieve information from company data sources and show step- by-step reasoning for recommendations. The application must also link data claims to source documents and maintain response latency under 3 seconds.
Which solution will meet these requirements with the LEAST operational overhead?

A. Use Amazon Bedrock Knowledge Bases with source attribution enabled. Use the Anthropic Claude Messages API with RAG to set high-relevance thresholds for source documents. Store reasoning and citations in Amazon S3 for auditing purposes.

B. Use Amazon Bedrock with Anthropic Claude models and extended thinking. Configure a 4,000-token thinking budget. Store reasoning traces and citations in Amazon DynamoDB for auditing purposes.

C. Configure Amazon SageMaker AI with a custom Anthropic Claude model. Use the model's reasoning parameter and AWS Lambda to process responses. Add source citations from a separate Amazon RDS database.

D. Use Amazon Bedrock with Anthropic Claude models and chain-of-thought reasoning. Configure custom retrieval tracking with the Amazon Bedrock Knowledge Bases API. Use Amazon CloudWatch to monitor response latency metrics.

Answer: A
Explanation:
Option A is the best solution because it natively delivers retrieval grounding, source attribution, and low operational overhead through Amazon Bedrock Knowledge Bases. The key requirements are: retrieve from company data sources, cite sources, link claims to source documents, and keep latency

under 3 seconds.
Knowledge Bases are a managed RAG capability that handles document ingestion, chunking, embeddings, retrieval, and assembly of context for model generation. This eliminates the need to build and maintain custom retrieval infrastructure.
Source attribution is crucial: the application must "link data claims to source documents." When source attribution is enabled, the RAG pipeline can return references to the underlying documents and segments used for generation. This enables traceable citations that can be surfaced to end users and used for internal auditing.
Using the Anthropic Claude Messages API (or equivalent conversational interface) with RAG allows the application to generate recommendations grounded in retrieved context while keeping responses conversational. Setting relevance thresholds helps reduce noisy retrieval, which supports both accuracy and latency targets by limiting the context passed to the model.
Storing reasoning and citations in Amazon S3 supports audit and retention needs with minimal operational burden. While the prompt may request step-by-step reasoning, AWS best practice is to produce user-facing explanations that are faithful and attributable without exposing internal reasoning traces unnecessarily. With source-grounded outputs, the system can provide concise rationale tied to citations while maintaining fast response times.
Option B emphasizes extended thinking, which increases latency and does not ensure source linkage. Option C adds significant operational overhead through custom model hosting and separate citation systems. Option D requires more custom tracking work than A while not improving retrieval attribution beyond what Knowledge Bases already provide.
Therefore, Option A best meets the requirements with the least operational overhead.

NO.45 A company is using AWS Lambda and REST APIs to build a reasoning agent to automate support workflows.
The system must preserve memory across interactions, share relevant agent state, and support event-driven invocation and synchronous invocation. The system must also enforce access control and session-based permissions.
Which combination of steps provides the MOST scalable solution? (Select TWO.)
A. Use Amazon Bedrock AgentCore to manage memory and session-aware reasoning. Deploy the agent with built-in identity support, event handling, and observability.
B. Register the Lambda functions and REST APIs as actions by using Amazon API Gateway and Amazon EventBridge. Enable Amazon Bedrock AgentCore to invoke the Lambda functions and REST APIs without custom orchestration code.
C. Use Amazon Bedrock Agents for reasoning and conversation management. Use AWS Step Functions and Amazon SQS for orchestration. Store agent state in Amazon DynamoDB.
D. Deploy the reasoning logic as a container on Amazon ECS behind API Gateway. Use Amazon Aurora to store memory and identity data.
E. Build a custom RAG pipeline by using Amazon Kendra and Amazon Bedrock. Use AWS Lambda to orchestrate tool invocations. Store agent state in Amazon S3.
Answer: A B
Explanation:
The combination of Options A and B provides the most scalable and AWS-native architecture for building reasoning agents with persistent memory, session awareness, secure access control, and flexible invocation models.

Amazon Bedrock AgentCore is purpose-built to manage agent memory, session context, and identity-aware reasoning across interactions. It eliminates the need for developers to manually store and retrieve agent state, manage session lifecycles, or implement custom memory layers. AgentCore natively supports both synchronous requests and event-driven execution, making it ideal for support workflow automation.
Option B complements AgentCore by enabling seamless tool invocation. By registering AWS Lambda functions and REST APIs as agent actions through API Gateway and EventBridge, the agent can invoke tools reactively or synchronously without custom orchestration code. EventBridge enables event-driven execution, while API Gateway supports synchronous request-response patterns.
This combination provides built-in security, observability, and scaling, while avoiding the operational burden of managing queues, databases, or custom workflow engines.
Option C introduces unnecessary orchestration complexity. Option D increases infrastructure management and cost. Option E stores agent state in S3, which is not suitable for low-latency, session-based reasoning.
Therefore, A and B together deliver the most scalable, secure, and low-overhead solution for production- grade reasoning agents on AWS.

NO.46 A specialty coffee company has a mobile app that generates personalized coffee roast profiles by using Amazon Bedrock with a three-stage prompt chain. The prompt chain converts user inputs into structured metadata, retrieves relevant logs for coffee roasts, and generates a personalized roast recommendation for each customer.
Users in multiple AWS Regions report inconsistent roast recommendations for identical inputs, slow inference during the retrieval step, and unsafe recommendations such as brewing at excessively high temperatures. The company must improve the stability of outputs for repeated inputs. The company must also improve app performance and the safety of the app's outputs. The updated solution must ensure 99.5% output consistency for identical inputs and achieve inference latency of less than 1 second. The solution must also block unsafe or hallucinated recommendations by using validated safety controls.
Which solution will meet these requirements?

A. Deploy Amazon Bedrock with provisioned throughput to stabilize inference latency. Apply Amazon Bedrock guardrails with semantic denial rules to block unsafe outputs. Use Amazon Bedrock Prompt Management to manage prompts by using approval workflows.

B. Use Amazon Bedrock Agents to manage chaining. Log model inputs and outputs to Amazon CloudWatch Logs. Use logs from CloudWatch to perform A/B testing for prompt versions.

C. Cache prompt results in Amazon ElastiCache. Use AWS Lambda functions to pre-process metadata and to trace end-to-end latency. Use AWS X-Ray to identify and remediate performance bottlenecks.

D. Use Amazon Kendra to improve roast log retrieval accuracy. Store normalized prompt metadata within Amazon DynamoDB. Use AWS Step Functions to orchestrate multi-step prompts.

Answer: A
Explanation:
Option A is the only choice that simultaneously addresses all three requirements: (1) higher output consistency for identical inputs, (2) sub-1-second performance, and (3) validated safety controls that block unsafe or hallucinated recommendations.
Provisioned throughput in Amazon Bedrock reserves capacity for the chosen model, which helps stabilize latency and reduces the chance of throttling or variable response times across Regions. This is important for a mobile app with strict latency goals and users distributed across multiple Regions.

While provisioned throughput primarily improves performance predictability, it also reduces variability caused by contention during peak demand.
Amazon Bedrock guardrails provide validated safety controls to filter or block unsafe content. Semantic denial rules are appropriate for preventing dangerous brewing guidance (for example, excessively high temperatures) and for reducing hallucinated instructions that violate safety policies. Guardrails can be enforced consistently regardless of prompt-chain complexity, providing a uniform safety layer around the model outputs.
Amazon Bedrock Prompt Management supports controlled prompt versioning and approval workflows. By standardizing prompts, controlling changes, and ensuring the same prompt version is used for identical inputs, the company improves output stability and reduces drift caused by unmanaged prompt edits. Combined with strict configuration control (including fixed inference parameters such as temperature where appropriate), this improves repeatability and increases the likelihood of achieving the 99.5% consistency target.
Option B improves observability and experimentation but does not provide strong safety enforcement or latency stabilization. Option C improves performance through caching and tracing but does not provide validated safety controls and does not directly address cross-Region output consistency. Option D may improve retrieval but does not enforce safety controls or ensure repeatable outputs.
Therefore, Option A best meets the stability, performance, and safety requirements using AWS-native controls.

NO.47 A company uses an AI assistant application to summarize the company's website content and provide information to customers. The company plans to use Amazon Bedrock to give the application access to a foundation model (FM).
The company needs to deploy the AI assistant application to a development environment and a production environment. The solution must integrate the environments with the FM. The company wants to test the effectiveness of various FMs in each environment. The solution must provide product owners with the ability to easily switch between FMs for testing purposes in each environment.
Which solution will meet these requirements?

A. Create one AWS CDK application. Create multiple pipelines in AWS CodePipeline. Configure each pipeline to have its own settings for each FM. Configure the application to invoke the Amazon Bedrock FMs by using the aws_bedrock.ProvisionedModel.fromProvisionedModelArn() method.

B. Create a separate AWS CDK application for each environment. Configure the applications to invoke the Amazon Bedrock FMs by using the aws_bedrock.FoundationModel.fromFoundationModelId() method. Create a separate pipeline in AWS CodePipeline for each environment.

C. Create one AWS CDK application. Configure the application to invoke the Amazon Bedrock FMs by using the aws_bedrock.FoundationModel.fromFoundationModelId() method. Create a pipeline in AWS CodePipeline that has a deployment stage for each environment that uses AWS CodeBuild deploy actions.

D. Create one AWS CDK application for the production environment. Configure the application to invoke the Amazon Bedrock FMs by using the aws_bedrock.ProvisionedModel.fromProvisionedModelArn() method. Create a pipeline in AWS CodePipeline. Configure the pipeline to deploy to the production environment by using an AWS CodeBuild deploy action. For the development environment, manually recreate the resources by

referring to the production application code.

Answer: C

Explanation:

Option C best satisfies the requirement for flexible FM testing across environments while minimizing operational complexity and aligning with AWS-recommended deployment practices. Amazon Bedrock supports invoking on-demand foundation models through the FoundationModel abstraction, which allows applications to dynamically reference different models without requiring dedicated provisioned capacity. This is ideal for experimentation and A/B testing in both development and production environments.

Using a single AWS CDK application ensures infrastructure consistency and reduces duplication. Environment-specific configuration, such as selecting different foundation model IDs, can be externalized through parameters, context variables, or environment-specific configuration files. This allows product owners to easily switch between FMs in each environment without modifying application logic.

A single AWS CodePipeline with distinct deployment stages for development and production is an AWS best practice for multi-environment deployments. It enforces consistent build and deployment steps while still allowing environment-level customization. AWS CodeBuild deploy actions enable automated, repeatable deployments, reducing manual errors and improving governance.

Option A increases complexity by introducing multiple pipelines and relies on provisioned models, which are not necessary for FM evaluation and experimentation. Provisioned throughput is better suited for predictable, high-volume production workloads rather than frequent model switching.

Option B creates unnecessary operational overhead by duplicating CDK applications and pipelines, making long-term maintenance more difficult.

Option D directly conflicts with infrastructure-as-code best practices by manually recreating development resources, which increases configuration drift and reduces reliability.

Therefore, Option C provides the most flexible, scalable, and AWS-aligned solution for testing and switching foundation models across development and production environments.

NO.48 A financial services company is building a customer support application that retrieves relevant financial regulation documents from a database based on semantic similarity to user queries. The application must integrate with Amazon Bedrock to generate responses. The application must search documents in English, Spanish, and Portuguese. The application must filter documents by metadata such as publication date, regulatory agency, and document type.

The database stores approximately 10 million document embeddings. To minimize operational overhead, the company wants a solution that minimizes management and maintenance effort while providing low-latency responses for real-time customer interactions.

Which solution will meet these requirements?

A. Use Amazon OpenSearch Serverless to provide vector search capabilities and metadata filtering. Integrate with Amazon Bedrock Knowledge Bases to enable Retrieval Augmented Generation (RAG) using an Anthropic Claude foundation model.

B. Deploy an Amazon Aurora PostgreSQL database with the pgvector extension. Store embeddings and metadata in tables. Use SQL queries for similarity search and send results to Amazon Bedrock for response generation.

C. Use Amazon S3 Vectors to configure a vector index and non-filterable metadata fields. Integrate S3 Vectors with Amazon Bedrock for RAG.

D. Set up an Amazon Neptune Analytics database with a vector index. Use graph-based retrieval and

Amazon Bedrock for response generation.

Answer: A

Explanation:

Option A is the optimal solution because it provides scalable semantic search, rich metadata filtering, and tight integration with Amazon Bedrock while minimizing operational overhead. Amazon OpenSearch Serverless is designed for high-volume, low-latency search workloads and removes the need to manage clusters, capacity planning, or scaling policies.

With support for vector search and structured metadata filtering, OpenSearch Serverless enables efficient similarity search across 10 million embeddings while applying constraints such as language, publication date, regulatory agency, and document type. This is critical for financial services use cases where relevance and compliance depend on precise filtering.

Integrating OpenSearch Serverless with Amazon Bedrock Knowledge Bases enables a fully managed RAG workflow. The knowledge base handles embedding generation, retrieval, and context assembly, while Amazon Bedrock generates responses using a foundation model. This significantly reduces custom glue code and operational complexity.

Multilingual support is handled at the embedding and retrieval layer, allowing documents in English, Spanish, and Portuguese to be searched semantically without language-specific query logic. OpenSearch's distributed architecture ensures consistent low-latency responses for real-time customer interactions.

Option B increases operational overhead by requiring database tuning and scaling for vector workloads.

Option C does not support advanced metadata filtering, which is a key requirement. Option D introduces unnecessary complexity and is not optimized for large-scale semantic document retrieval. Therefore, Option A best meets the requirements for performance, scalability, multilingual support, and minimal management effort in an Amazon Bedrock-based RAG application.

NO.49 A financial services company is developing a customer service AI assistant by using Amazon Bedrock. The AI assistant must not discuss investment advice with users. The AI assistant must block harmful content, mask personally identifiable information (PII), and maintain audit trails for compliance reporting. The AI assistant must apply content filtering to both user inputs and model responses based on content sensitivity.

The company requires an Amazon Bedrock guardrail configuration that will effectively enforce policies with minimal false positives. The solution must provide multiple handling strategies for multiple types of sensitive content.

Which solution will meet these requirements?

A. Configure a single guardrail and set content filters to high for all categories. Set up denied topics for investment advice and include sample phrases to block. Set up sensitive information filters that apply the block action for all PII entities. Apply the guardrail to all model inference calls.

B. Configure multiple guardrails by using tiered policies. Create one guardrail and set content filters to high. Configure the guardrail to block PII for public interactions. Configure a second guardrail and set content filters to medium. Configure the second guardrail to mask PII for internal use. Configure multiple topic-specific guardrails to block investment advice and set up contextual grounding checks.

C. Configure a guardrail and set content filters to medium for harmful content. Set up denied topics for investment advice and include clear definitions and sample phrases to block. Configure sensitive information filters to mask PII in responses and to block financial information in inputs. Enable both

input and output evaluations that use custom blocked messages for audits.

D. Create a separate guardrail for each use case. Create one guardrail that applies a harmful content filter.Create a guardrail to apply topic filters for investment advice. Create a guardrail to apply sensitive information filters to block PII. Use AWS Step Functions to chain the guardrails sequentially.

Answer: C

Explanation:

Option C is the correct solution because it uses a single, well-tuned Amazon Bedrock guardrail that applies different actions to different content types, which is the recommended approach for minimizing false positives while enforcing strong policy controls.

Setting content filters to medium rather than high reduces overblocking of benign customer conversations while still preventing harmful content. Amazon Bedrock guardrails are designed to balance precision and recall, and medium sensitivity is commonly recommended for customer-facing financial services use cases.

Denied topics explicitly prevent the assistant from discussing investment advice, which is a regulatory requirement. Including definitions and sample phrases improves detection accuracy and reduces ambiguity.

Sensitive information filters support different actions per context. Masking PII in responses preserves conversational usefulness for legitimate customer support while preventing exposure of sensitive data.

Blocking sensitive financial information in inputs prevents downstream processing of disallowed content before it reaches the foundation model.

Critically, enabling both input and output evaluation ensures that guardrails are applied consistently at every stage of interaction. Custom blocked messages and audit logging provide clear compliance evidence for regulators and internal audits.

Option A causes excessive false positives by blocking all PII outright. Option B introduces unnecessary complexity and is not how Bedrock guardrails are intended to be applied. Option D uses orchestration logic that Bedrock guardrails already handle natively.

Therefore, Option C best satisfies enforcement, flexibility, auditability, and accuracy requirements.

NO.50 A pharmaceutical company is developing a Retrieval Augmented Generation (RAG) application that uses an Amazon Bedrock knowledge base. The knowledge base uses Amazon OpenSearch Service as a data source for more than 25 million scientific papers. Users report that the application produces inconsistent answers that cite irrelevant sections of papers when queries span methodology, results, and discussion sections of the papers.

The company needs to improve the knowledge base to preserve semantic context across related paragraphs on the scale of the entire corpus of data.

Which solution will meet these requirements?

A. Configure the knowledge base to use fixed-size chunking. Set a 300-token maximum chunk size and a

10% overlap between chunks. Use an appropriate Amazon Bedrock embedding model.

B. Configure the knowledge base to use hierarchical chunking. Use parent chunks that contain 1,000 tokens and child chunks that contain 200 tokens. Set a 50-token overlap between chunks.

C. Configure the knowledge base to use semantic chunking. Use a buffer size of 1 and a breakpoint percentile threshold of 85% to determine chunk boundaries based on content meaning.

D. Configure the knowledge base not to use chunking. Manually split each document into separate

files before ingestion. Apply post-processing reranking during retrieval.

Answer: B

Explanation:

Option B is the best solution because hierarchical chunking is specifically designed to preserve broader semantic context while still enabling precise retrieval at paragraph or sub-paragraph granularity. The problem described-answers citing irrelevant sections when a query spans multiple paper sections-often occurs when chunks are either too small (losing cross-paragraph context) or too "flat" (retrieving isolated snippets without their surrounding rationale).

In a scientific paper, related information is frequently distributed across methodology, results, and discussion.

Flat, fixed-size chunking (Option A) can split these logically connected ideas into separate chunks, causing retrieval to surface fragments that match a term but not the full intent. Semantic chunking (Option C) improves boundary placement, but it does not inherently provide a multi-resolution structure that helps preserve section-level continuity at massive scale.

Hierarchical chunking solves this by creating parent chunks (larger context windows) that capture broader section context and child chunks (smaller units) that retain retrieval precision. When the retriever identifies relevant child chunks, it can also bring in the associated parent context so the foundation model sees the surrounding methodological or discussion framing. The defined overlaps further reduce the risk that key transitions or references are split across chunks.

This approach is well suited for a corpus of 25 million papers because it improves relevance without requiring a custom reranking model or a manual preprocessing pipeline. It remains operationally efficient because it is configured at the knowledge base level rather than implemented through custom code per document.

Option D introduces high operational complexity and inconsistent document handling at scale.

Therefore, Option B best meets the requirement to preserve semantic context across related paragraphs and improve citation relevance across scientific paper sections.

NO.51 A financial services company is developing a generative AI (GenAI) application that serves both premium customers and standard customers. The application uses AWS Lambda functions behind an Amazon API Gateway REST API to process requests. The company needs to dynamically switch between AI models based on which customer tier each user belongs to. The company also wants to perform A/B testing for new features without redeploying code. The company needs to validate model parameters like temperature and maximum token limits before applying changes. Which solution will meet these requirements with the LEAST operational overhead?

A. Create AWS Systems Manager Parameter Store parameters for each configuration. Use Lambda functions to poll for parameter updates. Use Amazon EventBridge events to trigger redeployments when configurations change.

B. Store model configurations in Amazon DynamoDB tables. Optimize access patterns to retrieve configurations according to customer tier. Configure Lambda functions to query DynamoDB at the beginning of each request to determine which model to use.

C. Use AWS AppConfig to manage model configurations. Use feature flags to perform A/B testing. Define JSON schema validation rules for model parameters. Configure Lambda functions to retrieve configurations by using the AWS AppConfig Agent.

D. Create an Amazon ElastiCache (Redis OSS) cluster to store model configurations. Set short TTL values. Run custom validation logic in Lambda functions. Use Amazon CloudWatch metrics to monitor

configuration usage.

Answer: C

Explanation:

Option C is the correct solution because AWS AppConfig is purpose-built to manage dynamic application configurations with low latency, strong validation, and minimal operational overhead, which directly matches the company's requirements.

AWS AppConfig enables the company to centrally manage model selection logic, inference parameters, and customer-tier routing rules without redeploying Lambda functions. By using feature flags, the company can easily perform A/B testing of new models or prompt strategies by gradually rolling out changes to a subset of users or customer tiers. This allows experimentation and controlled releases without code changes.

AppConfig also supports JSON schema validation, which is critical for validating parameters such as temperature, maximum token limits, and other model-specific settings before they are applied. This prevents invalid or unsafe configurations from being deployed and reduces the risk of runtime errors or degraded model behavior in production.

Using the AWS AppConfig Agent allows Lambda functions to retrieve configurations efficiently with built-in caching and polling mechanisms, minimizing latency and avoiding excessive calls to configuration services.

This approach scales well for high-throughput, low-latency applications such as GenAI APIs behind Amazon API Gateway.

Option A introduces unnecessary redeployment logic and polling complexity. Option B requires building and maintaining custom configuration access patterns in DynamoDB and does not natively support feature flags or schema validation. Option D adds operational overhead by requiring ElastiCache cluster management and custom validation logic.

Therefore, Option C provides the most scalable, flexible, and low-maintenance solution for dynamic model switching, A/B testing, and safe configuration management in a GenAI application.

NO.52 A healthcare company is using Amazon Bedrock to develop a real-time patient care AI assistant to respond to queries for separate departments that handle clinical inquiries, insurance verification, appointment scheduling, and insurance claims. The company wants to use a multi-agent architecture.

The company must ensure that the AI assistant is scalable and can onboard new features for patients. The AI assistant must be able to handle thousands of parallel patient interactions. The company must ensure that patients receive appropriate domain-specific responses to queries.

Which solution will meet these requirements?

A. Isolate data for each agent by using separate knowledge bases. Use IAM filtering to control access to each knowledge base. Deploy a supervisor agent to perform natural language intent classification on patient inquiries. Configure the supervisor agent to route queries to specialized collaborator agents to respond to department-specific queries. Configure each specialized collaborator agent to use Retrieval Augmented Generation (RAG) with the agent's department-specific knowledge base.

B. Create a separate supervisor agent for each department. Configure individual collaborator agents to perform natural language intent classification for each specialty domain within each department. Integrate each collaborator agent with department-specific knowledge bases only. Implement manual handoff processes between the supervisor agents.

C. Isolate data for each department in separate knowledge bases. Use IAM filtering to control access to each knowledge base. Deploy a single general-purpose agent. Configure multiple action groups

within the general-purpose agent to perform specific department functions. Implement rule-based routing logic in the general-purpose agent instructions.

D. Implement multiple independent supervisor agents that run in parallel to respond to patient inquiries for each department. Configure multiple collaborator agents for each supervisor agent. Integrate all agents with the same knowledge base. Use external routing logic to merge responses from multiple supervisor agents.

Answer: A

Explanation:

Option A best meets the requirements because it applies an AWS-aligned multi-agent pattern that cleanly separates responsibilities: a supervisor agent performs intent classification and orchestration, while specialized collaborator agents handle domain-specific tasks using the right knowledge sources. This structure is well suited for healthcare workflows where clinical questions, scheduling, and insurance processes require different policies, terminology, and data access boundaries.

The requirement for appropriate domain-specific responses is addressed by routing each user query to a department-focused collaborator agent that is grounded with its own department-specific knowledge base.

Using Retrieval Augmented Generation with the correct knowledge base improves factual alignment and reduces cross-department leakage (for example, avoiding claims content in a clinical answer). It also supports better prompt grounding and more consistent tone and constraints per department.

The requirement to isolate data maps to using separate knowledge bases per agent and enforcing access through IAM controls, ensuring that each agent can retrieve only from the authorized datasets. This is important for minimizing unintended exposure of sensitive or irrelevant departmental data and supports governance and compliance needs.

For scalability and thousands of parallel interactions, this architecture minimizes contention and bottlenecks. Each collaborator agent can scale independently because requests are distributed across multiple agents and multiple retrieval backends. Operationally, onboarding new features is also simpler: the company can add a new collaborator agent (for example, "billing disputes" or "pharmacy refills") with its own knowledge base and policies without redesigning the entire assistant.

Option B introduces unnecessary complexity with multiple supervisors and manual handoffs. Option C overloads a single agent with broad instructions and rule-based routing, which increases prompt complexity and reduces maintainability as features grow. Option D creates high operational complexity and risks inconsistent outputs when merging responses from parallel supervisors, and it weakens data isolation by using a shared knowledge base across agents.

NO.53 A company is building a serverless application that uses AWS Lambda functions to help students around the world summarize notes. The application uses Anthropic Claude through Amazon Bedrock. The company observes that most of the traffic occurs during evenings in each time zone. Users report experiencing throttling errors during peak usage times in their time zones.

The company needs to resolve the throttling issues by ensuring continuous operation of the application. The solution must maintain application performance quality and must not require a fixed hourly cost during low traffic periods.

Which solution will meet these requirements?

A. Create custom Amazon CloudWatch metrics to monitor model errors. Set provisioned throughput to a value that is safely higher than the peak traffic observed.

B. Create custom Amazon CloudWatch metrics to monitor model errors. Set up a failover mechanism to redirect invocations to a backup AWS Region when the errors exceed a specified threshold.

C. Enable invocation logging in Amazon Bedrock. Monitor key metrics such as Invocations, InputTokenCount, OutputTokenCount, and InvocationThrottles. Distribute traffic across cross-Region inference endpoints.

D. Enable invocation logging in Amazon Bedrock. Monitor InvocationLatency, InvocationClientErrors, and InvocationServerErrors metrics. Distribute traffic across multiple versions of the same model.

Answer: C

Explanation:

Option C is the correct solution because it resolves throttling while preserving performance and avoiding fixed costs during low-traffic periods. Amazon Bedrock supports on-demand inference with usage-based pricing, making it well suited for applications with time-zone-dependent traffic spikes. Throttling during peak hours typically occurs when inference requests exceed available regional capacity.

Cross-Region inference allows Amazon Bedrock to automatically distribute requests across multiple AWS Regions, reducing contention and preventing throttling without requiring reserved or provisioned capacity.

This approach ensures continuous operation while maintaining low latency for users in different geographic locations.

Invocation logging and native metrics such as InvocationThrottles, InputTokenCount, and OutputTokenCount provide visibility into usage patterns and capacity constraints. Monitoring these metrics enables teams to validate that traffic distribution is working as intended and that performance remains consistent during peak periods.

Option A introduces fixed hourly costs by relying on provisioned throughput, which directly violates the requirement to avoid unnecessary spend during low-traffic periods. Option B introduces regional failover complexity and reactive behavior instead of proactive load distribution. Option D does not address the root cause of throttling, as distributing traffic across model versions within the same Region does not increase available capacity.

Therefore, Option C best aligns with AWS Generative AI best practices for scalable, cost-efficient, global serverless applications.

NO.54 A financial services company wants to develop an Amazon Bedrock application that gives analysts the ability to query quarterly earnings reports and financial statements. The financial documents are typically 5-100 pages long and contain both tabular data and text. The application must provide contextually accurate responses that preserve the relationship between financial metrics and their explanatory text. To support accurate and scalable retrieval, the application must incorporate document segmentation and context management strategies.

Which solution will meet these requirements?

A. Use a direct model invocation approach that uses Anthropic Claude to process each financial document as a single input. Use fine-tuned prompts that instruct the model to parse tables and text separately.

B. Use Amazon Bedrock Knowledge Bases to create a Retrieval Augmented Generation (RAG) application that retrieves relevant information from contextually chunked sections of financial documents. Segment documents based on their structural layout. Include citations that reference the original source materials.

C. Deploy an Amazon Bedrock agent that has an action group that calls custom AWS Lambda functions to analyze financial documents. Configure the Lambda functions to perform fixed-size

chunking when a user submits a query about financial metrics.

D. Create one specialized Amazon Bedrock application that is optimized for structured data. Create a second application that is optimized for unstructured data. Configure each application to use a tailored chunking strategy that is suited to the application's content type. Implement logic to link queries to the appropriate sources.

Answer: B

Explanation:

Option B best satisfies the requirements because it directly applies Retrieval Augmented Generation principles using managed Amazon Bedrock Knowledge Bases, which are designed to handle large, complex documents while preserving contextual relationships. Financial reports often interleave tables with explanatory narrative, and accurate analysis depends on keeping those elements logically connected. By segmenting documents based on their structural layout-for example, sections, subsections, tables, and surrounding commentary-the knowledge base can retrieve semantically relevant chunks that maintain this relationship during inference.

Amazon Bedrock Knowledge Bases support contextual chunking strategies that go beyond simple fixed-size segmentation. This is critical for financial documents, where a metric in a table may be explained in adjacent paragraphs or footnotes. Context-aware chunking ensures that retrieved content includes both the numeric data and its interpretation, enabling the foundation model to generate accurate, grounded responses. Including citations further improves analyst trust and auditability by allowing users to trace answers back to specific source sections, which is a common requirement in financial environments.

Scalability is another key requirement. Knowledge Bases manage embedding generation, indexing, and retrieval orchestration as a managed service, which allows the solution to scale across large document collections without requiring custom infrastructure or model hosting. This approach also supports efficient updates as new quarterly reports are added, ensuring the retrieval layer remains current.

Option A does not scale well because processing entire 5-100 page documents in a single prompt increases token usage, latency, and cost while risking context truncation. Option C relies on fixed-size chunking triggered at query time, which often breaks semantic relationships in structured financial content. Option D introduces unnecessary architectural complexity by splitting structured and unstructured data into separate applications, increasing operational overhead without providing better contextual retrieval than a unified RAG approach.

NO.55 A company is planning to deploy multiple generative AI (GenAI) applications to five independent business units that operate in multiple countries in Europe and the Americas. Each application uses Amazon Bedrock Retrieval Augmented Generation (RAG) patterns with business unit-specific knowledge bases that store terabytes of unstructured data.

The company must establish well-architected, standardized components for security controls, observability practices, and deployment patterns across all the GenAI applications. The components must be reusable, versioned, and governed consistently.

Which solution will meet these requirements?

A. Configure Amazon API Gateway REST API endpoints for the GenAI applications. Deploy common security, observability, and RAG patterns based on the AWS Well-Architected Generative AI Lens in standardized AWS CloudFormation templates. Use CloudFormation Guard after deployment to validate policy compliance in each business unit.

B. Create standardized AWS CloudFormation templates to implement security, observability, and

RAG patterns based on the AWS Well-Architected Generative AI Lens. Establish a centralized repository for version control. Integrate a CI/CD pipeline with CloudFormation Guard to enforce consistent and repeatable deployments across business units.

C. Use AWS Service Catalog to define standardized portfolios and versioned products for each business unit. Use the portfolios to enforce security, observability, and RAG patterns based on the AWS Well- Architected Generative AI Lens. Require business units to use the Service Catalog console to deploy resources.

D. Document security controls, observability requirements, and RAG patterns based on the AWS Well- Architected Generative AI Lens in a shared design document. Use Amazon Macie to enforce deployment. Delegate implementation responsibility to each business unit.

Answer: B

Explanation:

Option B best meets the requirement for reusable, versioned, and consistently governed components across multiple business units because it implements "platform-level standardization" through infrastructure as code plus automated compliance enforcement before deployment. Standardized CloudFormation templates provide reusable building blocks for security controls (identity, networking boundaries, encryption), observability practices (metrics, logs, traces), and RAG deployment patterns (knowledge base integration, ingestion pipelines, retrieval controls). This aligns with AWS guidance to operationalize well-architected patterns through repeatable templates rather than ad hoc implementations.

A centralized repository enables version control, change review, and governance of templates across all five business units. This satisfies the "versioned" and "reusable" requirements and provides a single source of truth for approved architectures. Integrating a CI/CD pipeline ensures that deployments are consistent and automated, reducing drift between business units and Regions.

CloudFormation Guard is most effective when used as a preventive control in the pipeline, not only after deployment. By running Guard rules during build or pre-deploy stages, the organization can enforce mandatory security and observability configurations and block noncompliant changes before they reach production. This supports consistent governance while still enabling business units to deploy quickly.

Option A performs compliance validation after deployment, which allows policy violations to be deployed first and remediated later. Option C provides governed provisioning but requiring console-based deployment reduces automation and can slow standardized CI/CD adoption; it also adds an additional governance layer that is not required to meet the stated needs. Option D is not enforceable and does not provide reusable, versioned, governed components.

Therefore, Option B provides the strongest, most scalable, and most consistently governed approach for standardized GenAI deployments across business units.

NO.56 A company uses AWS Lambda functions to build an AI agent solution. A GenAI developer must set up a Model Context Protocol (MCP) server that accesses user information. The GenAI developer must also configure the AI agent to use the new MCP server. The GenAI developer must ensure that only authorized users can access the MCP server.

Which solution will meet these requirements?

A. Use a Lambda function to host the MCP server. Grant the AI agent Lambda functions permission to invoke the Lambda function that hosts the MCP server. Configure the AI agent's MCP client to invoke the MCP server asynchronously.

B. Use a Lambda function to host the MCP server. Grant the AI agent Lambda functions permission to invoke the Lambda function that hosts the MCP server. Configure the AI agent to use the STDIO transport with the MCP server.

C. Use a Lambda function to host the MCP server. Create an Amazon API Gateway HTTP API that proxies requests to the Lambda function. Configure the AI agent solution to use the Streamable HTTP transport to make requests through the HTTP API. Use Amazon Cognito to enforce OAuth 2.1.

D. Use a Lambda layer to host the MCP server. Add the Lambda layer to the AI agent Lambda functions.Configure the agentic AI solution to use the STDIO transport to send requests to the MCP server. In the AI agent's MCP configuration, specify the Lambda layer ARN as the command. Specify the user credentials as environment variables.

Answer: C

Explanation:

Option C is the correct solution because it provides a secure, scalable, and standards-compliant way to expose an MCP server to an AI agent while enforcing strong user authorization. The Model Context Protocol supports HTTP-based transports for remote MCP servers, making Streamable HTTP the appropriate choice when the server is hosted as a managed service rather than a local process.

Hosting the MCP server in AWS Lambda enables automatic scaling and cost-efficient execution. By placing Amazon API Gateway in front of the Lambda function, the company creates a secure, managed HTTP endpoint that the AI agent can invoke reliably. This architecture cleanly separates transport, authentication, and business logic, which aligns with AWS serverless best practices.

Using Amazon Cognito to enforce OAuth 2.1 ensures that only authenticated and authorized users can access the MCP server. This satisfies security and compliance requirements when the MCP server handles sensitive user information. Cognito integrates natively with API Gateway, removing the need for custom authentication logic and reducing operational overhead.

Option A lacks user-level authorization controls. Option B and Option D rely on STDIO transport, which is intended for local or tightly coupled processes and is not suitable for distributed, serverless architectures.

Option D also introduces security risks by handling credentials through environment variables.

Therefore, Option C best meets the requirements for secure access control, scalability, and correct MCP integration in an AWS-based AI agent architecture.

NO.57 A company has a customer service application that uses Amazon Bedrock to generate personalized responses to customer inquiries. The company needs to establish a quality assurance process to evaluate prompt effectiveness and model configurations across updates. The process must automatically compare outputs from multiple prompt templates, detect response quality issues, provide quantitative metrics, and allow human reviewers to give feedback on responses. The process must prevent configurations that do not meet a predefined quality threshold from being deployed.

Which solution will meet these requirements?

A. Create an AWS Lambda function that sends sample customer inquiries to multiple Amazon Bedrock model configurations and stores responses in Amazon S3. Use Amazon QuickSight to visualize response patterns. Manually review outputs daily. Use AWS CodePipeline to deploy configurations that meet the quality threshold.

B. Use Amazon Bedrock evaluation jobs to compare model outputs by using custom prompt datasets .

Configure AWS CodePipeline to run the evaluation jobs when prompt templates change. Configure

CodePipeline to deploy only configurations that exceed the predefined quality threshold.

C. Set up Amazon CloudWatch alarms to monitor response latency and error rates from Amazon Bedrock.
Use Amazon EventBridge rules to notify teams when thresholds are exceeded. Configure a manual approval workflow in AWS Systems Manager.

D. Use AWS Lambda functions to create an automated testing framework that samples production traffic and routes duplicate requests to the updated model version. Use Amazon Comprehend sentiment analysis to compare results. Block deployment if sentiment scores decrease.

Answer: B

Explanation:
Option B is the correct solution because Amazon Bedrock evaluation jobs are purpose-built to assess prompt effectiveness, model behavior, and response quality in a repeatable and automated manner. Evaluation jobs support both quantitative metrics and LLM-based judgment, making them suitable for detecting subtle response quality regressions that simple sentiment or latency metrics cannot capture.
By using custom prompt datasets, the company can consistently test multiple prompt templates and model configurations against the same inputs. This enables accurate comparison across updates and eliminates variability introduced by live traffic sampling. Amazon Bedrock evaluation jobs also support structured scoring outputs, which can be used to enforce objective quality thresholds.
Integrating evaluation jobs directly into AWS CodePipeline ensures that quality checks are automatically triggered whenever prompt templates or configurations change. This creates a gated deployment workflow in which only configurations that meet or exceed the predefined quality threshold are promoted. This directly satisfies the requirement to prevent low-quality configurations from being deployed.
Human reviewers can be incorporated by reviewing evaluation results and scores produced by the jobs, enabling informed feedback without manual data collection. Option A and D rely on custom frameworks and indirect quality signals, increasing complexity and reducing reliability. Option C focuses on operational health rather than response quality.
Therefore, Option B provides the most robust, scalable, and AWS-aligned quality assurance process for Amazon Bedrock-based applications.

NO.58 A company is building a legal research AI assistant that uses Amazon Bedrock with an Anthropic Claude foundation model (FM). The AI assistant must retrieve highly relevant case law documents to augment the FM's responses. The AI assistant must identify semantic relationships between legal concepts, specific legal terminology, and citations. The AI assistant must perform quickly and return precise results.
Which solution will meet these requirements?

A. Configure an Amazon Bedrock knowledge base to use a default vector search configuration. Use Amazon Bedrock to expand queries to improve retrieval for legal documents based on specific terminology and citations.

B. Use Amazon OpenSearch Service to deploy a hybrid search architecture that combines vector search with keyword search. Apply an Amazon Bedrock reranker model to optimize result relevance.

C. Enable the Amazon Kendra query suggestion feature for end users. Use Amazon Bedrock to perform post-processing of search results to identify semantic similarity in the documents and to produce precise results.

D. Use Amazon OpenSearch Service with vector search and Amazon Bedrock Titan Embeddings to index and search legal documents. Use custom AWS Lambda functions to merge results with keyword-based filters that are stored in an Amazon RDS database.

Answer: B

Explanation:

Option B is the correct solution because legal research workloads require both semantic understanding and exact lexical precision, especially for statutes, citations, and domain-specific terminology. A hybrid search architecture directly addresses this need by combining vector similarity search with traditional keyword-based retrieval.

Vector search alone is often insufficient for legal research because exact phrases, citation formats, and jurisdiction-specific terms must be matched precisely. Keyword search ensures high recall and precision for citations and legal terms, while vector search captures deeper semantic relationships between legal concepts, precedents, and arguments. Amazon OpenSearch Service natively supports hybrid search, enabling efficient scoring and ranking without external orchestration.

Applying an Amazon Bedrock reranker model further improves relevance by reordering retrieved documents based on deeper contextual understanding. Reranking is especially valuable in legal research because multiple documents may appear relevant, but only a subset truly addresses the user's legal question. The reranker optimizes final results before they are passed to the Anthropic Claude FM, improving answer accuracy and reducing hallucinations.

Option A relies on default vector search, which does not reliably handle citations and exact terminology.

Option C focuses on query suggestions and post-processing rather than retrieval quality. Option D introduces unnecessary operational complexity by merging results across multiple systems.

Therefore, Option B best meets the requirements for precision, performance, and semantic understanding in a legal research AI assistant.

NO.59 A financial services company is creating a Retrieval Augmented Generation (RAG) application that uses Amazon Bedrock to generate summaries of market activities. The application relies on a vector database that stores a small proprietary dataset with a low index count. The application must perform similarity searches.

The Amazon Bedrock model's responses must maximize accuracy and maintain high performance.

The company needs to configure the vector database and integrate it with the application.

Which solution will meet these requirements?

A. Launch an Amazon MemoryDB cluster and configure the index by using the Flat algorithm. Configure a horizontal scaling policy based on performance metrics.

B. Launch an Amazon MemoryDB cluster and configure the index by using the Hierarchical Navigable Small World (HNSW) algorithm. Configure a vertical scaling policy based on performance metrics.

C. Launch an Amazon Aurora PostgreSQL cluster and configure the index by using the Inverted File with Flat Compression (IVFFlat) algorithm. Configure the instance class to scale to a larger size when the load increases.

D. Launch an Amazon DocumentDB cluster that has an IVFFlat index and a high probe value. Configure connections to the cluster as a replica set. Distribute reads to replica instances.

Answer: B

Explanation:

Option B is the optimal solution because it maximizes similarity search accuracy and performance for

a small, proprietary dataset while maintaining low operational complexity. Amazon MemoryDB is a fully managed, in- memory database that provides microsecond-level latency, making it ideal for real -time RAG workloads that require fast vector similarity searches.
For small datasets with low index counts, the Hierarchical Navigable Small World (HNSW) algorithm is recommended by AWS for its high recall and accuracy. Unlike approximate methods optimized for massive datasets, HNSW excels at returning the most semantically relevant vectors with minimal loss of precision, which directly improves the quality of responses generated by the Amazon Bedrock foundation model.
Vertical scaling in MemoryDB is sufficient for this use case because the dataset size is limited. Scaling up instance size provides increased memory and compute capacity without the complexity of managing distributed indexes or sharding strategies. This simplifies operations while maintaining predictable performance.
Option A's Flat algorithm is computationally expensive and inefficient at scale, even for moderate query volumes. Option C introduces higher latency and operational overhead by using a relational database not optimized for in-memory vector search. Option D is unsuitable because Amazon DocumentDB is not designed for high-performance vector similarity workloads and introduces unnecessary replica management complexity.
Therefore, Option B best meets the requirements for accuracy, performance, and efficient integration with an Amazon Bedrock-based RAG application.

NO.60 A medical company uses Amazon Bedrock to power a clinical documentation summarization system. The system produces inconsistent summaries when handling complex clinical documents. The system performed well on simple clinical documents.
The company needs a solution that diagnoses inconsistencies, compares prompt performance against established metrics, and maintains historical records of prompt versions.
Which solution will meet these requirements?

A. Create multiple prompt variants by using Prompt management in Amazon Bedrock. Manually test the prompts with simple clinical documents. Deploy the highest performing version by using the Amazon Bedrock console.

B. Implement version control for prompts in a code repository with a test suite that contains complex clinical documents and quantifiable evaluation metrics. Use an automated testing framework to compare prompt versions and document performance patterns.

C. Deploy each new prompt version to separate Amazon Bedrock API endpoints. Split production traffic between the endpoints. Configure Amazon CloudWatch to capture response metrics and user feedback for automatic version selection.

D. Create a custom prompt evaluation flow in Amazon Bedrock Flows that applies the same clinical document inputs to different prompt variants. Use Amazon Comprehend Medical to analyze and score the factual accuracy of each version.

Answer: B
Explanation:
Option B best meets the requirements because it provides systematic diagnosis, measurable comparison, and historical traceability of prompt performance. By placing prompts under version control and testing them against complex clinical documents, the company can consistently reproduce issues, track regressions, and compare prompt behavior using quantifiable metrics such as factual accuracy, completeness, and consistency.

Automated testing ensures scalability and repeatability, while version history preserves prompt evolution over time.
Option A lacks objective metrics and does not address complex documents. Option C focuses on live traffic experimentation but does not inherently diagnose prompt inconsistencies or preserve detailed historical evaluations. Option D adds medical entity analysis but introduces unnecessary service coupling and does not provide robust prompt version history or automated comparative benchmarking. Therefore, Option B is the most complete and disciplined solution.

NO.61 A company is using Amazon Bedrock and Anthropic Claude 3 Haiku to develop an AI assistant. The AI assistant normally processes 10,000 requests each hour but experiences surges of up to 30,000 requests each hour during peak usage periods. The AI assistant must respond within 2 seconds while operating across multiple AWS Regions.
The company observes that during peak usage periods, the AI assistant experiences throughput bottlenecks that cause increased latency and occasional request timeouts. The company must resolve the performance issues.
Which solution will meet this requirement?

A. Purchase provisioned throughput and sufficient model units (MUs) in a single Region. Configure the application to retry failed requests with exponential backoff.

B. Implement token batching to reduce API overhead. Use cross-Region inference profiles to automatically distribute traffic across available Regions.

C. Set up auto scaling AWS Lambda functions in each Region. Implement client-side round-robin request distribution. Purchase one model unit (MU) of provisioned throughput as a backup.

D. Implement batch inference for all requests by using Amazon S3 buckets across multiple Regions. Use Amazon SQS to set up an asynchronous retrieval process.

Answer: B
Explanation:
Option B is the correct solution because it directly addresses both throughput bottlenecks and latency requirements using native Amazon Bedrock performance optimization features that are designed for real-time, high-volume generative AI workloads.
Amazon Bedrock supports cross-Region inference profiles, which allow applications to transparently route inference requests across multiple AWS Regions. During peak usage periods, traffic is automatically distributed to Regions with available capacity, reducing throttling, request queuing, and timeout risks. This approach aligns with AWS guidance for building highly available, low-latency GenAI applications that must scale elastically across geographic boundaries.
Token batching further improves efficiency by combining multiple inference requests into a single model invocation where applicable. AWS Generative AI documentation highlights batching as a key optimization technique to reduce per-request overhead, improve throughput, and better utilize model capacity. This is especially effective for lightweight, low-latency models such as Claude 3 Haiku, which are designed for fast responses and high request volumes.
Option A does not meet the requirement because purchasing provisioned throughput in a single Region creates a regional bottleneck and does not address multi-Region availability or traffic spikes beyond reserved capacity. Retries increase load and latency rather than resolving the root cause.
Option C improves application-layer scaling but does not solve model-side throughput limits. Client-side round-robin routing lacks awareness of real-time model capacity and can still send traffic to saturated Regions.
Option D is unsuitable because batch inference with asynchronous retrieval is designed for offline or

non- interactive workloads. It cannot meet a strict 2-second response time requirement for an interactive AI assistant.
Therefore, Option B provides the most effective and AWS-aligned solution to achieve low latency, global scalability, and high throughput during peak usage periods.

NO.62 A company is developing a customer support application that uses Amazon Bedrock foundation models (FMs) to provide real-time AI assistance to the company's employees. The application must display AI- generated responses character by character as the responses are generated. The application needs to support thousands of concurrent users with minimal latency. The responses typically take 15 to 45 seconds to finish.
Which solution will meet these requirements?
A. Configure an Amazon API Gateway WebSocket API with an AWS Lambda integration. Configure the WebSocket API to invoke the Amazon Bedrock InvokeModelWithResponseStream API and stream partial responses through WebSocket connections.
B. Configure an Amazon API Gateway REST API with an AWS Lambda integration. Configure the REST API to invoke the Amazon Bedrock standard InvokeModel API and implement frontend client-side polling every 100 ms for complete response chunks.
C. Implement direct frontend client connections to Amazon Bedrock by using IAM user credentials and the InvokeModelWithResponseStream API without any intermediate gateway or proxy layer.
D. Configure an Amazon API Gateway HTTP API with an AWS Lambda integration. Configure the HTTP API to cache complete responses in an Amazon DynamoDB table and serve the responses through multiple paginated GET requests to frontend clients.
Answer: A
Explanation:
This requirement explicitly calls for character-by-character streaming, long-running responses, low latency, and massive concurrency, which aligns directly with Amazon Bedrock streaming inference patterns.
Amazon Bedrock provides the InvokeModelWithResponseStream API specifically for streaming partial model outputs as tokens are generated. This enables near-instant feedback to users instead of waiting for the full response to complete, which is essential when responses last up to 45 seconds.
Amazon API Gateway WebSocket APIs are purpose-built for bidirectional, low-latency, server-initiated communication, allowing the backend to push characters or tokens to clients in real time. This eliminates inefficient polling and supports thousands of concurrent open connections.
AWS Lambda integrates natively with WebSocket APIs and scales automatically with connection volume, enabling a fully managed, serverless architecture. This approach maintains security, centralized authentication, throttling, and observability while avoiding direct client access to Bedrock APIs.
Option B introduces polling latency and unnecessary API overhead and does not provide true streaming.
Option C violates AWS security best practices by exposing Bedrock directly to clients and does not scale securely. Option D only serves completed responses and cannot meet the real-time streaming requirement.
Therefore, Option A is the only solution that fully satisfies streaming behavior, concurrency, latency, and managed-service constraints.

NO.63 A media company is launching a platform that allows thousands of users every hour to upload

images and text content. The platform uses Amazon Bedrock to process the uploaded content to generate creative compositions.
The company needs a solution to ensure that the platform does not process or produce inappropriate content.
The platform must not expose personally identifiable information (PII) in the compositions. The solution must integrate with the company's existing Amazon S3 storage workflow.
Which solution will meet these requirements with the LEAST infrastructure management overhead?

A. Enable the Enhanced Monitoring tool. Use an Amazon CloudWatch alarm to filter traffic to the platform. Use Amazon Comprehend PII detection to pre-process the data. Create a CloudWatch alarm to monitor for Amazon Comprehend PII detection events. Create an AWS Step Functions workflow that includes an Amazon Rekognition image moderation step.

B. Use an Amazon API Gateway HTTP API with request validation templates to screen content before storing the uploaded content in Amazon S3. Use Amazon SageMaker AI to build custom content moderation models that process content before sending the processed content to Amazon Bedrock.

C. Create an Amazon Cognito user pool that uses pre-authentication AWS Lambda functions to run content moderation checks. Use Amazon Textract to filter text content and Amazon Rekognition to filter image content before allowing users to upload content to the platform.

D. Create an AWS Step Functions workflow that uses built-in Amazon Bedrock guardrails to filter content. Use Amazon Comprehend PII detection to pre-process the content. Use Amazon Rekognition image moderation.

Answer: D

Explanation:
Option D is the correct solution because it relies primarily on managed, purpose-built AWS services and minimizes custom infrastructure and model management. Amazon Bedrock guardrails provide native, configurable content safety controls that can block or redact disallowed content before or after model inference. This directly ensures that the platform does not process or produce inappropriate outputs while maintaining low operational overhead.
Using Amazon Comprehend PII detection as a preprocessing step integrates cleanly with an Amazon S3- based ingestion workflow. Comprehend is a fully managed service that detects and optionally redacts PII in text without requiring custom models or pipelines. This ensures that sensitive information is removed before content is passed to Amazon Bedrock for generation.
Amazon Rekognition image moderation is purpose-built for detecting unsafe or inappropriate visual content and integrates naturally into Step Functions workflows. Step Functions provides orchestration without requiring servers or long-running infrastructure, allowing the company to integrate text and image moderation steps in a clear, auditable pipeline.
Option A introduces redundant monitoring logic and alarms that do not directly enforce content safety. Option B requires building and maintaining custom SageMaker models, increasing complexity and operational burden. Option C applies moderation at authentication time and uses services like Textract that are not designed for content moderation, increasing latency and management overhead.
Therefore, Option D best satisfies content safety, PII protection, S3 integration, and minimal infrastructure management requirements.

NO.64 An ecommerce company is developing a generative AI (GenAI) solution that uses Amazon Bedrock with Anthropic Claude to recommend products to customers. Customers report that some

recommended products are not available for sale or are not relevant. Customers also report long response times for some recommendations.
The company confirms that most customer interactions are unique and that the solution recommends products not present in the product catalog.
Which solution will meet this requirement?

A. Increase grounding within Amazon Bedrock Guardrails. Enable automated reasoning checks. Set up provisioned throughput.

B. Use prompt engineering to restrict model responses to relevant products. Use streaming inference to reduce perceived latency.

C. Create an Amazon Bedrock Knowledge Bases and implement Retrieval Augmented Generation (RAG).
Set the PerformanceConfigLatency parameter to optimized.

D. Store product catalog data in Amazon OpenSearch Service. Validate model recommendations against the catalog. Use Amazon DynamoDB for response caching.

Answer: C

Explanation:
Option C is the correct solution because it directly addresses both correctness and performance issues by grounding the model's responses in authoritative product data using Retrieval Augmented Generation.
Amazon Bedrock Knowledge Bases are designed to connect foundation models to trusted enterprise data sources, ensuring that generated responses are constrained to known, validated content.
By ingesting the product catalog into a knowledge base, the GenAI application retrieves only products that actually exist in the catalog. This prevents hallucinated or unavailable recommendations, which is a common issue when models rely solely on prompt instructions without retrieval grounding. RAG ensures that the model's output is based on retrieved facts rather than learned generalizations.
Setting the PerformanceConfigLatency parameter to optimized enables Bedrock to prioritize lower-latency retrieval and inference paths, improving responsiveness for real-time recommendation scenarios. This directly addresses the reported performance issues without requiring provisioned throughput or caching strategies that are ineffective for mostly unique interactions.
Option A improves safety and latency predictability but does not ensure recommendations are limited to valid products. Option B relies on prompt constraints, which are not sufficient to prevent hallucinations. Option D introduces additional validation and caching layers but increases complexity and does not improve generation relevance.
Therefore, Option C best resolves both relevance and latency challenges using AWS-native, low-maintenance GenAI integration patterns.

NO.65 A financial services company needs to pre-process unstructured data such as customer transcripts, financial reports, and documentation. The company stores the unstructured data in Amazon S3 to support an Amazon Bedrock application.
The company must validate data quality, create auditable metadata, monitor data metrics, and customize text chunking to optimize foundation model (FM) performance.
Which solution will meet these requirements with the LEAST development effort?

A. Use Amazon SageMaker Data Wrangler to create a data flow. Configure Amazon CloudWatch metrics and alarms to monitor data quality. Use a custom AWS Lambda function to pre-process the data. Load processed data into Amazon Bedrock.

B. Set up an AWS Glue crawler to catalog data sources. Create AWS Glue ETL jobs to run custom transformation scripts. Use AWS Glue Data Quality to validate and monitor data quality. Load processed data into Amazon Bedrock.

C. Use Amazon Comprehend to extract entities. Create an AWS Lambda function to chunk text. Run Amazon Athena to query and validate data quality. Load processed data into Amazon Bedrock.

D. Create an AWS Step Functions workflow to orchestrate data pre-processing tasks. Run custom code on Amazon EC2 instances. Use Amazon SageMaker Model Monitor to monitor data quality. Load processed data into Amazon Bedrock.

Answer: B

Explanation:

Option B is the most appropriate solution because it uses AWS-native, purpose-built data engineering and governance services to address data quality validation, metadata creation, monitoring, and transformation with minimal custom development. AWS Glue is designed specifically for large-scale data preparation and integrates seamlessly with Amazon S3, making it ideal for preprocessing unstructured datasets for downstream GenAI applications.

AWS Glue crawlers automatically infer schemas and populate the AWS Glue Data Catalog, creating auditable, queryable metadata for all datasets. This satisfies the requirement for traceability and governance, which is especially critical in financial services environments. Glue ETL jobs allow teams to implement customizable transformation logic, including text normalization and chunking strategies optimized for foundation model context windows.

AWS Glue Data Quality provides built-in rulesets for validating completeness, accuracy, and consistency. It also publishes quality metrics that can be monitored over time, meeting the requirement for ongoing data quality monitoring without building custom validation frameworks.

Because AWS Glue is fully managed, it eliminates the need to manage infrastructure, scaling, or orchestration. This significantly reduces development and operational effort compared to custom Lambda pipelines or EC2-based processing. The processed and validated data can then be safely ingested into Amazon Bedrock workflows or knowledge bases.

Option A and C require custom logic for validation, monitoring, and chunking, increasing development complexity. Option D introduces unnecessary infrastructure management and services not optimized for data preprocessing.

Therefore, Option B best meets the requirements while minimizing development effort and aligning with AWS Generative AI data preparation best practices.

NO.66 A medical company is creating a generative AI (GenAI) system by using Amazon Bedrock. The system processes data from various sources and must maintain end-to-end data lineage. The system must also use real- time personally identifiable information (PII) filtering and audit trails to automatically report compliance.

Which solution will meet these requirements?

A. Use AWS Glue Data Catalog to register all data sources and track lineage. Use Amazon Bedrock Guardrails PII filters. Enable AWS CloudTrail logging for all Amazon Bedrock API calls with Amazon S3 integration. Use Amazon Macie to scan stored data for sensitive information and publish findings to Amazon CloudWatch Logs. Create CloudWatch dashboards to visualize the findings and generate automated compliance reports.

B. Use AWS Config to track data source configurations and changes. Use AWS WAF with custom rules to filter PII at the application layer before Amazon Bedrock processes the data. Configure Amazon

EventBridge to capture and route audit events to Amazon S3. Use Amazon Comprehend Medical with scheduled AWS Lambda functions to analyze stored outputs for compliance violations.

C. Use AWS DataSync to replicate data sources to track lineage. Configure Amazon Macie to scan Amazon Bedrock outputs for sensitive information. Use AWS Systems Manager Session Manager to log user interactions. Deploy Amazon Textract with AWS Step Functions workflows to identify and redact PII from generated reports.

D. Configure Amazon Athena to query data sources to analyze and report on data lineage. Use Amazon CloudWatch custom metrics to monitor PII exposure in Amazon Bedrock responses and establish AWS X-Ray tracing to generate an audit trail. Use an Amazon Rekognition Custom Labels model to detect sensitive information in the data that Amazon Bedrock processes.

Answer: A

Explanation:

Option A is the most comprehensive and architecturally aligned solution for meeting end-to-end data lineage, real-time PII filtering, and automated compliance reporting requirements in a medical GenAI system built on Amazon Bedrock. Each requirement maps directly to a managed AWS service that is purpose-built for governance, security, and compliance.

AWS Glue Data Catalog is designed to register datasets across multiple sources and maintain metadata that supports lineage tracking. By cataloging all inputs that flow into the Bedrock-based system, the organization can trace how data moves from ingestion through processing and storage, which is essential for regulatory audits in healthcare environments.

For real-time PII filtering, Amazon Bedrock Guardrails provide native PII detection and filtering during model inference. Guardrails operate inline with model invocation, ensuring sensitive information is blocked or redacted before responses are returned to users. This satisfies the requirement for real-time protection rather than post-processing analysis.

AWS CloudTrail delivers a complete audit trail of all Amazon Bedrock API calls, including InvokeModel requests and configuration changes. Storing these logs in Amazon S3 enables long-term retention and supports compliance audits. CloudTrail ensures traceability of who accessed the system, when, and how it was used.

To strengthen compliance monitoring, Amazon Macie continuously scans stored data for sensitive information and automatically classifies findings. Publishing Macie findings to Amazon CloudWatch Logs and visualizing them through dashboards enables near-real-time visibility into compliance posture and supports automated reporting workflows.

The other options fall short. Option B performs PII filtering at the application edge rather than at inference time and relies on scheduled analysis instead of real-time enforcement. Option C focuses on replication and document processing rather than inline GenAI governance. Option D uses services that are not designed for PII detection in text-based GenAI workflows and lacks native lineage tracking.

Therefore, A best fulfills all stated requirements using AWS-recommended governance and security capabilities.

NO.67 A hotel company wants to enhance a legacy Java-based property management system (PMS) by adding AI capabilities. The company wants to use Amazon Bedrock Knowledge Bases to provide staff with room availability information and hotel-specific details. The solution must maintain separate access controls for each hotel that the company manages. The solution must provide room availability information in near real time and must maintain consistent performance during peak usage periods.

Which solution will meet these requirements?

A. Deploy a single Amazon Bedrock knowledge base that contains combined data for all hotels. Configure AWS Lambda functions to synchronize data from each hotel's PMS database through direct API connections. Implement AWS CloudTrail logging with hotel-specific filters to audit access logs for each hotel's data.

B. Create an Amazon EventBridge rule for each hotel that is invoked by changes to the PMS database .
Configure the rule to send updates to a centralized Amazon Bedrock knowledge base in a management AWS account. Configure resource-based policies to enforce hotel-specific access controls.

C. Implement one Amazon Bedrock knowledge base for each hotel in a multi-account structure. Use direct data ingestion to provide near real-time room availability information. Schedule regular synchronization for less critical information.

D. Build a centralized Amazon Bedrock Agents solution that uses multiple knowledge bases. Implement AWS IAM Identity Center with hotel-specific permission sets to control staff access.

Answer: C

Explanation:

Option C best meets the requirements by aligning with AWS best practices for data isolation, access control, and scalable GenAI retrieval. Implementing a separate Amazon Bedrock knowledge base for each hotel ensures strict separation of data and permissions. This approach naturally enforces hotel-level access control without requiring complex policy logic or post-query filtering.

A multi-account structure further strengthens security and governance by isolating each hotel's data plane.

AWS recommends account-level isolation for workloads with strong tenancy or compliance boundaries. Hotel staff can be granted access only to their hotel's account and corresponding knowledge base, eliminating the risk of cross-hotel data exposure.

Direct data ingestion into each knowledge base enables near real-time updates for critical data such as room availability. For information that does not change frequently, scheduled synchronization reduces ingestion cost while maintaining accuracy. This hybrid ingestion model balances freshness and operational efficiency.

Because Amazon Bedrock Knowledge Bases are fully managed, performance remains consistent during peak usage periods without the company managing indexing, scaling, or retrieval infrastructure. Each knowledge base scales independently, preventing noisy-neighbor issues that could arise in a centralized design.

Option A and B rely on a centralized knowledge base, which increases policy complexity and introduces risk of misconfigured access controls. Option D adds unnecessary orchestration complexity and does not inherently solve real-time data freshness requirements.

Therefore, Option C provides the most secure, scalable, and operationally efficient solution for enhancing the PMS with Amazon Bedrock Knowledge Bases.

NO.68 An ecommerce company operates a global product recommendation system that needs to switch between multiple foundation models (FMs) in Amazon Bedrock based on regulations, cost optimization, and performance requirements. The company must apply custom controls based on proprietary business logic, including dynamic cost thresholds, AWS Region-specific compliance rules, and real-time A/B testing across multiple FMs. The system must be able to switch between FMs

without deploying new code. The system must route user requests based on complex rules including user tier, transaction value, regulatory zone, and real-time cost metrics that change hourly and require immediate propagation across thousands of concurrent requests.
Which solution will meet these requirements?

A. Deploy an AWS Lambda function that uses environment variables to store routing rules and Amazon Bedrock FM IDs. Use the Lambda console to update the environment variables when business requirements change. Configure an Amazon API Gateway REST API to read request parameters to make routing decisions.

B. Deploy Amazon API Gateway REST API request transformation templates to implement routing logic based on request attributes. Store Amazon Bedrock FM endpoints as REST API stage variables. Update the variables when the system switches between models.

C. Configure an AWS Lambda function to fetch routing configuration from the AWS AppConfig Agent for each user request. Run business logic in the Lambda function to select the appropriate FM for each request. Expose the FM through a single Amazon API Gateway REST API endpoint.

D. Use AWS Lambda authorizers for an Amazon API Gateway REST API to evaluate routing rules that are stored in AWS AppConfig. Return authorization contexts based on business logic. Route requests to model-specific Lambda functions for each Amazon Bedrock FM.

Answer: C

Explanation:

Option C best satisfies the requirement to change routing decisions without redeploying code while supporting complex, frequently changing business logic at scale. AWS AppConfig is designed for centrally managing dynamic configuration (feature flags, rules, thresholds, and policy parameters) and deploying changes safely. It supports controlled deployments, validation, and rapid propagation of updated configuration values, which aligns with "real-time cost metrics that change hourly" and the need for "immediate propagation across thousands of concurrent requests." In this design, the Lambda function becomes the policy decision point. For each request, it evaluates user attributes (tier, transaction value), context (regulatory zone, Region), and live cost/performance thresholds stored in AppConfig to determine which Amazon Bedrock FM to invoke. Because the routing rules and FM identifiers are delivered as configuration, the company can switch models, adjust A/B testing weights, or update compliance routing rules by deploying new AppConfig configuration versions rather than pushing new application code. This reduces operational risk and accelerates iteration. Exposing a single API Gateway endpoint also minimizes client complexity and keeps routing logic server- side, which is important when rules change frequently. Lambda can cache configuration between invocations (within the execution environment) to reduce repeated fetch overhead while still picking up changes quickly, enabling both low latency and rapid rule rollout under high concurrency.

Option A relies on Lambda environment variables, which are not intended for frequent real-time updates and typically require function configuration updates that are slower and operationally brittle. Option B uses mapping templates and stage variables, which are limited for complex rule evaluation and safe rollout patterns. Option D misuses authorizers for business routing, adds extra latency and complexity, and complicates observability and error handling by splitting decisioning from execution.

NO.69 A healthcare company is developing a document management system that stores medical research papers in an Amazon S3 bucket. The company needs a comprehensive metadata framework

to improve search precision for a GenAI application. The metadata must include document timestamps, author information, and research domain classifications.
The solution must maintain a consistent metadata structure across all uploaded documents and allow foundation models (FMs) to understand document context without accessing full content.
Which solution will meet these requirements?

A. Store document timestamps in Amazon S3 system metadata. Use S3 object tags for domain classification. Implement custom user-defined metadata to store author information.

B. Set up S3 Object Lock with legal holds to track document timestamps. Use S3 object tags for author information. Implement S3 access points for domain classification.

C. Use S3 Inventory reports to track timestamps. Create S3 access points for domain classification. Store author information in S3 Storage Lens dashboards.

D. Use custom user-defined metadata to store author information. Use S3 Object Lock retention periods for timestamps. Use S3 Event Notifications for domain classification.

Answer: A

Explanation:
Option A is the correct solution because it uses native Amazon S3 metadata mechanisms to create a consistent, queryable, and model-friendly metadata framework with minimal complexity. S3 system metadata automatically records object creation and modification timestamps, providing reliable and consistent temporal context without additional processing.
Custom user-defined metadata is the appropriate mechanism for storing structured attributes such as author information. These key-value pairs are stored directly with the object, remain consistent across uploads, and can be accessed programmatically by downstream indexing or retrieval systems used by GenAI applications.
S3 object tags are ideal for domain classification because they are designed for lightweight categorization, filtering, and access control. Tags can be standardized across the organization to ensure consistent research domain labeling and can be consumed by search indexes or knowledge base ingestion pipelines without requiring access to the full document body.
Together, system metadata, user-defined metadata, and object tags provide a clean separation of concerns:
timestamps for temporal context, metadata for authorship, and tags for classification. This structure allows foundation models to reason about document context (such as recency, domain relevance, and authorship) based on metadata alone, improving retrieval precision and reducing unnecessary token usage.
Options B, C, and D misuse features like Object Lock, access points, Storage Lens, or event notifications for purposes they were not designed for, adding complexity without improving metadata quality or model understanding.
Therefore, Option A best satisfies the metadata consistency, context enrichment, and low-overhead requirements for GenAI-driven document analysis.

NO.70 A company is designing an API for a generative AI (GenAI) application that uses a foundation model (FM) that is hosted on a managed model service. The API must stream responses to reduce latency, enforce token limits to manage compute resource usage, and implement retry logic to handle model timeouts and partial responses.
Which solution will meet these requirements with the LEAST operational overhead?

A. Integrate an Amazon API Gateway HTTP API with an AWS Lambda function to invoke Amazon

Bedrock. Use Lambda response streaming to stream responses. Enforce token limits within the Lambda function. Implement retry logic for model timeouts by using Lambda and API Gateway timeout configurations.

B. Connect an Amazon API Gateway HTTP API directly to Amazon Bedrock. Simulate streaming by using client-side polling. Enforce token limits on the frontend. Configure retry behavior by using API Gateway integration settings.

C. Connect an Amazon API Gateway WebSocket API to an Amazon ECS service that hosts a containerized inference server. Stream responses by using the WebSocket protocol. Enforce token limits within Amazon ECS. Handle model timeouts by using ECS task lifecycle hooks and restart policies.

D. Integrate an Amazon API Gateway REST API with an AWS Lambda function that invokes Amazon Bedrock. Use Lambda response streaming to stream responses. Enforce token limits within the Lambda function. Implement retry logic by using Lambda and API Gateway timeout configurations.

Answer: A

Explanation:

Option A is the best solution because it satisfies streaming, token control, and retry requirements while keeping operational overhead low by using fully managed, serverless AWS services. Amazon API Gateway HTTP APIs provide a lightweight, cost-effective front door for APIs and integrate cleanly with AWS Lambda for request processing and security controls.

AWS Lambda response streaming allows the API to begin returning content to the client as soon as partial model output is available, reducing perceived latency and improving user experience for long responses.

Using Lambda as the integration layer also provides a centralized place to enforce token-aware request handling, such as rejecting oversized requests, truncating optional context, or applying consistent limits across users and tenants to manage compute usage.

Retry logic is best handled in the client or integration layer for transient failures such as timeouts and throttling. Lambda can implement controlled retries with exponential backoff and jitter, while API Gateway timeouts help bound request lifetimes and prevent hung connections from consuming resources indefinitely.

Because the model service is managed, the company avoids infrastructure management and focuses only on request shaping, safety, and resiliency behavior.

Option B is not suitable because client-side polling is not true streaming, front-end token enforcement is insecure and inconsistent, and API Gateway does not provide model-aware retry behavior on its own. Option C introduces container hosting and scaling complexity, which increases operational overhead compared to serverless. Option D can work, but REST APIs are generally heavier than HTTP APIs for this pattern and do not reduce overhead compared to Option A.

Therefore, Option A provides the required streaming and resiliency capabilities with the least infrastructure management effort.

NO.71 A company is creating a workflow to review customer-facing communications before the company sends the communications. The company uses a pre-defined message template to generate the communications and stores the communications in an Amazon S3 bucket. The workflow needs to capture a specific portion from the template and send it to an Amazon Bedrock model. The workflow must store model responses back to the original S3 bucket.

Which solution will meet these requirements?

A. Create a flow in Amazon Bedrock Flows. Configure S3 action nodes at the beginning and end of the flow to retrieve and store the communications and the model responses. In the middle of the flow, configure an expression to parse each communication. Configure an agent step to send the parsed input to the model for review.

B. Create an AWS Step Functions Express workflow state machine. Use an Amazon S3 integration GetObject step to retrieve the original communications. Use an intrinsic function Pass step to parse the communications and to pass the results to an Amazon Bedrock InvokeModel step. Configure an Amazon S3 integration PutObject step to store the model responses back to the S3 bucket.

C. Create an Amazon Bedrock agent that has an action group. Configure instructions to define how the agent should parse the communications. Configure the action group to retrieve the communications from the S3 bucket, invoke the Amazon Bedrock model, and store the model responses back to the S3 bucket.

D. Create an Amazon Bedrock agent that has a single action group. Configure three AWS Lambda functions in the action group. Configure the functions to retrieve the communications from the S3 bucket, parse the communications and invoke the Amazon Bedrock model, and store the model responses back to the S3 bucket.

Answer: A

Explanation:

Option A is the correct answer because Amazon Bedrock Flows is purpose-built to orchestrate generative AI workflows that combine data access, deterministic transformations, and model invocation with minimal operational overhead. The requirements explicitly state that the workflow must retrieve content from Amazon S3, extract a specific portion of a predefined template, send that portion to an Amazon Bedrock model, and store the model's response back into the same S3 bucket. Amazon Bedrock Flows natively supports all of these steps.

By configuring S3 action nodes at the beginning and end of the flow, the workflow can retrieve the original communications and persist the reviewed output without custom code. The expression step allows deterministic parsing of a specific portion of the template, which is essential when only part of the message should be reviewed. This avoids relying on generative logic for parsing, which would be less predictable and harder to audit. The agent step is then used specifically for the review task, where the foundation model evaluates or modifies the extracted content.

Option B uses AWS Step Functions, which can achieve similar outcomes but requires more explicit orchestration logic and does not provide GenAI-native constructs such as expressions and agent steps in a single managed experience. Options C and D rely on Amazon Bedrock agents and AWS Lambda functions to handle parsing and data movement, which increases complexity, operational overhead, and maintenance burden.

Because Amazon Bedrock Flows directly integrates S3 actions, parsing expressions, and model review steps in a single managed workflow, Option A best meets the requirements with the least development and operational effort.

NO.72 A healthcare company is developing an application to process medical queries. The application must answer complex queries with high accuracy by reducing semantic dilution. The application must refer to domain- specific terminology in medical documents to reduce ambiguity in medical terminology. The application must be able to respond to 1,000 queries each minute with response times less than 2 seconds.

Which solution will meet these requirements with the LEAST operational overhead?

A. Use Amazon API Gateway to route incoming queries to an Amazon Bedrock agent. Configure the agent to use an Anthropic Claude model to decompose queries and an Amazon Titan model to expand queries. Create an Amazon Bedrock knowledge base to store the reference medical documents.
B. Configure an Amazon Bedrock knowledge base to store the reference medical documents. Enable query decomposition in the knowledge base. Configure an Amazon Bedrock flow that uses a foundation model and the knowledge base to support the application.
C. Use Amazon SageMaker AI to host custom ML models for both query decomposition and query expansion. Configure Amazon Bedrock knowledge bases to store the reference medical documents. Encrypt the documents in the knowledge base.
D. Create an Amazon Bedrock agent to orchestrate multiple AWS Lambda functions to decompose queries. Create an Amazon Bedrock knowledge base to store the reference medical documents. Use the agent's built-in knowledge base capabilities. Add deep research and reasoning capabilities to the agent to reduce ambiguity in the medical terminology.
Answer: B
Explanation:
Option B provides the least operational overhead because it keeps the solution primarily inside managed Amazon Bedrock capabilities, minimizing custom orchestration code and infrastructure to operate. The core requirements are domain grounding, reduced semantic dilution for complex questions, and consistent low- latency responses at high request volume. A Bedrock knowledge base is purpose-built for Retrieval Augmented Generation by ingesting domain documents, chunking content, generating embeddings, and retrieving the most relevant passages at runtime. This directly addresses the need to reference domain-specific medical terminology from authoritative documents to reduce ambiguity and improve factual accuracy.
Reducing semantic dilution typically requires improving the retrieval query so that the retriever focuses on the most relevant concepts, especially for long or multi-intent questions. Enabling query decomposition allows the system to break a complex medical query into smaller, more targeted sub-queries. This increases retrieval precision and recall for each sub-question, which helps the model generate a more accurate synthesized response grounded in the retrieved medical context.
Amazon Bedrock Flows provide a managed way to orchestrate multi-step generative AI workflows, such as preprocessing the input, performing retrieval against the knowledge base, invoking a foundation model, and formatting the final response. Because flows are managed, the company avoids maintaining custom state machines, multiple Lambda functions, or bespoke routing logic. This reduces operational overhead while still supporting repeatable, observable execution.
Compared with the alternatives, option A introduces an agent plus API Gateway routing and multiple model choices, increasing configuration and runtime complexity. Option C requires hosting and scaling custom models on SageMaker AI, which adds significant operational burden and latency risk. Option D relies on multiple Lambda functions orchestrated by an agent, which adds more moving parts and increases cold-start and integration overhead. Option B most directly meets the requirements with the smallest operational footprint.

NO.73 A company deploys multiple Amazon Bedrock-based generative AI (GenAI) applications across multiple business units for customer service, content generation, and document analysis. Some applications show unpredictable token consumption patterns. The company requires a comprehensive observability solution that provides real-time visibility into token usage patterns

across multiple models. The observability solution must support custom dashboards for multiple stakeholder groups and provide alerting capabilities for token consumption across all the foundation models that the company's applications use.
Which combination of solutions will meet these requirements with the LEAST operational overhead? (Select TWO.)

A. Use Amazon CloudWatch metrics as data sources to create custom Amazon QuickSight dashboards that show token usage trends and usage patterns across FMs.

B. Use CloudWatch Logs Insights to analyze Amazon Bedrock invocation logs for token consumption patterns and usage attribution by application. Create custom queries to identify high-usage scenarios.
Add log widgets to dashboards to enable continuous monitoring.

C. Create custom Amazon CloudWatch dashboards that combine native Amazon Bedrock token and invocation CloudWatch metrics. Set up CloudWatch alarms to monitor token usage thresholds.

D. Create dashboards that show token usage trends and patterns across the company's FMs by using an Amazon Bedrock zero-ETL integration with Amazon Managed Grafana.

E. Implement Amazon EventBridge rules to capture Amazon Bedrock model invocation events. Route token usage data to Amazon OpenSearch Serverless by using Amazon Data Firehose. Use OpenSearch dashboards to analyze usage patterns.

Answer: C D
Explanation:
The combination of Options C and D delivers comprehensive, real-time observability for Amazon Bedrock workloads with the least operational overhead by relying on native integrations and managed services.
Amazon Bedrock publishes built-in CloudWatch metrics for model invocations and token usage. Option C leverages these native metrics directly, allowing teams to build centralized CloudWatch dashboards without additional data pipelines or custom processing. CloudWatch alarms provide threshold-based alerting for token consumption, enabling proactive cost and usage control across all foundation models. This approach aligns with AWS guidance to use native service metrics whenever possible to reduce operational complexity.
Option D complements CloudWatch by enabling advanced, stakeholder-specific visualizations through Amazon Managed Grafana. The zero-ETL integration allows Bedrock and CloudWatch metrics to be visualized directly in Grafana without building ingestion pipelines or managing storage layers. Grafana dashboards are particularly well suited for serving different audiences, such as engineering, finance, and product teams, each with customized views of token usage and trends.
Option A introduces unnecessary complexity by adding a business intelligence layer that is better suited for historical analytics than real-time operational monitoring. Option B is useful for deep log analysis but requires query maintenance and does not provide efficient real-time dashboards at scale. Option E involves multiple services and custom data flows, significantly increasing operational overhead compared to native metric- based observability.
By combining CloudWatch dashboards and alarms with Managed Grafana's zero-ETL visualization capabilities, the company achieves real-time visibility, flexible dashboards, and automated alerting across all Amazon Bedrock foundation models with minimal operational effort.

NO.74 A company is building a legal research AI assistant that uses Amazon Bedrock with an Anthropic Claude foundation model (FM). The AI assistant must retrieve highly relevant case law

documents to augment the FM's responses. The AI assistant must identify semantic relationships between legal concepts, specific legal terminology, and citations. The AI assistant must perform quickly and return precise results.
Which solution will meet these requirements?

A. Configure an Amazon Bedrock knowledge base to use a default vector search configuration. Use Amazon Bedrock to expand queries to improve retrieval for legal documents based on specific terminology and citations.

B. Use Amazon OpenSearch Service to deploy a hybrid search architecture that combines vector search with keyword search. Apply an Amazon Bedrock reranker model to optimize result relevance.

C. Use Amazon OpenSearch Service with vector search and Amazon Bedrock Titan Embeddings to index and search legal documents. Use custom AWS Lambda functions to merge results with keyword-based filters that are stored in an Amazon RDS database.

D. Enable the Amazon Kendra query suggestion feature for end users. Use Amazon Bedrock to perform post-processing of search results to identify semantic similarity in the documents and to produce precise results.

Answer: B

Explanation:
Option B is the correct solution because legal research workloads require both semantic understanding and exact lexical precision, especially for statutes, citations, and domain-specific terminology. A hybrid search architecture directly addresses this need by combining vector similarity search with traditional keyword-based retrieval.
Vector search alone is often insufficient for legal research because exact phrases, citation formats, and jurisdiction-specific terms must be matched precisely. Keyword search ensures high recall and precision for citations and legal terms, while vector search captures deeper semantic relationships between legal concepts, precedents, and arguments. Amazon OpenSearch Service natively supports hybrid search, enabling efficient scoring and ranking without external orchestration.
Applying an Amazon Bedrock reranker model further improves relevance by reordering retrieved documents based on deeper contextual understanding. Reranking is especially valuable in legal research because multiple documents may appear relevant, but only a subset truly addresses the user's legal question. The reranker optimizes final results before they are passed to the Anthropic Claude FM, improving answer accuracy and reducing hallucinations.
Option A relies on default vector search, which does not reliably handle citations and exact terminology.
Option C focuses on query suggestions and post-processing rather than retrieval quality. Option D introduces unnecessary operational complexity by merging results across multiple systems.
Therefore, Option B best meets the requirements for precision, performance, and semantic understanding in a legal research AI assistant.

NO.75 A company uses Amazon Bedrock to implement a Retrieval Augmented Generation (RAG)-based system to serve medical information to users. The company needs to compare multiple chunking strategies, evaluate the generation quality of two foundation models (FMs), and enforce quality thresholds for deployment.
Which Amazon Bedrock evaluation configuration will meet these requirements?

A. Create a retrieve-only evaluation job that uses a supported version of Anthropic Claude Sonnet as the evaluator model. Configure metrics for context relevance and context coverage. Define

deployment thresholds in a separate CI/CD pipeline.

B. Create a retrieve-and-generate evaluation job that uses custom precision-at-k metrics and an LLM-as-a- judge metric with a scale of 1-5. Include each chunking strategy in the evaluation dataset. Use a supported version of Anthropic Claude Sonnet to evaluate responses from both FMs.

C. Create a separate evaluation job for each chunking strategy and FM combination. Use Amazon Bedrock built-in metrics for correctness and completeness. Manually review scores before deployment approval.

D. Set up a pipeline that uses multiple retrieve-only evaluation jobs to assess retrieval quality. Create separate evaluation jobs for both FMs that use Amazon Nova Pro as the LLM-as-a-judge model.Evaluate based on faithfulness and citation precision metrics.

Answer: B

Explanation:

Option B is the correct evaluation configuration because it enables end-to-end assessment of both retrieval and generation quality while supporting direct comparison of chunking strategies and foundation models.

Amazon Bedrock evaluation jobs are designed to support RAG workflows by evaluating how well retrieved context supports accurate and high-quality model outputs.

A retrieve-and-generate evaluation job evaluates the complete RAG pipeline, not just retrieval. This is essential for medical information use cases, where both the relevance of retrieved content and the correctness of generated responses directly impact user safety and trust. Including multiple chunking strategies in the evaluation dataset allows side-by-side comparison under identical prompts and conditions.

Custom precision-at-k metrics measure how effectively the retrieval component surfaces relevant chunks, while an LLM-as-a-judge metric provides qualitative scoring of generated responses. Using a numeric scale enables consistent, repeatable evaluation and supports automated quality gates.

Amazon Bedrock supports LLM-based evaluators to score dimensions such as accuracy, completeness, and relevance.

Using the same evaluator model to assess outputs from both FMs ensures consistent scoring and eliminates evaluator bias. This configuration allows the company to define quantitative thresholds that must be met before deployment, enabling automated promotion through CI/CD pipelines.

Option A evaluates retrieval only and cannot assess generation quality. Option C introduces manual review, which does not scale and delays deployment. Option D separates retrieval and generation evaluation, making it harder to correlate chunking strategies with final output quality.

Therefore, Option B best meets the requirements for systematic evaluation, comparison, and quality enforcement in an Amazon Bedrock-based RAG system.

NO.76 A finance company is developing an AI assistant to help clients plan investments and manage their portfolios.

The company identifies several high-risk conversation patterns such as requests for specific stock recommendations or guaranteed returns. High-risk conversation patterns could lead to regulatory violations if the company cannot implement appropriate controls.

The company must ensure that the AI assistant does not provide inappropriate financial advice, generate content about competitors, or make claims that are not factually grounded in the company's approved financial guidance. The company wants to use Amazon Bedrock Guardrails to implement a solution.

Which combination of steps will meet these requirements? (Select THREE)

A. Add the high-risk conversation patterns to a denied topics guardrail.
B. Configure a content filter guardrail to filter prompts that contain the high-risk conversation patterns.
C. Configure a content filter guardrail to filter prompts that contain competitor names.
D. Add the names of competitors as custom word filters. Set the input and output actions to block.
E. Set a low grounding score threshold.
F. Set a high grounding score threshold.
Answer: A D F
Explanation:
The correct combination is A, D, and F because these guardrail features directly map to the stated financial compliance and governance requirements.
Option A is required because denied topics guardrails are explicitly designed to block entire categories of requests, such as requests for guaranteed returns or specific stock recommendations. These are regulatory- sensitive scenarios where partial filtering is insufficient and full blocking is required to prevent violations.
Option D is correct because custom word filters are the appropriate guardrail mechanism to block references to specific competitor names. Content filters are category-based (such as hate, sexual, or violence-related content) and are not suitable for blocking organization-specific competitor references. Custom word filters allow precise blocking at both input and output stages.
Option F is required because a high grounding score threshold enforces that model outputs must be strongly supported by approved source material. This prevents the AI assistant from making speculative or unfounded claims that are not aligned with the company's approved financial guidance, which is critical in regulated financial environments.
Option B is incorrect because content filters do not target domain-specific financial advice patterns.
Option C is incorrect for the same reason-competitor names are not a content filter category. Option E would weaken factual grounding and increase hallucination risk.
Therefore, A, D, and F together provide topic blocking, competitor exclusion, and factual grounding enforcement.

NO.77 An ecommerce company operates a global product recommendation system that needs to switch between multiple foundation models (FM) in Amazon Bedrock based on regulations, cost optimization, and performance requirements. The company must apply custom controls based on proprietary business logic, including dynamic cost thresholds, AWS Region-specific compliance rules, and real-time A/B testing across multiple FMs.
The system must be able to switch between FMs without deploying new code. The system must route user requests based on complex rules including user tier, transaction value, regulatory zone, and real-time cost metrics that change hourly and require immediate propagation across thousands of concurrent requests.
Which solution will meet these requirements?
A. Deploy an AWS Lambda function that uses environment variables to store routing rules and Amazon Bedrock FM IDs. Use the Lambda console to update the environment variables when business requirements change. Configure an Amazon API Gateway REST API to read request parameters to make routing decisions.
B. Deploy Amazon API Gateway REST API request transformation templates to implement routing logic based on request attributes. Store Amazon Bedrock FM endpoints as REST API stage variables.

Update the variables when the system switches between models.

C. Configure an AWS Lambda function to fetch routing configurations from the AWS AppConfig Agent for each user request. Run business logic in the Lambda function to select the appropriate FM for each request. Expose the FM through a single Amazon API Gateway REST API endpoint.

D. Use AWS Lambda authorizers for an Amazon API Gateway REST API to evaluate routing rules that are stored in AWS AppConfig. Return authorization contexts based on business logic. Route requests to model-specific Lambda functions for each Amazon Bedrock FM.

Answer: C

Explanation:

Option C is the correct solution because AWS AppConfig is designed for real-time, validated, centrally managed configuration changes with safe rollout, immediate propagation, and rollback support-exactly matching the company's requirements.

By storing routing rules, cost thresholds, regulatory constraints, and A/B testing logic in AWS AppConfig, the company can switch between Amazon Bedrock foundation models without redeploying Lambda code.

AppConfig supports feature flags, dynamic configuration updates, JSON schema validation, and staged rollouts, which are essential for safely managing complex and frequently changing routing logic.

Using the AWS AppConfig Agent, Lambda functions can retrieve cached configurations efficiently, ensuring low latency even under thousands of concurrent requests. This approach allows the Lambda function to apply proprietary business logic-such as user tier, transaction value, Region compliance, and real-time cost metrics-before selecting the appropriate FM.

Option A is operationally fragile because environment variable changes require function restarts and do not support validation or controlled rollouts. Option B is too limited for complex, dynamic logic and is difficult to maintain at scale. Option D misuses Lambda authorizers, which are intended for authentication and authorization, not high-frequency dynamic routing decisions.

Therefore, Option C provides the most scalable, flexible, and low-overhead architecture for dynamic, regulation-aware FM routing in a global GenAI system.

NO.78 A bank is building a generative AI (GenAI) application that uses Amazon Bedrock to assess loan applications by using scanned financial documents. The application must extract structured data from the documents. The application must redact personally identifiable information (PII) before inference. The application must use foundation models (FMs) to generate approvals. The application must route low-confidence document extraction results to human reviewers who are within the same AWS Region as the loan applicant.

The company must ensure that the application complies with strict Regional data residency and auditability requirements. The application must be able to scale to handle 25,000 applications each day and provide 99.9% availability.

Which combination of solutions will meet these requirements? (Select THREE.)

A. Deploy Amazon Textract and Amazon Augmented AI within the same Region to extract relevant data from the scanned documents. Route low-confidence pages to human reviewers.

B. Use AWS Lambda functions to detect and redact PII from submitted documents before inference. Apply Amazon Bedrock guardrails to prevent inappropriate or unauthorized content in model outputs.

Configure Region-specific IAM roles to enforce data residency requirements and to control access to

the extracted data.

C. Use Amazon Kendra and Amazon OpenSearch Service to extract field-level values semantically from the uploaded documents before inference.

D. Store uploaded documents in Amazon S3 and apply object metadata. Configure IAM policies to store original documents within the same Region as each applicant. Enable object tagging for future audits.

E. Use AWS Glue Data Quality to validate the structured document data. Use AWS Step Functions to orchestrate a review workflow that includes a prompt engineering step that transforms validated data into optimized prompts before invoking Amazon Bedrock to assess loan applications.

F. Use Amazon SageMaker Clarify to generate fairness and bias reports based on model scoring decisions that Amazon Bedrock makes.

Answer: A B D

Explanation:

The correct combination is A, B, and D because these three options collectively satisfy the mandatory requirements for structured extraction, PII redaction before inference, regional human review, data residency, auditability, and high-scale availability with managed AWS services.

Option A is essential because Amazon Textract is the AWS-managed service designed to extract structured data from scanned documents such as forms, tables, and financial statements. Textract provides confidence scores, and Amazon Augmented AI (A2I) is purpose-built to route low-confidence extractions to human reviewers. Deploying Textract and A2I within the same Region ensures that the human review loop remains regionally constrained, meeting strict data residency requirements for applicants.

Option B satisfies the requirement to redact PII before inference by using AWS Lambda preprocessing. It also adds Amazon Bedrock guardrails to enforce safety controls on model outputs. Region-specific IAM roles ensure that only authorized principals in the correct Region can access the extracted data and invoke downstream services, strengthening residency enforcement and auditability.

Option D ensures that source documents are stored in Amazon S3 in the same Region as the applicant. Object metadata and tagging provide an auditable trail, supporting compliance reporting and traceability. S3 also provides the durability and availability needed to support 99.9% application availability as part of a well- architected pipeline.

Option C is not the correct approach for structured extraction from scans. Option E adds useful quality validation but is not strictly required to meet the stated requirements compared to A, B, and D. Option F is unrelated to the extraction/redaction/residency workflow requirements.

Therefore, A, B, and D are the best three choices to meet all stated requirements with minimal operational overhead.

NO.79 An elevator service company has developed an AI assistant application by using Amazon Bedrock. The application generates elevator maintenance recommendations to support the company's elevator technicians.

The company uses Amazon Kinesis Data Streams to collect the elevator sensor data.

New regulatory rules require that a human technician must review all AI-generated recommendations. The company needs to establish human oversight workflows to review and approve AI recommendations. The company must store all human technician review decisions for audit purposes.

Which solution will meet these requirements?

A. Create a custom approval workflow by using AWS Lambda functions and Amazon SQS queues for human review of AI recommendations. Store all review decisions in Amazon DynamoDB for audit purposes.
B. Create an AWS Step Functions workflow that has a human approval step that uses the waitForTaskToken API to pause execution. After a human technician completes a review, use an AWS Lambda function to call the SendTaskSuccess API with the approval decision. Store all review decisions in Amazon DynamoDB.
C. Create an AWS Glue workflow that has a human approval step. After the human technician review, integrate the application with an AWS Lambda function that calls the SendTaskSuccess API. Store all human technician review decisions in Amazon DynamoDB.
D. Configure Amazon EventBridge rules with custom event patterns to route AI recommendations to human technicians for review. Create AWS Glue jobs to process human technician approval queues.Use Amazon ElastiCache to cache all human technician review decisions.
Answer: B
Explanation:
AWS Step Functions provides native support for human-in-the-loop workflows, making it the best fit for regulatory oversight requirements. The waitForTaskToken integration pattern is explicitly designed to pause a workflow until an external actor-such as a human reviewer-completes a task.
In this architecture, AI-generated recommendations are sent to a human technician for review. The workflow pauses execution using a task token. Once the technician approves or rejects the recommendation, an AWS Lambda function calls SendTaskSuccess or SendTaskFailure, allowing the workflow to continue deterministically.
This approach ensures full auditability, as Step Functions records every state transition, timestamp, and execution path. Storing review outcomes in Amazon DynamoDB provides durable, queryable audit records required for regulatory compliance.
Option A requires custom orchestration and lacks native workflow state management. Option C incorrectly uses AWS Glue, which is not designed for approval workflows. Option D uses caching instead of durable audit storage and introduces unnecessary complexity.
Therefore, Option B is the AWS-recommended, lowest-risk, and most auditable solution for mandatory human review of AI outputs.

NO.80 An ecommerce company is developing a generative AI application that uses Amazon Bedrock with Anthropic Claude to recommend products to customers. Customers report that some recommended products are not available for sale on the website or are not relevant to the customer. Customers also report that the solution takes a long time to generate some recommendations.
The company investigates the issues and finds that most interactions between customers and the product recommendation solution are unique. The company confirms that the solution recommends products that are not in the company's product catalog. The company must resolve these issues.
Which solution will meet this requirement?
A. Increase grounding within Amazon Bedrock Guardrails. Enable Automated Reasoning checks. Set up provisioned throughput.
B. Use prompt engineering to restrict the model responses to relevant products. Use streaming techniques such as the InvokeModelWithResponseStream action to reduce perceived latency for the customers.
C. Create an Amazon Bedrock knowledge base. Implement Retrieval Augmented Generation RAG. Set

the PerformanceConfigLatency parameter to optimized.

D. Store product catalog data in Amazon OpenSearch Service. Validate the model's product recommendations against the product catalog. Use Amazon DynamoDB to implement response caching.

Answer: C

Explanation:

Option C best addresses both core problems: hallucinated recommendations that do not exist in the catalog and slow response times, while keeping operational overhead low. The most direct way to prevent the model from recommending unavailable products is to ground generation on authoritative product catalog data at inference time. An Amazon Bedrock knowledge base is designed for this pattern by ingesting domain data, chunking content, creating embeddings, and retrieving the most relevant catalog entries when a user asks for recommendations. Implementing Retrieval Augmented Generation ensures the foundation model receives only approved, catalog-backed context and can cite or base its output on those retrieved items. This sharply reduces the likelihood of inventing products, because the response is conditioned on retrieved catalog records rather than relying on the model's parametric memory.

The requirement also notes that most interactions are unique. That makes response caching far less effective, because there are fewer repeated prompts to benefit from cached outputs. Instead, improving the retrieval and model invocation path is the better optimization. Using the PerformanceConfigLatency parameter set to optimized prioritizes lower latency behavior for model inference, helping meet faster recommendation generation without requiring the company to build and operate additional infrastructure.

The other options do not solve the root cause as reliably. Prompt engineering and streaming can improve perceived latency, but they do not guarantee catalog-only recommendations because the model can still hallucinate items. Guardrails can help detect or block certain undesired outputs, but without consistent catalog grounding they do not ensure every recommendation is derived from the company's product data. Building a custom OpenSearch validation and caching layer increases operational complexity, and caching is misaligned with predominantly unique interactions.

NO.81 A financial services company is developing a real-time generative AI (GenAI) assistant to support human call center agents. The GenAI assistant must transcribe live customer speech, analyze context, and provide incremental suggestions to call center agents while a customer is still speaking. To preserve responsiveness, the GenAI assistant must maintain end-to-end latency under 1 second from speech to initial response display.

The architecture must use only managed AWS services and must support bidirectional streaming to ensure that call center agents receive updates in real time.

Which solution will meet these requirements?

A. Use Amazon Transcribe streaming to transcribe calls. Pass the text to Amazon Comprehend for sentiment analysis. Feed the results to Anthropic Claude on Amazon Bedrock by using the InvokeModel API. Store results in Amazon DynamoDB. Use a WebSocket API to display the results.

B. Use Amazon Transcribe streaming with partial results enabled to deliver fragments of transcribed text before customers finish speaking. Forward text fragments to Amazon Bedrock by using the InvokeModelWithResponseStream API. Stream responses to call center agents through an Amazon API Gateway WebSocket API.

C. Use Amazon Transcribe batch processing to convert calls to text. Pass complete transcripts to

Anthropic Claude on Amazon Bedrock by using the ConverseStream API. Return responses through an Amazon Lex chatbot interface.

D. Use the Amazon Transcribe streaming API with an AWS Lambda function to transcribe each audio segment. Call the Amazon Titan Embeddings model on Amazon Bedrock by using the InvokeModel API. Publish results to Amazon SNS.

Answer: B

Explanation:

Option B is the only solution that satisfies all strict real-time, streaming, and latency requirements. Amazon Transcribe streaming with partial results allows transcription fragments to be delivered before the speaker finishes a sentence. This significantly reduces perceived latency and enables downstream processing to begin immediately, which is essential for maintaining sub-1-second end-to-end response times.

Using Amazon Bedrock's InvokeModelWithResponseStream API enables token-level or chunk-level streaming responses from the foundation model. This allows the GenAI assistant to begin delivering suggestions to call center agents incrementally instead of waiting for a full model response. This streaming inference capability is critical for interactive, real-time agent assistance use cases.

Amazon API Gateway WebSocket APIs provide fully managed, bidirectional communication between backend services and agent dashboards. This ensures that updates flow continuously to agents as new transcription fragments and model outputs become available, preserving real-time responsiveness without requiring custom socket infrastructure.

Option A introduces additional synchronous processing layers and storage writes that increase latency. Option C uses batch transcription and post-call processing, which cannot meet real-time requirements. Option D uses embeddings and asynchronous messaging, which are not suitable for live incremental suggestions and bidirectional streaming.

Therefore, Option B best aligns with AWS real-time GenAI architecture patterns by combining streaming transcription, streaming model inference, and managed bidirectional communication while maintaining low latency and operational simplicity.

NO.82 A company runs a Retrieval Augmented Generation (RAG) application that uses Amazon Bedrock Knowledge Bases to perform regulatory compliance queries. The application uses the RetrieveAndGenerateStream API. The application retrieves relevant documents from a knowledge base that contains more than 50,000 regulatory documents, legal precedents, and policy updates. The RAG application is producing suboptimal responses because the initial retrieval often returns semantically similar but contextually irrelevant documents. The poor responses are causing model hallucinations and incorrect regulatory guidance. The company needs to improve the performance of the RAG application so it returns more relevant documents.

Which solution will meet this requirement with the LEAST operational overhead?

A. Deploy an Amazon SageMaker endpoint to run a fine-tuned ranking model. Use an Amazon API Gateway REST API to route requests. Configure the application to make requests through the REST API to rerank the results.

B. Use Amazon Comprehend to classify documents and apply relevance scores. Integrate the RAG application's reranking process with Amazon Textract to run document analysis. Use Amazon Neptune to perform graph-based relevance calculations.

C. Implement a retrieval pipeline that uses the Amazon Bedrock Knowledge Bases Retrieve API to perform initial document retrieval. Call the Amazon Bedrock Rerank API to rerank the results. Invoke

the InvokeModelWithResponseStream operation to generate responses.

D. Use the latest Amazon reranker model through the reranking configuration within Amazon Bedrock Knowledge Bases. Use the model to improve document relevance scoring and to reorder results based on contextual assessments.

Answer: D

Explanation:

Option D is the correct solution because Amazon Bedrock Knowledge Bases natively support reranking by using Amazon-managed reranker models, which are specifically designed to improve contextual relevance after the initial vector retrieval step. This approach directly addresses the root cause of the issue: semantically similar but contextually irrelevant documents being passed to the foundation model.

By enabling the reranking configuration within Amazon Bedrock Knowledge Bases, the application can automatically reorder retrieved documents based on deeper contextual understanding, such as regulatory scope, legal applicability, and semantic intent. This significantly improves retrieval precision, which reduces hallucinations and improves the factual accuracy of generated regulatory guidance.

Option D requires no additional infrastructure, no custom orchestration logic, and no separate model hosting.

The reranking is fully managed by Amazon Bedrock and integrates seamlessly with the existing RetrieveAndGenerateStream workflow. This makes it the lowest operational overhead solution.

Option A introduces operational complexity by requiring a custom SageMaker endpoint, API Gateway routing, and model lifecycle management. Option B combines multiple unrelated services and introduces significant complexity without being purpose-built for RAG relevance ranking. Option C improves relevance but requires explicitly calling the Rerank API and modifying the application pipeline, which increases operational and integration effort compared to built-in reranking.

Therefore, Option D provides the most efficient, scalable, and AWS-recommended method to improve RAG retrieval quality while minimizing operational burden.

NO.83 A healthcare company uses Amazon Bedrock to deploy an application that generates summaries of clinical documents. The application experiences inconsistent response quality with occasional factual hallucinations.

Monthly costs exceed the company's projections by 40%. A GenAI developer must implement a near real- time monitoring solution to detect hallucinations, identify abnormal token consumption, and provide early warnings of cost anomalies. The solution must require minimal custom development work and maintenance overhead.

Which solution will meet these requirements?

A. Configure Amazon CloudWatch alarms to monitor InputTokenCount and OutputTokenCount metrics to detect anomalies. Store model invocation logs in an Amazon S3 bucket. Use AWS Glue and Amazon Athena to identify potential hallucinations.

B. Run Amazon Bedrock evaluation jobs that use LLM-based judgments to detect hallucinations. Configure Amazon CloudWatch to track token usage. Create an AWS Lambda function to process CloudWatch metrics. Configure the Lambda function to send usage pattern notifications.

C. Configure Amazon Bedrock to store model invocation logs in an Amazon S3 bucket. Enable text output logging. Configure Amazon Bedrock guardrails to run contextual grounding checks to detect hallucinations. Create Amazon CloudWatch anomaly detection alarms for token usage metrics.

D. Use AWS CloudTrail to log all Amazon Bedrock API calls. Create a custom dashboard in Amazon QuickSight to visualize token usage patterns. Use Amazon SageMaker Model Monitor to detect quality drift in generated summaries.

Answer: C

Explanation:

Option C is the correct solution because it provides near real-time monitoring, hallucination detection, and cost anomaly awareness using built-in Amazon Bedrock and Amazon CloudWatch capabilities, with minimal custom development.

By configuring Amazon Bedrock invocation logging with text output logging, the company captures detailed prompt and response data for auditing and analysis without building custom logging pipelines. This data is stored in Amazon S3, providing durable storage for compliance and retrospective investigation.

Using Amazon Bedrock guardrails with contextual grounding checks allows the application to automatically detect hallucinations by verifying whether generated summaries are grounded in the provided clinical documents. This is the AWS-recommended approach for hallucination detection in RAG and summarization workloads and avoids the need to maintain custom evaluation models or pipelines.

Creating Amazon CloudWatch anomaly detection alarms for InputTokenCount and OutputTokenCount metrics enables automatic detection of abnormal token usage patterns that often correlate with runaway prompts, inefficient summarization, or prompt injection attempts. Anomaly detection adapts dynamically to usage trends, making it more effective than static thresholds for early cost warnings.

Option A introduces batch analytics with Glue and Athena, which is not near real time and increases operational overhead. Option B requires managing evaluation jobs and Lambda-based notification logic.

Option D focuses on infrastructure-level monitoring and offline dashboards rather than near real-time GenAI quality and cost signals.

Therefore, Option C best meets the requirements with the least operational effort and maintenance overhead.

NO.84 A company wants to select a new FM for its AI assistant. A GenAI developer needs to generate evaluation reports to help a data scientist assess the quality and safety of various foundation models FMs. The data scientist provides the GenAI developer with sample prompts for evaluation. The GenAI developer wants to use Amazon Bedrock to automate report generation and evaluation.

Which solution will meet this requirement?

A. Combine the sample prompts into a single JSON document. Create an Amazon Bedrock knowledge base with the document. Write a prompt that asks the FM to generate a response to each sample prompt.

Use the RetrieveAndGenerate API to generate a report for each model.

B. Combine the sample prompts into a single JSONL document. Store the document in an Amazon S3 bucket. Create an Amazon Bedrock evaluation job that uses a judge model. Specify the S3 location as input and a different S3 location as output. Run an evaluation job for each FM and select the FM as the generator.

C. Combine the sample prompts into a single JSONL document. Store the document in an Amazon S3

bucket. Create an Amazon Bedrock evaluation job that uses a judge model. Specify the S3 location as input and Amazon QuickSight as output. Run an evaluation job for each FM and select the FM as the evaluator.

D. Combine the sample prompts into a single JSON document. Create an Amazon Bedrock knowledge base from the document. Create an Amazon Bedrock evaluation job that uses the retrieval and response generation evaluation type. Specify an Amazon S3 bucket as the output. Run an evaluation job for each FM.

Answer: B

Explanation:

Option B is correct because it uses the managed evaluation capability in Amazon Bedrock that is intended specifically for comparing foundation models using a consistent prompt set and producing structured results with minimal custom tooling. In a Bedrock evaluation workflow, you provide an input dataset of prompts, typically in JSON Lines format so each line represents one evaluation record. Storing the JSONL file in Amazon S3 allows Bedrock to read the dataset at scale and write standardized evaluation outputs back to S3 for downstream analysis, sharing, and retention.

The key requirement is to assess both quality and safety across multiple models. A Bedrock evaluation job can use a judge model to score the generated outputs against defined criteria. This approach supports repeatable, apples-to-apples comparisons because the same judge model and scoring rubric can be applied to every candidate foundation model. The candidate models are configured as generators, meaning each evaluation job run uses one selected FM to produce answers for the same prompt set, and the judge model evaluates those answers. That matches the requirement to generate evaluation reports that help a data scientist select the best FM.

Option A does not use Bedrock evaluation jobs, and a knowledge base plus RetrieveAndGenerate is a RAG pattern, not an evaluation framework. It would produce responses but not standardized scoring and reporting suitable for model selection. Option C is incorrect because Bedrock evaluation outputs are delivered to S3, not directly to a BI destination, and selecting the candidate FM as the evaluator conflicts with the intended pattern of using a stable judge model. Option D misuses knowledge bases and retrieval evaluation types when the requirement is prompt-based model assessment rather than evaluating retrieval quality.

NO.85 A company is implementing a serverless inference API by using AWS Lambda. The API will dynamically invoke multiple AI models hosted on Amazon Bedrock. The company needs to design a solution that can switch between model providers without modifying or redeploying Lambda code in real time. The design must include safe rollout of configuration changes and validation and rollback capabilities.

Which solution will meet these requirements?

A. Store the active model provider in AWS Systems Manager Parameter Store. Configure a Lambda function to read the parameter at runtime to determine which model to invoke.

B. Store the active model provider in AWS AppConfig. Configure a Lambda function to read the configuration at runtime to determine which model to invoke.

C. Configure an Amazon API Gateway REST API to route requests to separate Lambda functions. Hardcode each Lambda function to a specific model provider. Switch the integration target manually.

D. Store the active model provider in a JSON file hosted on Amazon S3. Use AWS AppConfig to reference the S3 file as a hosted configuration source. Configure a Lambda function to read the file through AppConfig at runtime to determine which model to invoke.

Answer: B
Explanation:
Option B is the correct solution because AWS AppConfig is specifically designed to support dynamic configuration management with safe rollout, validation, and rollback, which are explicit requirements in the scenario.
By storing the active model provider configuration in AWS AppConfig, the company can switch between Amazon Bedrock model providers in real time without redeploying Lambda code. AppConfig supports deployment strategies such as canary releases, linear rollouts, and immediate deployments, allowing safe and controlled changes. If a configuration causes issues, AppConfig supports automatic rollback, reducing operational risk.
AWS AppConfig also supports schema validation, ensuring that configuration values such as model identifiers, provider names, or inference parameters are valid before being applied. This prevents misconfiguration from impacting production workloads.
Option A uses Parameter Store, which lacks native rollout strategies, validation, and automated rollback, making it unsuitable for safe real-time switching. Option C requires manual routing changes and code coupling, increasing operational overhead and deployment risk. Option D introduces unnecessary complexity by hosting configuration files in Amazon S3 when AppConfig already supports native hosted configurations.
Therefore, Option B provides the most robust, scalable, and low-maintenance solution for dynamic model switching in a serverless Amazon Bedrock inference architecture.

NO.86 A university recently digitized a collection of archival documents, academic journals, and manuscripts. The university stores the digital files in an AWS Lake Formation data lake.
The university hires a GenAI developer to build a solution to allow users to search the digital files by using text queries. The solution must return journal abstracts that are semantically similar to a user's query. Users must be able to search the digitized collection based on text and metadata that is associated with the journal abstracts. The metadata of the digitized files does not contain keywords. The solution must match similar abstracts to one another based on the similarity of their text. The data lake contains fewer than 1 million files.
Which solution will meet these requirements with the LEAST operational overhead?

A. Use Amazon Titan Embeddings in Amazon Bedrock to create vector representations of the digitized files. Store embeddings in the OpenSearch Neural plugin for Amazon OpenSearch Service.

B. Use Amazon Comprehend to extract topics from the digitized files. Store the topics and file metadata in an Amazon Aurora PostgreSQL database. Query the abstract metadata against the data in the Aurora database.

C. Use Amazon SageMaker AI to deploy a sentence-transformer model. Use the model to create vector representations of the digitized files. Store embeddings in an Amazon Aurora PostgreSQL database that has the pgvector extension.

D. Use Amazon Titan Embeddings in Amazon Bedrock to create vector representations of the digitized files. Store embeddings in an Amazon Aurora PostgreSQL Serverless database that has the pgvector extension.

Answer: D
Explanation:
Option D is the best choice because it delivers true semantic search with the smallest operational footprint by combining a fully managed embedding service with an automatically scaling vector-

capable database. The university's requirement is explicitly semantic: the metadata has no keywords, and the system must match abstracts based on similarity of meaning. This is a direct fit for an embeddings-based approach, where each abstract is converted into a vector representation and searched using vector similarity. Amazon Titan Embeddings in Amazon Bedrock provides a managed way to generate these vectors without hosting or maintaining an ML model, eliminating the operational work of model provisioning, patching, scaling, and lifecycle management.
For storage and retrieval, Amazon Aurora PostgreSQL Serverless with the pgvector extension supports vector storage and similarity search while minimizing infrastructure operations. Aurora Serverless reduces capacity planning and scaling tasks because it can automatically adjust to changes in workload, which is valuable for a university search application with variable usage patterns. With fewer than 1 million files, a PostgreSQL-based vector store is commonly operationally simpler than running a dedicated search cluster, while still meeting the requirement to query using both text-derived similarity and associated metadata filters stored alongside the vectors.
Option A can also enable vector search, but operating an OpenSearch domain typically introduces additional concerns such as domain sizing, shard strategy, cluster scaling, and performance tuning for k-NN workloads.
Option C increases operational overhead the most because it requires deploying and operating a sentence- transformer model endpoint in SageMaker AI, including scaling, monitoring, and model management. Option B does not meet the semantic similarity requirement reliably because topic extraction is not equivalent to embedding-based semantic matching, especially when the metadata lacks keywords and the system must compare abstracts by meaning.
Therefore, D best satisfies semantic search needs with the least operational overhead.

NO.87 A financial services company uses an AI application to process financial documents by using Amazon Bedrock. During business hours, the application handles approximately 10,000 requests each hour, which requires consistent throughput.
The company uses the CreateProvisionedModelThroughput API to purchase provisioned throughput. Amazon CloudWatch metrics show that the provisioned capacity is unused while on-demand requests are being throttled. The company finds the following code in the application:

```
response = bedrock_runtime.invoke_model(
modelId="anthropic.claude-v2",
body=json.dumps(payload)
)
```

The company needs the application to use the provisioned throughput and to resolve the throttling issues.
Which solution will meet these requirements?

A. Increase the number of model units (MUs) in the provisioned throughput configuration.

B. Replace the model ID parameter with the ARN of the provisioned model that the CreateProvisionedModelThroughput API returns.

C. Add exponential backoff retry logic to handle throttling exceptions during peak hours.

D. Modify the application to use the invokeModelWithResponseStream API instead of the invokeModel API.

Answer: B

Explanation:
Option B is the correct solution because Amazon Bedrock provisioned throughput is only used when the application explicitly invokes the provisioned model ARN, not the base foundation model ID. In

the provided code, the application is calling the standard model identifier (anthropic.claude-v2), which routes requests to on-demand capacity instead of the purchased provisioned throughput. When the CreateProvisionedModelThroughput API is used, Amazon Bedrock returns a provisioned model ARN that represents the reserved capacity. Applications must reference this ARN in the modelId parameter when invoking the model. If the base model ID is used instead, Bedrock treats the request as on-demand traffic, which explains why CloudWatch metrics show unused provisioned capacity alongside throttled on- demand requests.
Option A would increase capacity but would not fix the root cause because the application is not using the provisioned resource at all. Option C adds resiliency but does not ensure usage of provisioned throughput and would still incur throttling. Option D changes the response delivery mechanism but does not affect capacity routing.
Therefore, Option B directly resolves the throttling issue by correctly routing traffic to the reserved capacity and ensures that the company benefits from the provisioned throughput it has purchased.

NO.88 A company developed a multimodal content analysis application by using Amazon Bedrock. The application routes different content types (text, images, and code) to specialized foundation models (FMs).
The application needs to handle multiple types of routing decisions. Simple routing based on file extension must have minimal latency. Complex routing based on content semantics requires analysis before FM selection. The application must provide detailed history and support fallback options when primary FMs fail.
Which solution will meet these requirements?
A. Configure AWS Lambda functions that call Amazon Bedrock FMs for all routing logic. Use conditional statements to determine the appropriate FM based on content type and semantics.
B. Create a hybrid solution. Handle simple routing based on file extensions in application code. Handle complex content-based routing by using an AWS Step Functions state machine with JSONata for content analysis and the InvokeModel API for specialized FMs.
C. Deploy separate AWS Step Functions workflows for each content type with routing logic in AWS Lambda functions. Use Amazon EventBridge to coordinate between workflows when fallback to alternate FMs is required.
D. Use Amazon SQS with different SQS queues for each content type. Configure AWS Lambda consumers that analyze content and invoke appropriate FMs based on message attributes by using Amazon Bedrock with an AWS SDK.
Answer: B
Explanation:
Option B is the most appropriate solution because it directly aligns with AWS-recommended architectural patterns for building scalable, observable, and resilient generative AI applications on Amazon Bedrock. The requirements clearly distinguish between simple and complex routing decisions, and this option addresses both in an optimal way.
Simple routing based on file extension is latency sensitive. Handling this logic directly in the application code avoids unnecessary orchestration, state transitions, and service calls. This approach ensures that straightforward requests, such as routing images to vision-capable foundation models or text files to language models, are processed with minimal overhead and maximum performance.
For complex routing based on content semantics, AWS Step Functions is specifically designed for multi-step workflows that require analysis, branching logic, and error handling. Semantic routing

often requires inspecting meaning, intent, or structure before selecting the appropriate foundation model. Step Functions enables this by orchestrating analysis steps and applying conditional logic to determine the correct model to invoke using the Amazon Bedrock InvokeModel API.
A key requirement is detailed execution history. Step Functions provides built-in execution tracing, including state inputs, outputs, and error details, which is essential for auditing, debugging, and compliance.
Additionally, Step Functions supports native retry and catch mechanisms, allowing the workflow to automatically fall back to alternate foundation models if a primary model invocation fails. This directly satisfies the fallback requirement without introducing excessive custom code.
The other options lack one or more critical capabilities. Lambda-only logic lacks deep observability and structured fallback handling, SQS introduces additional latency and limited workflow visibility, and multiple coordinated workflows increase architectural complexity without added benefit.

NO.89 A company uses AWS Lake Formation to set up a data lake that contains databases and tables for multiple business units across multiple AWS Regions. The company wants to use a foundation model (FM) through Amazon Bedrock to perform fraud detection. The FM must ingest sensitive financial data from the data lake.
The data includes some customer personally identifiable information (PII).
The company must design an access control solution that prevents PII from appearing in a production environment. The FM must access only authorized data subsets that have PII redacted from specific data columns. The company must capture audit trails for all data access.
Which solution will meet these requirements?

A. Create a separate dataset in a separate Amazon S3 bucket for each business unit and Region combination. Configure S3 bucket policies to control access based on IAM roles that are assigned to FM training instances. Use S3 access logs to track data access.

B. Configure the FM to authenticate by using AWS Identity and Access Management roles and Lake Formation permissions based on LF-Tag expressions. Define business units and Regions as LF-Tags that are assigned to databases and tables. Use AWS CloudTrail to collect comprehensive audit trails of data access.

C. Use direct IAM principal grants on specific databases and tables in Lake Formation. Create a custom application layer that logs access requests and further filters sensitive columns before sending data to the FM.

D. Configure the FM to request temporary credentials from AWS Security Token Service. Access the data by using presigned S3 URLs that are generated by an API that applies business unit and Regional filters. Use AWS CloudTrail to collect comprehensive audit trails of data access.

Answer: B

Explanation:
Option B is the correct solution because it uses native AWS governance, access control, and auditing capabilities to protect PII while enabling controlled FM access to authorized data subsets. AWS Lake Formation is designed specifically to manage fine-grained permissions for data lakes, including column-level access control, which is critical when handling sensitive financial and PII data.
LF-Tags allow data administrators to define scalable, attribute-based access control policies. By tagging databases, tables, and columns with business unit and Region metadata, the company can enforce policies that ensure the foundation model only accesses approved datasets with PII-redacted columns. This eliminates the risk of sensitive data leaking into production inference workflows.

IAM role-based authentication ensures that the FM accesses data using least-privilege credentials. This integrates cleanly with Amazon Bedrock, which supports IAM-based authorization for service-to-service access. AWS CloudTrail provides immutable audit logs for all access attempts, satisfying compliance and regulatory requirements.
Option A introduces unnecessary data duplication and weak governance controls. Option C relies on custom application logic, increasing operational risk and complexity. Option D bypasses Lake Formation's fine- grained controls and relies on presigned URLs, which reduces governance visibility and control.
Therefore, Option B best meets the requirements for security, compliance, scalability, and auditability when integrating Amazon Bedrock with a Lake Formation-governed data lake.

NO.90 An ecommerce company is building an internal platform to develop generative AI applications by using Amazon Bedrock foundation models (FMs). Developers need to select models based on evaluations that are aligned to ecommerce use cases. The platform must display accuracy metrics for text generation and summarization in dashboards. The company has custom ecommerce datasets to use as standardized evaluation inputs.
Which combination of steps will meet these requirements with the LEAST operational overhead? (Select TWO.)
A. Import the datasets to an Amazon S3 bucket. Provide appropriate IAM permissions and cross-origin resource sharing (CORS) permissions to give the evaluation jobs access to the datasets.
B. Import the datasets to an Amazon S3 bucket. Provide appropriate IAM permissions and a VPC endpoint configuration to give the evaluation jobs access to the datasets.
C. Configure an AWS Lambda function to create model evaluation jobs on a schedule in the Amazon Bedrock console. Provide the URI of the S3 bucket that contains the datasets as an input. Configure the evaluation jobs to measure the real world knowledge (RWK) score for text generation and BERTScore for summarization. Configure a second Lambda function to check the status of the jobs and publish custom logs to Amazon CloudWatch. Create a custom Amazon CloudWatch Logs Insights dashboard.
D. Use Amazon SageMaker Clarify on a schedule to create model evaluation jobs. Use open source frameworks to create and run standardized evaluations. Publish results to Amazon CloudWatch namespaces. Use an AWS Lambda function to check the status of the jobs and publish custom logs to Amazon CloudWatch. Create a custom Amazon CloudWatch Logs Insights dashboard.
E. Run an Amazon SageMaker AI notebook job on a schedule by using the fmvelos or ragas framework to run evaluations that use the datasets in the S3 bucket. Write Python code in the notebook that makes direct InvokeModel API calls to the FMs and processes their responses for evaluation. Publish job status and results to Amazon CloudWatch Logs to measure the real world knowledge (RWK) score for text generation and toxicity for summarization as metrics for accuracy. Create a custom CloudWatch Logs Insights dashboard.
Answer: B C
Explanation:
The least operational overhead approach is to use managed Amazon Bedrock model evaluation workflows with datasets stored in Amazon S3, and then publish results into Amazon CloudWatch for dashboards. That is exactly what options B and C combine.
Step B correctly places standardized evaluation inputs in Amazon S3 and focuses on granting the evaluation workflow the right permissions to read those datasets. In practice, the key requirement is

controlled access to the S3 objects used as evaluation datasets. Establishing IAM permissions and private access patterns (such as using VPC connectivity patterns where applicable to the organization's networking posture) is aligned with enterprise requirements and avoids building custom storage or data distribution systems for evaluators.
Step C then operationalizes the evaluation lifecycle with minimal infrastructure: a scheduled AWS Lambda function starts evaluation jobs using the S3 dataset location, and a second Lambda function checks job status and pushes results and operational signals to CloudWatch. This meets the platform requirement to surface accuracy metrics in dashboards because CloudWatch metrics/logs can be visualized in dashboards and queried through CloudWatch Logs Insights. It also supports continuous, standardized comparisons across models without requiring developers to run ad-hoc experiments.
The alternatives introduce more operational burden. D and E rely on Amazon SageMaker-based tooling, notebook jobs, and open source evaluation frameworks, which require more environment management, dependency control, scaling considerations, and maintenance over time. A includes CORS, which is primarily a browser-access concern and does not address how Bedrock-managed evaluation jobs securely access S3 in the typical service-to-service pattern.
Therefore, B + C achieves standardized model evaluation, automated scheduling, and dashboard-ready observability with the smallest operations footprint.

NO.91 A wildlife conservation agency operates zoos globally. The agency uses various sensors, trackers, and audiovisual recorders to monitor animal behavior. The agency wants to launch a generative AI (GenAI) assistant that can ingest multimodal data to study animal behavior.
The GenAI assistant must support natural language queries, avoid speculative behavioral interpretations, and maintain audit logs for ethical research audits.
Which solution will meet these requirements?

A. Ingest raw videos into Amazon Rekognition to detect animal postures and expressions. Use Amazon Data Firehose to stream sensor and GPS data into Amazon S3. Prompt an Amazon Bedrock FM using basic templates stored in AWS Systems Manager Parameter Store. Use IAM for access control. Use AWS CloudTrail for audit logging.

B. Use Amazon SageMaker Processing and Amazon Transcribe to pre-process multimodal data. Ingest curated summaries into an Amazon Bedrock Knowledge Bases. Apply Amazon Bedrock guardrails to restrict speculative outputs. Use AWS AppConfig to manage prompt templates. Use AWS CloudTrail to log research activity for audits.

C. Use Amazon OpenSearch Serverless to index behavioral logs and telemetry. Use Amazon Comprehend to extract entities. Use Amazon Bedrock to answer questions over indexed data. Use IAM for access control and CloudTrail for audit logging.

D. Configure Amazon O Business to federate data across Amazon S3, Amazon Kinesis, and Amazon SageMaker Feature Store. Use EventBridge for ingestion orchestration. Use custom AWS Lambda functions to filter LLM outputs for ethical compliance.

Answer: B

Explanation:
Option B best meets the multimodal, ethical, and auditability requirements using managed AWS services designed for research-grade GenAI systems. Multimodal data such as audio, video, sensor telemetry, and tracking data must be curated and summarized before being consumed by a foundation model. Amazon SageMaker Processing and Amazon Transcribe provide scalable, managed preprocessing for audiovisual and textual data.

By ingesting summarized, validated observations into Amazon Bedrock Knowledge Bases, the GenAI assistant can answer natural language queries using grounded, evidence-based context instead of raw sensor signals. This significantly reduces the risk of speculative or anthropomorphic interpretations.
Amazon Bedrock guardrails are critical for preventing speculative behavioral claims, enforcing scientific and ethical constraints at inference time. Guardrails provide a validated, auditable safety layer that custom Lambda-based filters cannot reliably replicate.
AWS AppConfig enables controlled prompt management and change governance, ensuring that research prompts remain consistent and reviewable. AWS CloudTrail captures all access, query, and configuration changes, supporting ethical research audits and regulatory reviews.
Option A lacks grounding and speculative safeguards. Option C focuses on text analytics and does not properly handle multimodal reasoning or safety enforcement. Option D relies heavily on custom logic and introduces unnecessary operational risk.
Therefore, Option B provides the most robust, ethical, and auditable GenAI architecture for wildlife behavior research.

NO.92 A financial services company is developing a customer service AI assistant application that uses a foundation model (FM) in Amazon Bedrock. The application must provide transparent responses by documenting reasoning and by citing sources that are used for Retrieval Augmented Generation (RAG). The application must capture comprehensive audit trails for all responses to users. The application must be able to serve up to
10,000 concurrent users and must respond to each customer inquiry within 2 seconds.
Which solution will meet these requirements with the LEAST operational overhead?

A. Enable tracing for Amazon Bedrock Agents. Configure structured prompts that direct the FM to provide evidence presentations. Integrate Amazon Bedrock Knowledge Bases with data sources to enable RAG.
Configure the application to reference and cite authoritative content. Deploy the application in a Multi- AZ architecture. Use Amazon API Gateway and AWS Lambda functions to scale the application. Use Amazon CloudFront to provide low-latency delivery.

B. Enable tracing for Amazon Bedrock agents. Integrate a custom RAG pipeline with Amazon OpenSearch Service to retrieve and cite sources. Configure structured prompts to present retrieved evidence. Deploy the application behind an Amazon API Gateway REST API. Use AWS Lambda functions and Amazon CloudFront to scale the application and to provide low latency. Store logs in Amazon S3 and use AWS CloudTrail to capture audit trails.

C. Use Amazon CloudWatch to monitor latency and error rates. Embed model prompts directly in the application backend to cite sources. Store application interactions with users in Amazon RDS for audits.

D. Store generated responses and supporting evidence in an Amazon S3 bucket. Enable versioning on the bucket for audits. Use AWS Glue to catalog retrieved documents. Process the retrieved documents in Amazon Athena to generate periodic compliance reports.

Answer: A

Explanation:
Option A is the correct solution because it relies on native Amazon Bedrock capabilities to deliver transparency, auditability, scalability, and low latency with minimal operational overhead. Amazon Bedrock Knowledge Bases provide a fully managed Retrieval Augmented Generation (RAG)

implementation that automatically handles document ingestion, embedding, retrieval, and source attribution, enabling the application to cite authoritative content without building custom pipelines. Enabling tracing for Amazon Bedrock Agents provides end-to-end visibility into agent reasoning steps, tool usage, and model interactions. This satisfies the requirement for comprehensive audit trails and supports regulatory review in financial services environments. Structured prompts further ensure that responses explicitly present reasoning and supporting evidence in a controlled, auditable format. Using Amazon API Gateway and AWS Lambda allows the application to scale automatically to thousands of concurrent users without capacity planning. These services are designed for bursty workloads and can easily support the stated requirement of up to 10,000 concurrent users. Amazon CloudFront reduces latency by caching and accelerating content delivery, helping the application meet the strict 2-second response-time requirement.
Option B introduces a custom RAG pipeline with OpenSearch, increasing operational complexity and maintenance effort. Option C lacks native RAG integration and does not provide transparent reasoning or citation management. Option D focuses on offline compliance reporting rather than real-time transparency and low-latency responses.
Therefore, Option A best meets all requirements while minimizing infrastructure and operational overhead.

NO.93 A company is using Amazon Bedrock to develop a customer support AI assistant. The AI assistant must respond to customer questions about their accounts. The AI assistant must not expose personal information in responses. The company must comply with data residency policies by ensuring that all processing occurs within the same AWS Region where each customer is located.
The company wants to evaluate how effective the AI assistant is at preventing the exposure of personal information before the company makes the AI assistant available to customers.
Which solution will meet these requirements?

A. Configure a cross-Region Amazon Bedrock guardrail to apply sensitive information filters. Set the guardrail to detect mode during development and testing. Switch to block mode for production deployment.

B. Configure an Amazon Bedrock guardrail to apply sensitive information filters. Set the guardrail to mask mode during development and testing. Switch to block mode for production deployment. Deploy a copy of the guardrail to each Region where the company operates.

C. Configure an Amazon Bedrock guardrail to apply content and topic filters. Set the guardrail to detect mode during development, testing, and production. Disable invocation logging for the Amazon Bedrock model.

D. Configure a cross-Region Amazon Bedrock guardrail to apply a set of content and word filters. Set the guardrail to detect mode during development and testing. Switch to mask mode for production deployment.

Answer: B
Explanation:
Option B best meets all stated requirements by correctly combining PII protection, evaluation before launch
, and data residency compliance using Amazon Bedrock Guardrails. Amazon Bedrock guardrails provide native sensitive information filtering that operates inline during model invocation, making them well suited for preventing personal data exposure in customer-facing AI assistants.
The requirement to evaluate how effective the AI assistant is at preventing exposure before release is

best addressed by using mask mode during development and testing. Mask mode allows responses to be generated while automatically redacting detected personal information, making it easy for developers and reviewers to see where and how PII would have appeared. This provides concrete validation that the guardrail rules are correctly configured without fully blocking responses, which is ideal for quality assurance and pre- production evaluation.
For production, switching the guardrail to block mode ensures that responses containing personal information are fully prevented from being returned to users. This offers the strongest protection and aligns with compliance expectations for customer account data. Block mode is appropriate once confidence in the guardrail configuration has been established during testing.
The data residency requirement is addressed by deploying a copy of the guardrail in each AWS Region where the application operates. Amazon Bedrock guardrails are Region-specific resources, and using Region- local guardrails ensures that inference, filtering, and enforcement all occur within the same Region as the customer data. This avoids cross-Region processing and helps the company comply with regulatory and contractual data residency policies.
Option A and D incorrectly rely on cross-Region guardrails, which can violate data residency constraints.
Option C focuses on topic filtering rather than sensitive information filtering and keeps detect mode enabled in production, which does not actively prevent PII exposure. Therefore, B is the only option that fully satisfies safety, compliance, and evaluation requirements.

NO.94 A company is building a generative AI (GenAI) application that produces content based on a variety of internal and external data sources. The company wants to ensure that the generated output is fully traceable.
The application must support data source registration and enable metadata tagging to attribute content to its original source. The application must also maintain audit logs of data access and usage throughout the pipeline.
Which solution will meet these requirements?

A. Use AWS Lake Formation to catalog data sources and control access. Apply metadata tags directly in Amazon S3. Use AWS CloudTrail to monitor API activity.

B. Use AWS Glue Data Catalog to register and tag data sources. Use Amazon CloudWatch Logs to monitor access patterns and application behavior.

C. Store data in Amazon S3 and use object tagging for attribution. Use AWS Glue Data Catalog to manage schema information. Use AWS CloudTrail to log access to S3 buckets.

D. Use AWS Glue Data Catalog to register all data sources. Apply metadata tags to attribute data sources.Use AWS CloudTrail to log access and activity across services.

Answer: D
Explanation:
Option D is the correct solution because it directly satisfies all three core requirements: data source registration, metadata-based attribution, and end-to-end audit logging, while remaining service-agnostic and scalable across internal and external data sources.
The AWS Glue Data Catalog is the AWS-native service for registering datasets and managing metadata centrally. It supports structured registration of diverse data sources and enables consistent tagging that can be used to attribute generated content back to its original source. This is essential for GenAI applications that combine multiple datasets and must provide traceability for outputs.
Metadata tags applied within the Glue Data Catalog ensure a consistent attribution framework that downstream systems-such as Retrieval Augmented Generation (RAG) pipelines or evaluation

systems-can reference without embedding attribution logic directly in application code. This improves maintainability and governance.
AWS CloudTrail provides immutable audit logs of API activity across AWS services, including data access, metadata changes, and pipeline interactions. CloudTrail logs are critical for compliance and regulatory review because they capture who accessed which data, when, and through which service. This satisfies the requirement to maintain audit logs "throughout the pipeline," not just at storage or application layers.
Option A introduces Lake Formation, which is primarily intended for fine-grained data lake permissions and is not required solely for traceability. Option B relies on CloudWatch Logs, which does not provide authoritative audit logging across services. Option C limits audit scope to S3 access and does not register or govern all data sources comprehensively.
Therefore, Option D provides the most complete and least intrusive solution for traceable, auditable GenAI data pipelines.

NO.95 A retail company has a generative AI (GenAI) product recommendation application that uses Amazon Bedrock. The application suggests products to customers based on browsing history and demographics. The company needs to implement fairness evaluation across multiple demographic groups to detect and measure bias in recommendations between two prompt approaches. The company wants to collect and monitor fairness metrics in real time. The company must receive an alert if the fairness metrics show a discrepancy of more than 15% between demographic groups. The company must receive weekly reports that compare the performance of the two prompt approaches. Which solution will meet these requirements with the LEAST custom development effort?

A. Configure an Amazon CloudWatch dashboard to display default metrics from Amazon Bedrock API calls. Create custom metrics based on model outputs. Set up Amazon EventBridge rules to invoke AWS Lambda functions that perform post-processing analysis on model responses and publish custom fairness metrics.

B. Create the two prompt variants in Amazon Bedrock Prompt Management. Use Amazon Bedrock Flows to deploy the prompt variants with defined traffic allocation. Configure Amazon Bedrock guardrails to monitor demographic fairness. Set up Amazon CloudWatch alarms on the GuardrailContentSource dimension by using InvocationsIntervened metrics to detect recommendation discrepancy threshold violations.

C. Set up Amazon SageMaker Clarify to analyze model outputs. Publish fairness metrics to Amazon CloudWatch. Create CloudWatch composite alarms that combine SageMaker Clarify bias metrics with Amazon Bedrock latency metrics.

D. Create an Amazon Bedrock model evaluation job to compare fairness between the two prompt variants.Enable model invocation logging in Amazon CloudWatch. Set up CloudWatch alarms for InvocationsIntervened metrics with a dimension for each demographic group.

Answer: B
Explanation:
Option B best satisfies the requirements with the least custom development effort by using native Amazon Bedrock capabilities for prompt experimentation, traffic management, fairness monitoring, and alerting.
Amazon Bedrock Prompt Management allows teams to define and manage multiple prompt variants without code changes, making it ideal for comparing recommendation strategies across demographic groups.

Amazon Bedrock Flows enables controlled traffic allocation between prompt variants, which supports real- time A/B testing. This allows the company to collect live fairness metrics under production conditions instead of relying on offline analysis. Because Flows are fully managed, they eliminate the need for custom routing or experimentation frameworks.
Amazon Bedrock guardrails provide built-in monitoring and intervention mechanisms. When configured for fairness-related checks, guardrails can detect policy violations and surface metrics such as InvocationsIntervened, which indicate when outputs are modified or blocked due to rule enforcement. These metrics integrate directly with Amazon CloudWatch, enabling real-time dashboards and threshold-based alarms. Setting an alarm at a 15% discrepancy threshold satisfies the alerting requirement with minimal configuration.
Weekly reporting can be generated from CloudWatch metrics using scheduled exports or dashboards without building custom analytics pipelines. Option A requires significant custom post-processing logic. Option C introduces an additional service with higher operational overhead and is not optimized for real-time monitoring. Option D focuses on offline evaluation jobs and does not provide continuous real-time fairness monitoring.
Therefore, Option B provides the most AWS-native, scalable, and low-effort solution for fairness evaluation and monitoring.

NO.96 Company configures a landing zone in AWS Control Tower. The company handles sensitive data that must remain within the European Union. The company must use only the eu-central-1 Region. The company uses Service Control Policies (SCPs) to enforce data residency policies. GenAI developers at the company are assigned IAM roles that have full permissions for Amazon Bedrock. The company must ensure that GenAI developers can use the Amazon Nova Pro model through Amazon Bedrock only by using cross-Region inference (CRI) and only in eu-central-1. The company enables model access for the GenAI developer IAM roles in Amazon Bedrock. However, when a GenAI developer attempts to invoke the model through the Amazon Bedrock Chat/Text playground, the GenAI developer receives the following error:
User arn:aws:sts:123456789012:assumed-role/AssumedDevRole/DevUserName
Action: bedrock:InvokeModelWithResponseStream
On resource(s): arn:aws:bedrock:eu-west-3::foundation-model/amazon.nova-pro-v1:0 Context: a service control policy explicitly denies the action The company needs a solution to resolve the error. The solution must retain the company's existing governance controls and must provide precise access control. The solution must comply with the company's existing data residency policies.
Which combination of solutions will meet these requirements? (Select TWO.)

A. Add an AdministratorAccess policy to the GenAI developer IAM role

B. Extend the existing SCPs to enable CRI for the eu.amazon.nova-pro-v1:0 inference profile

C. Enable Amazon Bedrock model access for Amazon Nova Pro in the eu-west-3 Region

D. Validate that the GenAI developer IAM roles have permissions to invoke Amazon Nova Pro through the eu.amazon.nova-pro-v1:0 inference profile on all European Union AWS Regions that can serve the model

E. Extend the existing SCP to enable CRI for the eu-* inference profile

Answer: B E

Explanation:
This error occurs because SCPs override IAM permissions, and the SCP currently blocks Bedrock inference calls that resolve to eu-west-3, even though the company intends to use cross-Region

inference (CRI) from eu- central-1.
Amazon Nova Pro is not hosted in eu-central-1, so when invoked, Amazon Bedrock transparently routes the request to a supporting Region (such as eu-west-3) through CRI inference profiles. However, SCPs that restrict Regions or specific Bedrock resources will block this routing unless explicitly allowed.
Option B is required because the SCP must explicitly allow the eu.amazon.nova-pro-v1:0 inference profile, which is the Bedrock abstraction that enables CRI while preserving data residency guarantees. Without this, Bedrock cannot legally route the request.
Option E is also required to allow EU-scoped inference profiles rather than individual Regions. This preserves precise governance while allowing Bedrock-managed CRI routing within the EU boundary, ensuring no data leaves Europe.
Option A violates least-privilege and does not override SCPs. Option C breaks data residency by enabling direct eu-west-3 access. Option D does not resolve the SCP denial.
Therefore, Options B and E are the only combination that resolves the error while preserving governance and EU-only data residency.

NO.97 A company is creating a generative AI (GenAI) application that uses Amazon Bedrock foundation models (FMs). The application must use Microsoft Entra ID to authenticate. All FM API calls must stay on private network paths. Access to the application must be limited by department to specific model families. The company also needs a comprehensive audit trail of model interactions. Which solution will meet these requirements?

A. Configure SAML federation between Microsoft Entra ID and AWS Identity and Access Management.
Create department-specific IAM roles that allow only the required ModelId values. Create AWS PrivateLink interface VPC endpoints for Amazon Bedrock runtime services. Enable AWS CloudTrail to capture Amazon Bedrock API calls. Configure Amazon Bedrock model invocation logging to record detailed model interactions.

B. Create a SAML identity provider (IdP) in IAM to authenticate by using Microsoft Entra ID. Use IAM permissions boundaries to limit department roles' access to specific model families. Configure public Amazon Bedrock API endpoints with VPC routing to maintain private network connectivity. Set up CloudTrail with Amazon S3 Lifecycle rules to manage audit logs of model interactions.

C. Create an identity provider (IdP) connection in IAM to authenticate by using Microsoft Entra ID. Assign department permission sets to control access to specific model families. Deploy AWS Lambda functions in private subnets with a NAT gateway for egress to Amazon Bedrock public endpoints. Enable CloudWatch Logs to capture model interactions for auditing purposes.

D. Configure OpenID Connect (OIDC) federation between Microsoft Entra ID and IAM. Use attribute-based access control to map department attributes to specific model access permissions. Apply SCP policies to restrict access to Amazon Bedrock FM families based on department. Use Microsoft Entra ID's built-in logging capabilities to maintain an audit trail of model interactions.

Answer: A

NO.98 A financial services company is deploying a generative AI (GenAI) application that uses Amazon Bedrock to assist customer service representatives to provide personalized investment advice to customers. The company must implement a comprehensive governance solution that follows responsible AI practices and meets regulatory requirements.

The solution must detect and prevent hallucinations in recommendations. The solution must have safety controls for customer interactions. The solution must also monitor model behavior drift in real time and maintain audit trails of all prompt-response pairs for regulatory review. The company must deploy the solution within 60 days. The solution must integrate with the company's existing compliance dashboard and respond to customers within 200 ms.
Which solution will meet these requirements with the LEAST operational overhead?

A. Configure Amazon Bedrock guardrails to apply custom content filters and toxicity detection. Use Amazon Bedrock Model Evaluation to detect hallucinations. Store prompt-response pairs in Amazon DynamoDB to capture audit trails and set a TTL. Integrate Amazon CloudWatch custom metrics with the existing compliance dashboard.

B. Deploy Amazon Bedrock and use AWS PrivateLink to access the application securely. Use AWS Lambda functions to implement custom prompt validation. Store prompt-response pairs in an Amazon S3 bucket and configure S3 Lifecycle policies. Create custom Amazon CloudWatch dashboards to monitor model performance metrics.

C. Use Amazon Bedrock Agents and Amazon Bedrock Knowledge Bases to ground responses. Use Amazon Bedrock Guardrails to enforce content safety. Use Amazon OpenSearch Service to store and index prompt-response pairs. Integrate OpenSearch Service with Amazon QuickSight to create compliance reports and to detect model behavior drift.

D. Use Amazon SageMaker Model Monitor to detect model behavior drift. Use AWS WAF to filter content. Store customer interactions in an encrypted Amazon RDS database. Use Amazon API Gateway to create custom HTTP APIs to integrate with the compliance dashboard.

Answer: A

Explanation:

Option A is the correct solution because it uses native Amazon Bedrock governance and evaluation capabilities to meet regulatory, performance, and deployment timeline requirements with the least operational overhead.

Amazon Bedrock guardrails provide built-in safety controls that enforce responsible AI policies directly during inference. Custom content filters and toxicity detection protect customer interactions and prevent disallowed investment guidance patterns without requiring custom application logic. Guardrails operate inline and are optimized for low latency, which helps meet the strict 200 ms response-time requirement.

Hallucination detection is addressed through Amazon Bedrock Model Evaluation, which supports automated evaluation at scale using LLM-as-a-judge techniques. This enables the company to detect factual inaccuracies and policy violations systematically, without building custom evaluation pipelines or requiring extensive human review. Evaluation outputs can be surfaced as metrics.

Storing all prompt-response pairs in Amazon DynamoDB provides a low-latency, highly scalable audit store that aligns with financial regulatory requirements. Using TTL enforces data retention policies automatically, reducing compliance risk and storage overhead.

Amazon CloudWatch custom metrics integrate seamlessly with existing compliance dashboards, allowing near-real-time monitoring of safety interventions, hallucination rates, and drift indicators. CloudWatch anomaly detection can be applied to these metrics to surface behavior changes quickly.

Option B relies on custom Lambda logic and S3-based auditing, increasing latency and operational complexity. Option C introduces additional services that increase setup time and may exceed the 60-day deployment window. Option D uses non-Bedrock-native monitoring and adds unnecessary infrastructure layers.

Therefore, Option A provides the most complete, compliant, and low-overhead governance solution for a regulated GenAI financial services application.

NO.99 A company is using Amazon Bedrock to build a customer-facing AI assistant that handles sensitive customer inquiries. The company must use defense-in-depth safety controls to block sophisticated prompt injection attacks. The company must keep audit logs of all safety interventions. The AI assistant must have cross- Region failover capabilities.
Which solution will meet these requirements?

A. Configure Amazon Bedrock guardrails with content filters set to high to protect against prompt injection attacks. Use a guardrail profile to implement cross-Region guardrail inference. Use Amazon CloudWatch Logs with custom metrics to capture detailed guardrail intervention events.

B. Configure Amazon Bedrock guardrails with content filters set to high. Use AWS WAF to block suspicious inputs. Use AWS CloudTrail to log API calls.

C. Deploy Amazon Comprehend custom classifiers to detect prompt injection attacks. Use Amazon API Gateway request validation. Use CloudWatch Logs to capture intervention events.

D. Configure Amazon Bedrock guardrails with custom content filters and word filters set to high.Configure cross-Region guardrail replication for failover. Store logs in AWS CloudTrail for compliance auditing.

Answer: A

Explanation:
Option A provides the most complete, AWS-native defense-in-depth solution for protecting against prompt injection attacks while meeting audit and resiliency requirements. Amazon Bedrock guardrails are designed specifically to enforce safety policies on both user inputs and model outputs, including protections against prompt injection and jailbreak attempts.
Setting content filters to high increases sensitivity to malicious or manipulative inputs. Guardrail profiles allow the same guardrail configuration to be applied consistently across multiple Regions, enabling cross- Region inference and failover without configuration drift. This directly satisfies the requirement for regional resilience.
Amazon CloudWatch Logs captures detailed guardrail intervention events, including when content is blocked, modified, or flagged. Custom metrics derived from these logs enable fine-grained auditing, alerting, and reporting on safety enforcement actions. This provides a more detailed audit trail of safety interventions than API-level logs alone.
Option B adds WAF protection but lacks detailed guardrail intervention logging. Option C introduces additional services and custom logic that increase complexity and may miss model-specific injection patterns.
Option D references replication concepts that are not aligned with Bedrock guardrail operational models and relies on word filters, which are insufficient against sophisticated prompt injection techniques.
Therefore, Option A best meets the requirements for layered protection, auditability, and cross-Region resilience using managed Amazon Bedrock safety controls.

NO.100 A company is using Amazon Bedrock to develop an AI-powered application that uses a foundation model that supports cross-Region inference and provisioned throughput. The application must serve users in Europe and North America with consistently low latency. The application must comply with data residency regulations that require European user data to remain within Europe-

based AWS Regions.
During testing, the application experiences service degradation when Regional traffic spikes reach service quotas. The company needs a solution that maintains application resilience and minimizes operational complexity.
Which solution will meet these requirements?

A. Deploy separate Amazon Bedrock instances in North American and European Regions. Use a custom routing layer that directs traffic based on user location. Configure Amazon CloudWatch alarms to monitor Regional service usage. Use Amazon SNS to send email alerts to the company when usage approaches specified thresholds.

B. Use Amazon Bedrock cross-Region inference profiles by specifying geographical codes in profile IDs when the application calls the InvokeModel API. Configure separate Amazon API Gateway HTTP APIs to direct European and North American users to the appropriate Regional endpoints.

C. Deploy a multi-Region Amazon API Gateway HTTP API and AWS Lambda functions that implement retry logic to handle throttling. Configure the Lambda functions to call the foundation model in the nearest secondary Region when the application reaches service quotas in the primary Region. Use intelligent routing to ensure compliance with data residency requirements.

D. Configure provisioned throughput for Amazon Bedrock in multiple Regions. Implement failover logic in the application code to switch between Regions when throttling occurs. Use AWS Global Accelerator to route traffic to the appropriate endpoints based on user location.

Answer: B

Explanation:
Option B best meets the latency, resilience, and data residency requirements while keeping operational complexity low by using built-in Amazon Bedrock cross-Region inference behavior through inference profiles. Cross-Region inference profiles are designed to provide higher availability and better traffic absorption when a single Region experiences throttling, transient capacity constraints, or quota-related degradation. By selecting the appropriate geography-scoped inference profile (for example, a Europe-scoped profile for European users and a North America-scoped profile for North American users), the application can keep inference traffic within the required geographic boundary. This directly supports EU data residency needs because European requests can be served only by Europe-based Regions while still benefiting from multi-Region resilience inside Europe.
The question also highlights degradation when Regional traffic spikes hit quotas. Cross-Region inference profiles help mitigate these conditions by allowing Bedrock to serve requests from another Region within the same geography, improving continuity during spikes without requiring the company to implement custom retry-and-failover logic across Regions. This reduces development and operational burden compared to building and maintaining a bespoke routing and fallback system.
Using separate Amazon API Gateway HTTP APIs to direct European and North American users to the correct endpoints simplifies request routing and provides a clean boundary for compliance controls, logging, and monitoring. It also allows each geography to scale independently and maintain consistently low latency by keeping users close to the entry point and the Bedrock geography they must use.
Option A requires custom routing and manual operational monitoring and does not inherently solve quota- driven degradation. Option C adds significant complexity by embedding throttling retries and cross-Region selection logic in Lambda while still needing careful controls to prevent cross-border routing mistakes. Option D introduces the highest operational complexity and can inadvertently

violate residency if failover crosses geographies unless additional safeguards are implemented.

NO.101 A company is building a video analysis platform on AWS. The platform will analyze a large video archive by using Amazon Rekognition and Amazon Bedrock. The platform must comply with predefined privacy standards. The platform must also use secure model I/O, control foundation model (FM) access patterns, and provide an audit of who accessed what and when.
Which solution will meet these requirements?

A. Configure VPC endpoints for Amazon Bedrock model API calls. Implement Amazon Bedrock guardrails to filter harmful or unauthorized content in prompts and responses. Use Amazon Bedrock trace events to track all agent and model invocations for auditing purposes. Export the traces to Amazon CloudWatch Logs as an audit record of model usage. Store all prompts and outputs in Amazon S3 with server-side encryption with AWS KMS keys (SSE-KMS).

B. Define access control by using IAM with attribute-based access control (ABAC) to map departments to specific permissions. Configure VPC endpoints for Amazon Bedrock model API calls. Use IAM condition keys to enforce specific GuardrailIdentifier and ModelId values. Configure AWS CloudTrail to capture management and data events for S3 objects and KMS key usage activities. Enable S3 server access logging to record detailed file-level interactions with the video archives. Send all CloudTrail logs to AWS CloudTrail Lake. Set up Amazon CloudWatch alarms to detect and alert on unexpected activity from Amazon Bedrock, Amazon Rekognition, and AWS KMS.

C. Restrict access to services by using VPC endpoint policies. Use AWS Config to track resource changes and compliance with security rules. Use server-side encryption with AWS KMS keys (SSE-KMS) to encrypt data at rest. Store the model's I/O in separate Amazon S3 buckets. Enable S3 server access logging to track file-level interactions.

D. Configure AWS CloudTrail Insights to analyze API call patterns across accounts and detect anomalous activity in Amazon Bedrock, Amazon Rekognition, Amazon S3, and AWS KMS. Deploy Amazon Macie to scan and classify the video archive. Use server-side encryption with AWS KMS keys (SSE- KMS) to encrypt all stored data. Configure CloudTrail to capture KMS API usage events for audit purposes. Configure Amazon EventBridge rules to process CloudTrail Insights anomalies and Macie findings. Use CloudWatch alarms to trigger automated notifications and security responses when potential security issues are detected.

Answer: B

Explanation:

Option B is the correct solution because it delivers end-to-end governance, security, and auditability across Amazon Bedrock, Amazon Rekognition, and the underlying data layer while meeting strict privacy and compliance requirements.

Using IAM attribute-based access control (ABAC) allows the company to control access to foundation models and data based on department, role, or workload attributes rather than static permissions. This is critical for controlling FM access patterns at scale. Enforcing specific ModelId and GuardrailIdentifier values with IAM condition keys ensures that only approved models and guardrails are used, which directly supports secure model I/O and governance requirements.

Configuring VPC endpoints for Amazon Bedrock ensures that all model invocations remain on private AWS network paths, reducing data exfiltration risk and supporting privacy standards. AWS CloudTrail captures both management and data events, providing a definitive audit trail of who accessed which resources and when. Sending logs to CloudTrail Lake enables centralized, long-term, queryable auditing across services.

Amazon S3 server access logging adds file-level visibility into video archive access, which is essential for compliance and forensic analysis. Amazon CloudWatch alarms provide near real-time detection of anomalous or unauthorized activity across Amazon Bedrock, Amazon Rekognition, and AWS KMS.
Option A focuses primarily on model-level tracing but lacks comprehensive IAM governance and S3 access auditing. Option C provides partial controls but lacks identity-aware auditing and model governance. Option D focuses on anomaly detection and classification but does not explicitly control FM access patterns.
Therefore, Option B best satisfies all stated requirements in a unified, auditable, and security-first architecture.

NO.102 A GenAI developer is evaluating Amazon Bedrock foundation models (FMs) to enhance a Europe-based company's internal business application. The company has a multi-account landing zone in AWS Control Tower. The company uses Service Control Policies (SCPs) to allow its accounts to use only the eu-north-1 and eu-west-1 Regions. All customer data must remain in private networks within the approved AWS Regions.
The GenAI developer selects an FM based on analysis and testing and hosts the model in the eu-central-1 Region and the eu-west-3 Region. The GenAI developer must enable access to the FM for the company's employees. The GenAI developer must ensure that requests to the FM are private and remain within the same Regions as the FM.
Which solution will meet these requirements?

A. Deploy an AWS Lambda function that is exposed by a private Amazon API Gateway REST API to a VPC in eu-north-1. Create a VPC endpoint for the selected FM in eu-central-1 and eu-west-3. Extend existing SCPs to allow employees to use the FM. Integrate the REST API with the business application.

B. Deploy the FM on Amazon EC2 instances in eu-north-1. Deploy a private Amazon API Gateway REST API in front of the EC2 instances. Configure an Amazon Bedrock VPC endpoint. Integrate the REST API with the business application.

C. Configure the FM to use cross-Region inference through a Europe-scoped endpoint. Configure an Amazon Bedrock VPC endpoint. Extend existing SCPs to allow employees to use the FM through inference profiles in Europe-based Regions where the FM is available. Use an inference profile to integrate Amazon Bedrock with the business application.

D. Deploy the FM in Amazon SageMaker in eu-north-1. Configure a SageMaker VPC endpoint. Extend existing SCPs to allow employees to use the SageMaker endpoint. Integrate the FM in SageMaker with the business application.

Answer: C
Explanation:
Option C is the correct solution because it uses Amazon Bedrock cross-Region inference profiles, which are explicitly designed to support regional data residency, private connectivity, and resilience with minimal operational overhead.
By using a Europe-scoped inference profile, the application ensures that all inference requests are routed only within European Regions where the FM is deployed, such as eu-central-1 and eu-west-3. This satisfies data residency requirements while still providing resilience and load distribution across Regions.
Configuring an Amazon Bedrock VPC endpoint ensures that all traffic remains on the AWS private network.
No public endpoints are used, which aligns with the company's private networking requirements.

Extending existing SCPs to allow inference profile usage ensures that employees can access the FM only in approved Regions, maintaining governance across the Control Tower environment.
Options A and B introduce unnecessary custom routing layers and EC2 management. Option D moves away from Amazon Bedrock entirely and increases operational complexity.
Therefore, Option C is the only solution that satisfies private access, regional confinement, governance controls, and low operational overhead.

NO.103 An ecommerce company is using Amazon Bedrock to build a generative AI (GenAI) application. The application uses AWS Step Functions to orchestrate a multi-agent workflow to produce detailed product descriptions. The workflow consists of three sequential states: a description generator, a technical specifications validator, and a brand voice consistency checker. Each state produces intermediate reasoning traces and outputs that are passed to the next state. The application uses an Amazon S3 bucket for process storage and to store outputs.
During testing, the company discovers that outputs between Step Functions states frequently exceed the 256 KB quota and cause workflow failures. A GenAI Developer needs to revise the application architecture to efficiently handle the Step Functions 256 KB quota and maintain workflow observability. The revised architecture must preserve the existing multi-agent reasoning and acting (ReAct) pattern.
Which solution will meet these requirements with the LEAST operational overhead?

A. Store intermediate outputs in Amazon DynamoDB. Pass only references between states. Create a Map state that retrieves the complete data from DynamoDB when required for each agent's processing step.

B. Configure an Amazon Bedrock integration to use the S3 bucket URI in the input parameters for large outputs. Use the ResultPath and ResultSelector fields to route S3 references between the agent steps while maintaining the sequential validation workflow.

C. Use AWS Lambda functions to compress outputs to less than 256 KB before each agent state. Configure each agent task to decompress outputs before processing and to compress results before passing them to the next state.

D. Configure a separate Step Functions state machine to handle each agent's processing. Use Amazon EventBridge to coordinate the execution flow between state machines. Use S3 references for the outputs as event data.

Answer: B

Explanation:
Option B is the best solution because it directly addresses the Step Functions 256 KB state payload quota by externalizing large intermediate artifacts to Amazon S3 and passing only lightweight references (URIs/keys) between states. This is a standard AWS pattern for workflows that produce large intermediate results, and it avoids introducing additional databases, compression logic, or cross-state-machine coordination that increases operational overhead.
In a multi-agent ReAct workflow, intermediate reasoning traces can be verbose and grow quickly as each agent produces chain-of-thought style artifacts, structured outputs, and supporting evidence. Step Functions is designed to orchestrate state transitions and pass JSON payloads, but large payloads should be stored outside the state machine and referenced by pointer values. Using Amazon S3 for intermediate outputs is operationally efficient because the application already uses S3 for storage, and S3 provides durable, low-cost storage with simple access patterns.
ResultPath and ResultSelector allow each state to store or reshape results so that only the required

reference fields (such as s3Uri, object key, metadata, trace IDs) are forwarded to subsequent states. This preserves observability because the workflow can still log trace references, correlate steps with S3 objects, and store structured metadata for debugging. It also preserves the sequential validation design, keeping the existing ReAct pattern intact while preventing failures due to oversized payloads.
Option A adds additional services and read/write patterns that increase operational complexity. Option C introduces custom compression/decompression logic that is fragile, adds latency, and complicates troubleshooting. Option D increases orchestration overhead by splitting workflows and coordinating with events, which makes debugging harder and increases failure modes.
Therefore, Option B meets the payload limit requirement while keeping the architecture simple and observable.

NO.104 A legal research company has a Retrieval Augmented Generation (RAG) application that uses Amazon Bedrock and Amazon OpenSearch Service. The application stores 768-dimensional vector embeddings for 15 million legal documents, including statutes, court rulings, and case summaries.
The company's current chunking strategy segments text into fixed-length blocks of 500 tokens. The current chunking strategy often splits contextually linked information such as legal arguments, court opinions, or statute references across separate chunks. Researchers report that generated outputs frequently omit key context or cite outdated legal information.
Recent application logs show a 40% increase in response times. The p95 latency metric exceeds 2 seconds.
The company expects storage needs for the application to grow from 90 GB to 360 GB within a year.
The company needs a solution to improve retrieval relevance and system performance at scale.
Which solution will meet these requirements?

A. Increase the embedding vector dimensionality from 768 to 4,096 without changing the existing chunking or pre-processing strategy.

B. Replace dynamic retrieval with static, pre-written summaries that are stored in Amazon S3. Use Amazon CloudFront to serve the summaries to reduce compute demand and improve predictability.

C. Update the chunking strategy to use semantic boundaries such as complete legal arguments, clauses, or sections rather than fixed token limits. Regenerate vector embeddings to align with the new chunk structure.

D. Migrate from OpenSearch Service to Amazon DynamoDB. Implement keyword-based indexes to enable faster lookups for legal concepts.

Answer: C
Explanation:
Option C directly addresses both retrieval relevance and performance scalability. Fixed token chunking breaks semantic continuity in legal texts, causing incomplete context retrieval and degraded response quality. By switching to semantic chunking-based on legal arguments, clauses, or sections-the application preserves contextual integrity, improving retrieval accuracy and reducing hallucinations.
Regenerating embeddings aligned with the new chunk structure also improves vector search efficiency, reducing unnecessary comparisons and helping control latency as the dataset scales.
Option A increases cost and latency without fixing the core issue. Option B removes dynamic reasoning, which defeats the purpose of a legal RAG system. Option D discards vector semantics entirely and is unsuitable for nuanced legal research. Therefore, Option C is the correct and scalable solution.

NO.105 A financial services company uses an AI application to process financial documents by using Amazon Bedrock. During business hours, the application handles approximately 10,000 requests each hour, which requires consistent throughput.
The company uses the CreateProvisionedModelThroughput API to purchase provisioned throughput. Amazon CloudWatch metrics show that the provisioned capacity is unused while on-demand requests are being throttled. The company finds the following code in the application:
python
response = bedrock_runtime.invoke_model(modelId="anthropic.claude-v2", body=json.dumps(payload)) The company needs the application to use the provisioned throughput and to resolve the throttling issues.
Which solution will meet these requirements?

A. Increase the number of model units (MUs) in the provisioned throughput configuration.

B. Replace the model ID parameter with the ARN of the provisioned model that the CreateProvisionedModelThroughput API returns.

C. Add exponential backoff retry logic to handle throttling exceptions during peak hours.

D. Modify the application to use the InvokeModelWithResponseStream API instead of the InvokeModel API.

Answer: B

Explanation:
Option B is correct because the application is currently invoking the base foundation model identifier, which routes traffic to the on-demand capacity pool rather than the company's purchased provisioned throughput. In Amazon Bedrock, provisioned throughput is attached to a specific provisioned resource created through the provisioned throughput APIs. To consume that reserved capacity, inference requests must target the provisioned resource identifier that represents the purchased throughput, not the generic model identifier used for on-demand inference.
The code snippet uses modelId="anthropic.claude-v2". This value selects the on-demand endpoint for that model. As a result, requests are subject to on-demand quotas and throttling behavior, while the provisioned throughput remains idle. This directly explains the CloudWatch observation: provisioned capacity metrics show unused capacity because no traffic is being directed to the provisioned resource, and the on-demand path is throttling because it is exceeding the applicable on-demand limits during peak volume.
Replacing the modelId value with the provisioned throughput ARN returned by the CreateProvisionedModelThroughput workflow ensures the runtime invocation is routed to the reserved capacity. Once traffic is directed correctly, the purchased model units provide the consistent throughput required for predictable performance during business hours, which is exactly why provisioned throughput is used.
Option A could increase capacity, but it does not fix the core issue that the application is not using the provisioned resource at all. Option C can reduce the impact of throttling temporarily, but it adds latency and does not guarantee consistent throughput; it also still wastes the provisioned capacity. Option D changes the response delivery mechanism, but throttling is a capacity routing and quota issue, not a streaming API issue.

NO.106 A company is developing a generative AI (GenAI) application that uses Amazon Bedrock foundation models.
The application has several custom tool integrations. The application has experienced unexpected

token consumption surges despite consistent user traffic.
The company needs a solution that uses Amazon Bedrock model invocation logging to monitor InputTokenCount and OutputTokenCount metrics. The solution must detect unusual patterns in tool usage and identify which specific tool integrations cause abnormal token consumption. The solution must also automatically adjust thresholds as traffic patterns change.
Which solution will meet these requirements?

A. Use Amazon CloudWatch Logs to capture model invocation logs. Create CloudWatch dashboards for token metrics. Configure static CloudWatch alarms with fixed thresholds for each tool integration.

B. Store model invocation logs in Amazon S3. Use AWS Glue and Amazon Athena to analyze token usage trends.

C. Use Amazon CloudWatch Logs to capture model invocation logs. Create CloudWatch metric filters to extract tool-specific invocation patterns. Apply CloudWatch anomaly detection alarms that automatically adjust baselines for each tool's token metrics.

D. Store model invocation logs in an Amazon S3 bucket. Use AWS Lambda to process logs in real time.Manually update CloudWatch alarm thresholds based on trends identified by the Lambda function.

Answer: C
Explanation:
Option C best meets the requirements by combining native Amazon Bedrock logging with adaptive monitoring and minimal operational overhead. Amazon Bedrock model invocation logging can be sent directly to CloudWatch Logs, where detailed fields such as InputTokenCount, OutputTokenCount, and tool invocation metadata are captured for each request.
CloudWatch metric filters allow extraction of structured metrics from logs, including tool-specific token consumption patterns. By defining filters per tool integration, the company can isolate which tools are responsible for increased token usage without building custom log-processing pipelines.
CloudWatch anomaly detection provides automatic baseline modeling and dynamic thresholds based on historical traffic patterns. Unlike static alarms, anomaly detection adapts as usage evolves, making it ideal for applications with changing workloads or seasonal usage patterns. This directly satisfies the requirement to automatically adjust thresholds as traffic patterns change.
When abnormal token consumption occurs, anomaly detection alarms trigger immediately, enabling rapid investigation and remediation. Because this solution uses fully managed AWS services without custom analytics jobs or manual threshold tuning, it significantly reduces operational effort.
Option A fails to adapt to changing patterns. Option B introduces batch analysis and delayed insights. Option D requires manual intervention and custom code, increasing maintenance burden.
Therefore, Option C provides the most scalable, adaptive, and low-maintenance solution for monitoring and controlling token consumption in Amazon Bedrock-based applications.

NO.107 A company is developing a generative AI (GenAI) application that analyzes customer service calls in real time and generates suggested responses for human customer service agents. The application must process
500,000 concurrent calls during peak hours with less than 200 ms end-to-end latency for each suggestion. The company uses existing architecture to transcribe customer call audio streams. The application must not exceed a predefined monthly compute budget and must maintain auto scaling capabilities.
Which solution will meet these requirements?

A. Deploy a large, complex reasoning model on Amazon Bedrock. Purchase provisioned throughput and optimize for batch processing.

B. Deploy a low-latency, real-time optimized model on Amazon Bedrock. Purchase provisioned throughput and set up automatic scaling policies.

C. Deploy a large language model (LLM) on an Amazon SageMaker real-time endpoint that uses dedicated GPU instances.

D. Deploy a mid-sized language model on an Amazon SageMaker serverless endpoint that is optimized for batch processing.

Answer: B

Explanation:

Option B is the correct solution because it aligns with AWS guidance for building high-throughput, ultra-low- latency GenAI applications while maintaining predictable costs and automatic scaling.

Amazon Bedrock provides access to foundation models that are specifically optimized for real-time inference use cases, including conversational and recommendation-style workloads that require responses within milliseconds.

Low-latency models in Amazon Bedrock are designed to handle very high request rates with minimal per- request overhead. Purchasing provisioned throughput ensures that sufficient model capacity is reserved to handle peak loads, eliminating cold starts and reducing request queuing during traffic surges. This is critical when supporting up to 500,000 concurrent calls with strict latency requirements.

Automatic scaling policies allow the application to dynamically adjust capacity based on demand, ensuring cost efficiency during off-peak hours while maintaining performance during peak usage. This directly supports the requirement to stay within a predefined monthly compute budget.

Option A fails because batch processing and complex reasoning models introduce higher latency and are not suitable for real-time suggestions. Option C introduces significantly higher operational and cost overhead due to dedicated GPU instances and manual scaling responsibilities. Option D is optimized for batch workloads and cannot meet the sub-200 ms latency requirement.

Therefore, Option B provides the best balance of performance, scalability, cost control, and operational simplicity using AWS-native GenAI services.

NO.108 A company is using Amazon Bedrock to develop an AI-powered application that uses a foundation model (FM) that supports cross-Region inference and provisioned throughput. The application must serve users in Europe and North America with consistently low latency. The application must comply with data residency regulations that require European user data to remain within Europe-based AWS Regions.

During testing, the application experiences service degradation when Regional traffic spikes reach service quotas. The company needs a solution that maintains application resilience and minimizes operational complexity.

Which solution will meet these requirements?

A. Deploy separate Amazon Bedrock instances in North American and European Regions. Use a custom routing layer that directs traffic based on user location. Configure Amazon CloudWatch alarms to monitor Regional service usage. Use Amazon SNS to send email alerts when usage approaches thresholds.

B. Use Amazon Bedrock cross-Region inference profiles by specifying geographical codes in profile IDs when calling the InvokeModel API. Configure separate Amazon API Gateway HTTP APIs to direct

European and North American users to the appropriate Regional endpoints.

C. Deploy a multi-Region Amazon API Gateway HTTP API and AWS Lambda functions that implement retry logic to handle throttling. Configure the Lambda functions to call the FM in the nearest secondary Region when quotas are reached.

D. Configure provisioned throughput for Amazon Bedrock in multiple Regions. Implement failover logic in application code to switch Regions when throttling occurs. Use AWS Global Accelerator to route traffic based on user location.

Answer: B

Explanation:

Option B is the most appropriate solution because it directly uses Amazon Bedrock cross-Region inference profiles, which are designed to provide resilience and load distribution while respecting data residency boundaries. Cross-Region inference profiles allow applications to distribute inference requests across multiple Regions within a defined geographic boundary, such as Europe or North America, without requiring custom failover logic.

By specifying geographical codes in the inference profile ID, the application ensures that European user data is processed only within Europe-based Regions, satisfying regulatory requirements. At the same time, Bedrock automatically routes requests to healthy Regions within that geography when traffic spikes or service quotas are reached, improving availability and maintaining low latency.

Using separate Amazon API Gateway HTTP APIs for Europe and North America provides a clean, simple routing layer that directs users to the appropriate regional inference profile. This avoids complex custom routing or retry logic in application code and minimizes operational overhead.

Option A relies on custom routing and manual monitoring, which increases complexity and does not provide automatic resilience. Option C introduces custom retry and fallback logic that risks violating data residency requirements if misconfigured. Option D requires significant application-level failover logic and adds operational burden with Global Accelerator configuration.

Therefore, Option B best meets the requirements for low latency, data residency compliance, resilience during traffic spikes, and minimal operational complexity.

NO.109 A financial services company uses multiple foundation models (FMs) through Amazon Bedrock for its generative AI (GenAI) applications. To comply with a new regulation for GenAI use with sensitive financial data, the company needs a token management solution.

The token management solution must proactively alert when applications approach model-specific token limits. The solution must also process more than 5,000 requests each minute and maintain token usage metrics to allocate costs across business units.

Which solution will meet these requirements?

A. Develop model-specific tokenizers in an AWS Lambda function. Configure the Lambda function to estimate token usage before sending requests to Amazon Bedrock. Configure the Lambda function to publish metrics to Amazon CloudWatch and trigger alarms when requests approach thresholds. Store detailed token usage in Amazon DynamoDB to report costs.

B. Implement Amazon Bedrock Guardrails with token quota policies. Capture metrics on rejected requests.

Configure Amazon EventBridge rules to trigger notifications based on Amazon Bedrock Guardrails metrics. Use Amazon CloudWatch dashboards to visualize token usage trends across models.

C. Deploy an Amazon SQS dead-letter queue for failed requests. Configure an AWS Lambda function to analyze token-related failures. Use Amazon CloudWatch Logs Insights to generate reports on token

usage patterns based on error logs from Amazon Bedrock API responses.

D. Use Amazon API Gateway to create a proxy for all Amazon Bedrock API calls. Configure request throttling based on custom usage plans with predefined token quotas. Configure API Gateway to reject requests that will exceed token limits.

Answer: A

Explanation:

Option A is the correct solution because it provides proactive, model-aware token management with fine- grained visibility and alerting, which is required for regulated financial workloads. Amazon Bedrock currently exposes token usage metrics after invocation, but it does not natively enforce proactive, model-specific token limits across multiple applications or business units.

By implementing model-specific tokenizers in AWS Lambda, the company can estimate input and output token usage before sending requests to Amazon Bedrock. This enables early detection of requests that are approaching or exceeding model limits and allows the application to block, truncate, or reroute requests proactively rather than reacting to failures.

Publishing token usage metrics to Amazon CloudWatch enables real-time monitoring and alerting at scale, easily supporting more than 5,000 requests per minute. Storing detailed token usage data in Amazon DynamoDB allows the company to attribute usage and costs to specific applications, teams, or business units-an essential requirement for regulatory reporting and internal chargeback.

Option B is incorrect because Amazon Bedrock Guardrails do not currently provide token quota enforcement or proactive token alerts. Option C is reactive and only analyzes failures after they occur. Option D throttles requests but cannot enforce token-based limits or provide per-model cost attribution.

Therefore, Option A best satisfies proactive alerting, scalability, compliance reporting, and cost allocation requirements with acceptable operational effort.